

Living Worlds Runtime — Complete Specification

A reconstruction manual. Everything needed to reimplement this runtime in another language, and to work on the theory behind it, without opening a single source file.

Version of the system described: **runtime 1.21.0** (`lwr.contracts.RUNTIME_VERSION`). Written against the implementation in `lwr/` — 49 modules, six layers, no third-party dependencies.

This document is a **specification**, not a tour. Every formula, constant, threshold, ordering rule and data shape stated here is the one the reference implementation uses. Where a number was chosen rather than derived, that is said. Where behaviour is a known weakness, that is said too, in the place where a reimplementer would otherwise have to discover it.

Related documents in this repository, and what each is for:

Document	Question it answers
<code>README.md</code>	What is this, and what has been measured?
<code>docs/CONCEPT.md</code>	Why is each mechanism like that? — the essay
<code>docs/SPECIFICATION.md</code> (this)	How do I rebuild it? — the manual
<code>docs/CALIBRATION.md</code>	Where does each constant come from?
<code>docs/ARCHITECTURE.md</code>	What may depend on what?
<code>docs/VALUES.md</code>	Every value that moves, and what reads it
<code>docs/SETTINGS.md</code>	Every configuration switch
<code>port/README.md</code>	How the C++ port was done, and what it cost
<code>conformance/README.md</code>	The fixtures a second implementation must reproduce

Table of contents

Part I — Theory

1. [The problem this exists to solve](#)
2. [Seven design commitments](#)
3. [Theoretical lineage, mechanism by mechanism](#)

Part II — Foundations

4. [Numeric types and helpers](#)
5. [Determinism: the seed algebra](#)
6. [Ontology: propositions, keys, contradiction, relevance](#)
7. [Events, observations, claims](#)
8. [Agent state](#)
9. [World state and the world-pack format](#)

Part III — The two pipelines

10. The event pipeline (`process_event`)
11. The tick pipeline (`advance_time`)
12. The turn pipeline (a player says something)

Part IV — Engines, in full

13. Belief
14. Revision: unwinding a liar
15. Memory
16. Affect
17. Perception
18. Interpretation
19. Distortion
20. Diffusion
21. Topology: where information stops
22. Medium and notes
23. Climate
24. Contagion
25. Relationship, reputation, identity
26. Standing
27. Appearance
28. Sociolinguistics: the style vector
29. Policy: choosing an action
30. Agency, allies and leverage
31. Factions and the Director
32. Routines
33. Pursuit
34. Common knowledge
35. Promises

Part V — Speech

36. The speech pipeline
37. Commitment and deception
38. Validator and grounding
39. Realizers, and where a language model may sit

Part VI — System properties

40. Persistence, snapshots, replay
41. Optional layers and the tuning surface
42. Scale: cost, bounds and level of detail
43. Integration surface

Part VII — Building and proving it

- 44. Reimplementation order
- 45. Conformance and verification
- 46. Traps: bugs that were real
- 47. Known weaknesses and open problems

Appendices

- A. Complete default-parameter table
 - B. Reason-code catalogue
 - C. Glossary
 - D. Bibliography
-
-

Part I — Theory

1. The problem this exists to solve

1.1 One shared brain

In almost every shipped game, what NPCs know is a set of global booleans. A quest flag flips and the whole cast knows simultaneously. There is no such thing as *finding out*, no such thing as *not having heard yet*, no such thing as *having heard a wrong version*. The knowledge state of the world is one variable and every character reads it.

This is not a failure of writing. It is a failure of representation. No quantity of extra dialogue fixes it, because every branch of that dialogue is still keyed on the same global flag.

1.2 Why a language model does not fix it

Putting a large language model behind each NPC makes characters *articulate*. It does not give them *separate epistemic states*. The model knows whatever is in its prompt; the prompt is assembled from world state; the world state is the same global flag set. Every character is fluent about exactly the same facts at exactly the same instant.

Worse, it introduces three new problems a shipped title cannot absorb:

1. **Non-determinism.** A bug is not reproducible, so it is not reportable, so it is not fixable. A save does not restore the world it was taken from.
2. **Unbounded assertion.** A model can state anything. A character can invent a person, a place, a number, an event. There is no representation in which that invention is *wrong* — it is just text.
3. **No provenance.** "Why does this character believe that?" has no answer, because there is no belief, only a completion.

1.3 The reframing

The problem is not how characters *talk*. It is that the world they live in does not *know* anything and does not *change*.

So the object of the design is not dialogue. It is:

a **per-character subjective knowledge state**, with **provenance**, that moves between characters through **modelled social channels**, **degrades** on the way, **decays** in storage, and can be **revised** when a source is discredited — deterministically, so a save reproduces and a bug is reportable.

Dialogue then becomes a *read* of that state, plus a *write* back into it (a spoken claim is a real event). Text generation becomes optional presentation.

1.4 The consequence worth having

Three characters in three rooms, asked separately, give the same wrong answer, because it came from the one source all three listen to. **Nobody authored that agreement.** A player who exposes that source watches every belief resting on it recompute across the block. **Nobody authored that either.**

That is the whole product: side quests have a *reason* to exist, because somebody knows something somebody else needs, and the reason is structural rather than scripted.

2. Seven design commitments

These are the invariants a reimplementation must preserve. Everything else is negotiable; these are not, because each one is load-bearing for a claim the system makes.

C1 — The runtime decides content; a text layer only phrases it

And by default it writes every sentence. `semantic_release` is "controlled" : every released line is the pack's own authored text, and a provider is not called at all. The guarantee is then structural rather than checked — the player and the room cannot be told different things, because the sentence the player reads is the one written for exactly this claim.

The mode once exempted plans that asserted nothing, on the reasoning that a line carrying no commitment cannot contradict one. It can. A plan with no facts does not make a model's answer factually empty: asked for a greeting, a provider is free to write *"He hides behind where drinks are served"* — an indirect description of a protected place — and it was released with `ACCEPT` because the turn had been classified as carrying no fact. **A guarantee that depends on the provider staying inside its brief is not a guarantee.**

Three further rules make "controlled" mean what it says, and each closes a way the sentence and the commitment came apart on the runtime's own side:

1. **Nothing is committed that the answer cannot say.** Selection picks up to two propositions; a pack authors one phrasing per *topic*, wording one thing — whether the topic's own query holds. A character who knew both *"the clinic was shut last night"* and *"the clinic is open today"* said the first and taught the room both. Moves with no authored wording are dropped from the commitment (`commitment.unsayable_dropped`) rather than propagated silently.
2. **Style chooses between wordings; it never removes a claim.** The mean-sentence-length pressure cut *"Listen carefully. Place was shut that night."* to its first sentence for an agitated speaker while the claim still propagated. Shortening is now skipped for any line carrying a fact, and the anti-repeat retry chooses between *complete* realizations (`variants()`).
3. **A rejection is two different answers.** Only `validator.secret_leak`, `validator.secret_referent` and `validator.canon_violation` are unsafe and void a commitment. A repetition is a *quality* objection: the runtime tries another complete wording and, failing that, says it again (`dialogue.reworded` / `dialogue.released_anyway`). Conflating the two let an earlier sentence — arbitrary provider prose from a previous turn — decide whether a later, fully controlled fact entered the society at all.

"provider" restores model prose for factual lines under an explicitly weaker contract. The checks that mode relies on are lexical, and three reviews found sentences that pass all of them and say the opposite of what propagates: *"The clinic was open all night, no doubt."* reads as a denial to any negation-word rule.

The runtime selects the exact propositions a character asserts, with what force, and whether they are honest. The text layer receives that **commitment** and renders it. It selects nothing.

Why: a spoken claim is a world event that moves other characters' beliefs. If the text layer chose which of several permitted facts to mention, the text layer would be deciding what information enters the society. The claim "the model only chooses wording" would be false.

C2 — No global RNG

Every stochastic draw derives its seed from (`global_seed`, `agent_id`, `event_id`, `world_time`, `module_id`, `salt`). There is no call to a process-global random source anywhere in the simulation.

Why: replay. Without it, none of determinism, save/restore, conformance fixtures, or a reportable bug exists.

C3 — Every stage writes a reason trace

Every decision emits (code, magnitude, detail) entries. These are what makes "why does this character believe/say/do that" a query rather than a guess.

C4 — The engine ships no world content

Predicates, facts, phrasings, place names, what counts as decency, what counts as restricted — all of it lives in world packs. The engine contains vocabulary, not world. A test fails the build if fictional content appears in an engine module.

C5 — Bounded state

Anything per-character that can grow has a cap and a metric: memory, provenance, the per-belief source index and its arithmetic ledger, summaries, traces, open promises, notes. The runtime is expected to survive tens of thousands of simulated days.

C6 — Layering is enforced, not described

Six layers; a module may import from its own layer or below, never above. This is checked against the real import graph, not against a document.

```
5  surfaces      cli · service · studio · webdemo · authoring · benchmark
                        golden · evidence · showcase
4  runtime      runtime · pack · sdk · state_view · inspect
3  world        agent · perception · diffusion · routine · parser · validator
                        world · snapshot · persistence · generator
2  adapters     provider · edge · metahuman · bridge · capabilities
1  engines      belief · memory · affect · relationship · social · sociolinguistics
                        agency · policy · content · distortion · dialogue · factions
0  foundation   types · statekey · determinism · events · ontology · contracts
                        actions · architecture
```

Adapters sit *below* the world deliberately: an adapter is an interface the runtime uses, not a consumer of it. Delete every adapter and the simulation still runs — which is exactly the guarantee a game needs when a model endpoint is gone.

C7 — Optional layers are byte-identical when off

Nine mechanics can be declined. "Off" does not mean "neutral-ish"; it means the arithmetic is not performed and the run reproduces the snapshot it produced before the layer existed. This is asserted by test, not promised.

3. Theoretical lineage, mechanism by mechanism

Every mechanism below names the work it comes from, what was adopted, and — more usefully — **what was deliberately not adopted**.

Mechanism	Source	Adopted	Rejected / departed from		
Belief	Bayesian evidence accumulation in log-odds	Additive log-odds; source reliability as evidence strength	Single-probability belief. Dual support (<code>support_for</code> / <code>support_against</code>) is kept so <i>ignorance</i> and <i>conflict</i> are distinguishable, which one probability cannot do		
Correlation discount	Rumour epistemology; the "independent sources" problem	Geometric attenuation per repeat from one origin, so one origin's total contribution is bounded by $1/(1-p)$	A flat repeat penalty. It attenuates but does not saturate: 200 repeats of one rumour reached $p = 0.995$		
Contradiction	Paraconsistent logic	Contradictions are <i>surfaced</i> , never collapsed. A character may hold both	Automatic belief revision on contradiction		
Memory	ACT-R base-level activation (Anderson)	$A = \ln \sum (t-t_k)^{-d}$; retrieval threshold with logistic noise; cue spreading	The textbook time unit. Minutes put a once-seen memory below threshold in 30 of them — a character forgot a murder before the player left the room. Activation is measured in hours		
Forgetting	—	Two distinct routes: decay (activation falls) and eviction (capacity)	Treating them as one. Measured over 83k simulated days, forgetting is 91% eviction		
Judgement	Schema abstraction	When episodes are evicted, a repeated pattern about one person becomes a <code>judgement</code> memory that keeps pointers to its evidence	Free-form generalisation. The pattern looked for is narrow: same predicate, same subject, ≥ 3 occasions		
Affect	ALMA / PAD (Mehrabian), OCC appraisal (Ortony, Clore & Collins), action tendencies (Frijda)	Personality $\rightarrow$ PAD baseline; appraisal $\rightarrow$ discrete emotions; emotions $\rightarrow$ PAD impulse; mood = slow integral; emotions decay fast	Unbounded impulse. Soft saturation was added: impulse is scaled by $\sqrt{1 - \text{saturation}}$	mood	$\sqrt{}$ per axis, so one event cannot slam an axis to the rail
Distortion	Allport & Postman, <i>The Psychology of Rumor</i> (1947); Bartlett, <i>Remembering</i> (1932)	Three processes — levelling , sharpening , assimilation — with levelling dominant	A coin-flip inversion model. Inversion is kept but rare (5%), because a story rarely becomes its own opposite		
Rumour termination	Daley & Kendall (1964)	Stifling : a spreader who tells someone who already knows loses interest, so saturation is an outcome of the model	A hop cap as the termination mechanism. A hop limit models nothing; it is kept only as a safety guard at 12		

Mechanism	Source	Adopted	Rejected / departed from	
Diffusion structure	Granovetter, weak ties; social network analysis	Tie strength from familiarity and liking; a bridge term for people standing in several circles	A uniform mixing model	
Knowledge topology	—	Claims are classified on independent axes (domain, locality, access, transmissibility); rank is <i>not</i> a knowledge ordering	A hierarchy. A street vendor knows more about the local gang than a corporate director does	
Media	Daft & Lengel, media richness (1986)	Three numbers per medium: fidelity (→ levelling rate), audience (→ bystanders), trust factor (→ assertion strength)	A parallel distortion mechanism. Richness lands on machinery that already exists	
Emotional contagion	Hatfield, Cacioppo & Rapson (1994)	Mood converges a little per exchange, damped by the listener's emotional stability and weighted by tie	Any effect on personality. It moves <i>mood</i> ; the baseline is untouched	
Common knowledge	Lewis (1969), Aumann (1976), Chwe, <i>Rational Ritual</i> (2001)	Public events (broadcast, or said in the open before a crowd) create <i>publics</i> ; publicity changes what can be done , not what is believed	Treating publicity as evidence. That would be the same error as treating repetition as proof	
Decision	Multi-attribute utility; Prospect Theory (Kahneman & Tversky)	Bounded group aggregation; personality/identity as multiplicative weights; Boltzmann selection	Prospect Theory <i>on by default</i> . It is a feature flag, and when on it transforms outcomes , not arbitrary terms	
Social identity	Tajfel & Turner	Salience by softmax over accessibility, comparative fit, normative fit and threat ; a neutral baseline; temporal inertia	Winner-take-all identity switching	
Register	Giles, Communication Accommodation Theory; Bell, audience design; Brown & Levinson, politeness	Convergence/divergence as a dial scaled by $\min(1, 2$	liking	)`; formality from education, setting, status, role, stress A sign flip on liking. That was a cliff at exactly zero: a stranger got full convergence, and -0.15 liking got full divergence
Reciprocity	Social exchange theory	Kept/broken commitments move trust, liking, and a reliable reputation	Ground-truth fulfilment. A promise is judged on what the promisee came to believe	
Reputation dynamics	—	Trust rises slowly ($\eta=0.10$) and falls fast ($\eta=0.40$)	Automatic healing. A standing does not decay on	

Mechanism	Source	Adopted	Rejected / departed from
			its own; letting it would let a player wait out a reputation
Faction macro-layer	<i>Blades in the Dark</i> progress clocks; heat/escalation	Clocks advance off-screen at a rate scaled by influence and resources; heat accumulates from observed violence and discharges on mobilisation	A full off-screen simulation

3.1 The one theoretical claim that is load-bearing

Origin travels with a claim, not with the mouth it came out of.

When A tells B, the event carries A's *original* origin token, not A . So a rumour relayed through five people is, evidentially, still one witness — and the correlation discount has something real to discount. Every other epistemic property in the system is downstream of this.

The same rule has an exception that is itself principled: **a lie has no evidential ancestor**. Its origin is `said_<speaker>_<time>`, because the claim genuinely begins with the speaker. Provenance records the speaker separately in both cases, which is what `discredit` matches on.

Part II — Foundations

4. Numeric types and helpers

Three semantic ranges, used consistently:

Name	Range	Used for
<code>unit_interval</code>	<code>[0, 1]</code>	intensities, probabilities, trust, competence
<code>signed_unit</code>	<code>[-1, 1]</code>	bipolar values: valence, dominance, liking, utility groups
<code>log-odds</code>	<code>(-∞, ∞)</code>	belief accumulation

```
clamp(x, lo, hi)    = lo if x < lo else hi if x > hi else x
clamp01(x)          = clamp(x, 0, 1)
clamp_signed(x)     = clamp(x, -1, 1)

sigmoid(x)          = 1 / (1 + e^(-x))          for x >= 0
                   = e^x / (1 + e^x)           for x < 0
logit(p, eps=1e-6) = ln(p' / (1 - p')), p' = clamp(p, eps, 1-eps)
```

The two-branch sigmoid is not stylistic. It is written that way to avoid overflow of `e^(-x)` for large negative `x`. A port that uses the one-line form will diverge in the tails, and the conformance fixtures compare bits.

Vec3 (PAD) — a triple `(p, a, d)` = (pleasure/valence, arousal, dominance), each in `[-1, 1]`, with `add`, `scale`, `clamped`. Serialised as `{"valence": p, "arousal": a, "dominance": d}` at **full precision** — persistence must be lossless; display code rounds separately.

5. Determinism: the seed algebra

Every stochastic decision in the simulation is a pure function of a derived seed.

```

derive_seed(global_seed, agent_id, event_id, world_time, module_id, salt="")
    key = "{global_seed}|{agent_id}|{event_id}|{world_time}|{module_id}|{salt}"
    return blake2b(key.encode("utf-8"), digest_size=16)          # 16 raw bytes

seeded_uniform(seed) = int.from_bytes(seed[:8], "big") / 2^64    # in [0, 1)

seeded_choice(seed, weights):
    total = sum(weights)
    if total <= 0: return 0
    r = seeded_uniform(seed) * total
    acc = 0
    for i, w in enumerate(weights):
        acc += w
        if r <= acc: return i
    return len(weights) - 1

```

5.1 Rules a port must follow

1. **The key is a UTF-8 string with `|` separators.** Every component is stringified with the host language's default integer/float formatting for the values actually passed. In practice all six components are strings or integers; no float ever enters a seed key.
2. **BLAKE2b with a 16-byte digest**, unkeyed, no salt/personalisation.
3. **Big-endian** interpretation of the first 8 bytes; division by 2^{64} as a float.
4. **Iteration order must be explicit.** Anything that walks a collection while drawing a seeded number must sort first. Python dicts preserve insertion order; C# `Dictionary` does not. This class of bug appears roughly once in a thousand ticks and a byte comparison finds it on the first.
5. Some call sites re-derive from a previous seed: `derive_seed(seed.hex(), ...)`. The hex form of the 16 raw bytes, lowercase, is the string used.

5.2 Where seeds are drawn

Module	module_id	Salts
perception	perception	—
interpretation	interpretation	recognise
distortion	distortion	kind, the kind name
diffusion	diffusion	fraction, speaker, listener, audience, topic, topology, distort, distort-kind, stifle
policy	policy	—
commitment	(parts)	commitment, pick{n}
agency	ally_response	caller id
director	clocktick	—
notes	notes	write, kind, intercept
pipeline (generator)	pipeline	joined decision path

6. Ontology: propositions, keys, contradiction, relevance

6.1 The proposition

A proposition is **predicate** + **named slots** + **polarity** + **qualifiers**, over a closed, versioned catalogue.

Proposition:

```
predicate    : str          # "was_at", "seal_breached"
slots        : {name: value} # values are entity ids or literals
polarity     : "+" | "-"
valid_start  : int | None    # temporal qualifier
valid_end    : int | None
spatial      : str | None    # spatial scope
degree       : float | None  # for gradable predicates
```

This — not a string — is what makes contradiction and relevance computable.

6.2 The predicate catalogue

The catalogue is a **schema**, not content. It fixes slot names, **slot order** (which determines the canonical key), and which slot is functionally determined by the others.

Engine core predicates (domain-neutral, shipped):

Predicate	Slots (ordered)	functional_in
space:exactly_at	entity, place	1
core:holds_role	agent, role, context	—
body:alive	agent	—
economy:owes_amount	debtor, creditor, amount	—
looking_for	seeker, target	—
was_at	agent, place, when	—
place:has_surveillance	place, level	1
place:has_privacy	place, level	1
place:has_noise	place, level	1

`functional_in: i` means slot `i` is determined by the others — an entity is at exactly one place — which is what C2 contradiction detection uses.

A world pack registers its own predicates from `canon.json` at load. Registering the same name with a different shape **raises**: slot order defines the canonical key, so two packs silently disagreeing would make their beliefs unmergeable.

6.3 Canonical keys

```
core_key() = "{predicate}({slot1}={v1},{slot2}={v2},...)[{valid_start},{valid_end},{spatial}]"
            slots emitted in CATALOGUE ORDER, missing slots emit "None"
key()      = polarity + core_key()
```

A belief is filed under its **core key**. Polarity lives in the accumulated log-odds, not in the key — which is what lets "he was there" and "he was not there" be one belief with evidence on both sides rather than two beliefs.

Distortion produces a **different core key**, hence a genuinely separate belief. This is how a town can end up confidently believing something that never happened.

6.4 Contradiction

`contradicts(a, b)` returns `"C1" | "C2" | "C3" | None`.

Precondition — qualifier overlap. If both have a `spatial` scope and they differ, no contradiction. If both have complete temporal intervals and they do not intersect, no contradiction.

Kind	Condition
C1 direct negation	same <code>core_key()</code> , opposite polarity
C2 functional conflict	same predicate with <code>functional_in = i</code> , both positive, all non- <code>i</code> slots equal, slot <code>i</code> differs
C3 declared mutex	a declared pair <code>((pred_a, pol_a), (pred_b, pol_b))</code> matches and the first slot values agree. Shipped list: <code>body:alive +</code> vs <code>body:alive -</code>

Contradictions are **reported as reason entries, never resolved**. A character holding two contradictory beliefs is a state the system permits and the inspector displays.

6.5 Relevance

Used for cue-based memory retrieval and for deciding whether a belief answers a topic.

```
predicate_similarity(pa, pb, lam=0.7):
    1.0                if pa == pb
    exp(-lam * d)      if a taxonomy distance d is declared for the pair
    0.0                otherwise

relevance(prop, topic, beta=0.5):
    psim = predicate_similarity(prop.predicate, topic.predicate)
    if psim == 0: return 0
    names = catalogue slots of topic.predicate (or topic's own slot names)
    argmatch = mean over names of:
        beta    if topic.slots[n] is None          (wildcard)
        1.0     if prop.slots[n] == topic.slots[n]
        0.0     otherwise
    qualmatch = 0.0 if both have spatial scopes and they differ, else 1.0
    return psim * argmatch * qualmatch
```

Shipped taxonomy distance: ("was_at", "space:exactly_at") = 1, giving similarity $e^{(-0.7)} \approx 0.497$.

7. Events, observations, claims

Three distinct things, and keeping them apart is the design:

Event	the smallest authoritative unit — <i>what happened</i>
Observation	one agent's quality-weighted perception of an event
Claim	a communicated proposition — <i>what was said</i> . It updates belief; it is not truth

Event:

```
event_id    : int
world_time  : int          # minutes
type        : str
actor       : str | None
location    : str | None
payload     : dict
canonical   : bool = True
```

Observation:

```
observer    : str
event_id    : int
modality    : "sight" | "hearing" | "media"
quality     : float # [0,1] after distance / noise / obstruction
event       : Event
```

7.1 Event types and what each triggers

type	Asserts its proposition?	Also triggers
claim	yes	belief update; the ordinary retelling/speech path
broadcast	yes	perception via media_exposure ; credibility from the channel; makes common knowledge
question	no — names a proposition without asserting it	perceived and remembered; never evidence
promise	yes	memory type commitment ; the promise ledger
threat, attack, crime	yes	witnessed-act credibility; faction heat; identity threat; climate reaction
gift	yes	witnessed-act credibility
move	—	routine arrivals; observable, remembered
faction_mobilize	—	narrated, discharges threat; returns early from the event path
ally_arrives	—	scheduled by mobilize
discredit	—	climate reaction

7.2 Payload keys the engines read

Key	Meaning
proposition	the claim, as a dict
asserter_polarity	"+" / "-" — how it was asserted, which may differ from the stored proposition
origin_event	the origin token that travels with the claim
assertion_strength	[0,1] — how forcefully it is asserted (fidelity along a chain, certainty of a spoken move)
audience	explicit set of agent ids who may perceive it. Absent means everyone co-located
medium	in_person / phone / note
domain	which competence bears on it; default "street"
importance	[0,1] , drives memory importance and attention
summary	human-readable memory content
appraisal	the OCC appraisal dict (see §16)
severity	[0,1] , drives climate reaction and identity threat
credibility	broadcasts only: the channel's credibility, used as both trust and competence
channel	broadcasts only
target	who it was done to / whom a promise was made to
diffusion	{speaker, listener, hops, distorted, distortion_kind, distortion_detail, fidelity}
spoken	{addressee, act, certainty, honesty}

8. Agent state

The complete per-character state. Fields marked **authored** are read from a pack and never written by the runtime (see §8.2).

8.1 Fields

Identity and presentation

Field	Type	Notes
id	str	agent:<name>
public_name, objective_name	str	what they are called; what they <i>are</i>
aliases	tuple[str]	names the parser accepts
appearance	dict	{"garment": "item:apron", "symbols": [...]} — any keys
voice	dict	base pitch/rate, say_as respellings
dialect	str	style vector passthrough

Disposition (fixed; never moved by play)

Field	Type	Notes
big_five	dict	openness, conscientiousness, extraversion, agreeableness, emotional_stability (and optionally impulsivity)
schwartz	dict	value name → base strength
needs	dict	
values	dict	truthfulness decides lie-vs-refuse; others read by policy
epistemic	dict	curiosity, confirmation_bias, appearance_sensitivity, drawn_to, source_trust
education	dict	domain → competence
social_status	float	prestige baseline [0,1]
resources	float	liquid wealth [0,1]

Position in society

Field	Type	Notes
identities	list[dict]	{id, group, accessibility}
roles	list[dict]	{role, context}
networks	tuple	family, shift, union, congregation
contacts	tuple	whose number they have; empty → inferred
goals	list[dict]	{content, priority, about?}
secrets	list[Secret]	see §9.6
routine	Routine	schedule, or None to stay put
location	str	

Owned mutable state

Field	Type	Owner
beliefs	{core_key: Belief}	BeliefEngine
memory	list[Memory]	MemoryEngine
affect	AffectState	AffectEngine
relationships	{other_id: {field: value}}	RelationshipEngine
reputation	{subject_id: {dimension: degree}}	ReputationEngine (this agent's view of others)
identity_salience	{identity_id: weight}	IdentityEngine
identity_threat	{identity_id: [0,1]}	runtime (raised on in-group harm, decays)
tom_models	{(other, prop_key): {...}}	TheoryOfMindEngine
attention_day, attention_spent	int	topology (daily intake budget)

Relationship vector fields (per other agent): trust (default 0.3), liking, familiarity, fear, respect, dependence, attraction, kinship, loyalty, favor_debt, owes_favor. Trust/respect/fear/dependence are clamped to [0,1]; the rest to [-1,1].

8.2 Authored-only fields

State an engine **reads** and nothing in the runtime **writes**. This is declared explicitly and checked by test, because the same defect has occurred six times: a field consulted by four engines that only ever holds what a pack authored.

Field	Why it is authored-only
<code>dependence</code>	about obligation and circumstance, not conduct
<code>kinship</code>	a fact about a cast, not a thing play changes
<code>loyalty</code>	a faction layer could move it; none does
<code>favor_debt</code> , <code>owes_favor</code>	the promises layer moves the player's debts, not NPCs' to each other
<code>reputation:status</code>	prestige. Conduct moves <code>decent</code> , appearance moves how somebody is <i>read</i> ; neither is prestige

Anything read without a producer must either acquire one or appear on this list.

8.3 Derived properties

```
trust_in(other)      = relationships[other].trust      else 0.3
competence_in(domain) = education[domain]              else 0.3
trait(name, 0.5)      = epistemic[name]                else default
value(name, 0.5)      = values[name]
                        else schwartz[proxy] where proxy maps
                        truthfulness→benevolence, loyalty→tradition,
                        self_preservation→security
                        else default

skepticism = clamp01(0.6 - 0.3*agreeableness - 0.2*(emotional_stability - 0.5))

will_lie_about(secret):
    False if secret has no cover_story      # the engine never invents a lie
    else value("truthfulness") < secret.min_trust
```

`will_lie_about` deserves comment: **no cover story means no lie**. The engine will not invent what somebody claims instead, any more than it invents a fact. A world that has not authored the lie gets a refusal — which is itself information.

8.4 The location epoch

Every write to any agent's `location` increments a module-global counter. The world's occupancy index is cached against that counter, so it can never go stale regardless of who moves somebody — routines, the player, a snapshot restore, or a test. This is cheap and total; the alternative is remembering to invalidate at each of the half-dozen sites that move somebody, which works until the seventh is added.

9. World state and the world-pack format

9.1 World container

Field	Contents
global_seed	the string every seed derives from
world_time	minutes since world epoch
places	id → attrs
entities	id → {name, ...}
channels	id → {type, ...}
agents	id → Agent
media_exposure	(agent_id, channel) → {attention}
scenario_events	scheduled Events
factions	raw dicts, built into Faction by the runtime
initial_beliefs	authored belief seeds
topics	topic_id → Topic
identity_values	identity → value names it amplifies
predicates	raw predicate declarations (topology reads these)
predicate_domains	predicate → which competence bears on it
predicate_competence	predicate → {domain, level} to recognise it
standing_conduct	predicate → {dimension, sign, weight}
appearance_reactions	token → {status, threat, signals}
dialogue_templates	act → register → line
slang_lexicon	plain word → in-group word
diffusion_params, routine_params, tuning	parameter overrides
command_aliases	intent → words, extending the parser lexicon
focus_agents, player_start, scenario_intro	scenario framing
canon_facts	propositions true in this world (documentation/validation)

9.2 Pack layout

```
worldpacks/<name>/
  world.json      places, entities, channels, factions, media exposure,
                  tuning, diffusion/routine params, vocabulary
  scenario.json   seed, start time, scheduled events, focus agents, intro
  canon.json      pack-declared predicates and canon facts
  topics.json     what the player can ask about, and how the world reacts
  initial_state.json authored starting beliefs
  characters/*.json one file per agent
```

9.3 world.json — place attributes

```
"place:mess": {
  "label": "Mess",
  "aliases": ["mess", "canteen", "galley", "cafeteria"],
  "exits": ["place:hub", "place:quarters"],
  "noise_level": 0.6,           // [0,1] degrades perception quality
  "surveillance_level": 0.2,    // [0,1] observability; norm cost; faction heat
  "privacy_level": 0.35,       // [0,1] gates common knowledge; climate baseline
  "formality": 0.25,           // [0,1] register setting
  "gossip_factor": 1.9,        // multiplier on encounter rate
  "climate": { "candour": ..., "suspicion": ..., "openness": ... } // optional override
}
```

Every attribute is optional; the defaults are `noise 0.0`, `surveillance 0.0` (0.3 where the climate baseline reads it), `privacy 0.3`, `formality 0.4`, `gossip_factor 1.0`.

9.4 canon.json — declaring predicates

```
"seal_breached": {
  "slots": ["section", "when"],
  "functional_in": 1,
  "domain": "engineering",
  "requires_competence": { "domain": "engineering", "level": 0.6 },
  "classification": "trade",
  "access": ["role:engineer"], // circles that will circulate it
  "locality": ["place:store"], // places where it circulates at all
  "transmissibility": 0.45    // overrides the class default
}
```

Four independent axes, each read by a different mechanism:

Key	Read by	Effect
domain	belief, topology	whose word counts; whether shop talk interests the listener
requires_competence	interpretation	what it takes to <i>identify</i> what you saw
classification / transmissibility	topology	how freely it travels
access	topology	a hard wall: outside the circle it does not pass
locality	topology	a hard wall: elsewhere it does not circulate

9.5 topics.json — what can be asked, and how the world answers

```
{
  "id": "breach",
  "label": "The seal that failed in Stores",
  "aliases": ["breach", "seal", "leak", "decompression", "stores"],
  "query": { "predicate": "seal_breached", "slots": {...}, "polarity": "+" },
  "player_claim": { ... }, // what the player asserts if they claim it
  "player_assertion": { ... },
  "reactions": [ // events emitted when the topic is raised
    { "type": "threat",
      "payload": { "summary": "...", "importance": 0.6,
        "appraisal": { "desirability": -0.5, "expectedness": 0.4,
          "agency": "other", "praiseworthiness": -0.2,
          "target": "$addressee" } } }
  ],
  "phrasings": {
    "affirm": { "formal": "...", "neutral": "...", "vernacular": "..." },
    "deny": { "formal": "...", "neutral": "...", "vernacular": "..." }
  }
}
```

`$player` and `$addressee` are substituted at resolution time.

A topic with no phrasings makes characters deflect. The engine ships no `inform` template: stating what is true about a world is the world's job. This is not a limitation to route around — it is what stops a character asserting a fact borrowed from another pack.

9.6 Secrets

```
{
  "id": "secret:the_seal",
  "avoid_label": "who_broke_the_seal",
  "guards_topics": ["breach", "the_check"],
  "summary": "Lindqvist signed off the seal check without doing it.",
  "surface_forms": ["i skipped", "never checked", "no sealant",
    "my fault", "i signed it off"],
  "protects": [ { "predicate": "skipped_check", "slots": {...}, "polarity": "+" } ],
  "about": ["agent:lindqvist"],
  "min_trust": 0.6,
  "cover_story": { "predicate": "...", "slots": {...}, "polarity": "..." },
  "cover_phrasings": { "formal": "...", "neutral": "...", "vernacular": "..." }
}
```

Field	Role
protects	propositions whose core keys are never allowed facts, at any trust level
guards_topics	topics the character will not discuss below <code>min_trust</code>
surface_forms	lowercase substrings the validator scans released text for
avoid_label	the <i>label</i> passed to the text layer — never the secret content
cover_story	the proposition asserted instead , if the character will lie
about	who it concerns; read by the leverage model

Trust unlocks engagement, not disclosure. A protected proposition is filtered out of the candidate set regardless of trust, so it never reaches a text provider's context at all.

9.7 characters/*.json

```
{
  "id": "agent:adeyemi",
  "names": { "public": "Adeyemi", "objective": "Adeyemi",
    "aliases": ["adeyemi", "ade", "nurse"] },
  "big_five": { "openness": 0.5, "conscientiousness": 0.6, "extraversion": 0.5,
    "agreeableness": 0.5, "emotional_stability": 0.55 },
  "schwartz": { "security": 0.5, "achievement": 0.5 },
  "needs": { "safety": 0.4, "recognition": 0.4 },
  "identities": [ { "id": "medic", "group": "group:crew", "accessibility": 0.8 } ],
  "roles": [ { "role": "role:doctor", "context": "group:crew" } ],
  "education": { "domains": { "general": 0.6, "medicine": 0.8, "engineering": 0.3 } },
  "relationships": { "agent:voss": { "trust": 0.5, "liking": 0.2, "familiarity": 0.6 } },
  "secrets": [],
  "goals": [ { "content": "keep_the_station_running", "priority": 0.7 } ],
  "social_status": 0.5,
  "resources": 0.4,
  "location": "place:med",
  "routine": [ { "from": "08:00", "place": "place:med", "activity": "ward" },
    { "from": "12:00", "place": "place:mess", "activity": "lunch" } ],
  "appearance": { "garment": "item:scrubs" },
  "networks": ["shift:day"],
  "epistemic": { "curiosity": 0.6, "confirmation_bias": 0.4,
    "source_trust": { "channel:station_net": 1.2, "told": 0.8 } },
  "values": { "truthfulness": 0.7 },
  "voice": { "pitch": 0.0, "rate": 1.0, "say_as": { "Adeyemi": "ah-DAY-ay-mee" } }
}
```

9.8 initial_state.json — seeding belief honestly

```
{ "agent": "agent:voss",
  "proposition": { "predicate": "seal_breached", "slots": {...}, "polarity": "+" },
  "summary": "Voss read the incident report first thing.",
  "origin_event": "incident:store_seal_night",
  "trust": 0.95, "competence": 0.9, "importance": 0.85 }
```

Seeded beliefs go through the **same** belief update as anything else, with an explicit origin. Two characters seeded from the same `origin_event` are correlated for the rest of the world's life — which is correct: they read the same report.

9.9 scenario.json

```
{ "global_seed": "relay-station-001",
  "world_time": 520,
  "player_start": "place:hub",
  "focus_agents": ["agent:voss", "agent:okonkwo", "agent:rask"],
  "intro": "...",
  "events": [ { "world_time": 610, "type": "broadcast",
    "actor": "channel:station_net", "location": null,
    "payload": { "channel": "channel:station_net",
      "credibility": 0.75,
      "proposition": {...},
      "summary": "Station net: a seal failed in Stores.",
      "origin_event": "incident:store_seal_night" } } ] }
```

9.10 What the pack validator checks

`lwr author / validate_world_pack` reports, per finding, a severity and a path:

- every place reachable from the player's start (`exits` connectivity);
- every character shares a place with somebody at some point in the day (otherwise diffusion can never fire for them);
- every topic is answerable by somebody;
- every secret guards a topic that exists;
- every proposition's predicate is registered, and its slots match the schema;
- values that must lie in `[0,1]` do;
- routines parse, and their places exist;
- aliases do not collide across places, agents and topics;
- topics have phrasings (otherwise characters deflect).

The generator (`lwr generate-pack`) runs this report **inside** its own loop — generate, read the report, repair what it names, repeat — so what comes out has passed the same gate a hand-written pack must pass.

Part III — The two pipelines

Everything in the runtime happens on one of three paths:

Path	Entry point	Granularity
Event	<code>process_event(event, dt)</code>	one thing happened; every observer reacts
Tick	<code>advance_time(delta_minutes)</code>	time passed; the society moves
Turn	<code>submit_player_text(text)</code>	a player did something; parse → apply → answer

The turn path is built out of the other two.

10. The event pipeline (`process_event`)

Contract: given one `Event` , update every agent who could perceive it, and return `{agent_id: [reason entries]}` plus `_`-prefixed layer buckets.

10.1 Ordering

```
process_event(event, dt = 1.0):

0.  climate.react(event)                → "_climate" trace
1.  world.world_time = max(world_time, event.world_time)
2.  store.append_event(event)            (ledger is authoritative)
3.  if event.type == "faction_mobilize": narrate, return early
4.  if event.type in (threat, attack, crime) and event.location:
    faction = director.faction_controlling(location)
    heat += 0.25 * |severity| * (0.5 + surveillance_level)
5.  observations = perception.filter(event, world)
6.  took_in = []
    for obs in observations:              # per observer
        agent = world.agents[obs.observer]
        catch_up(agent)                   # LOD: realise deferred decay

6a. prop = None if event.type == "question" else event.proposition
    if obs.observer == event.actor: prop = None      # you learn nothing
                                                    # by hearing yourself
6b. if prop is not None:
    prop, r = interpretation.interpret(agent, event, prop, world,
                                         obs.quality, obs.modality)
6c. if prop is not None and prop.core_key() not in agent.beliefs:
    if not topology.spend_attention(agent, world):
        prop = None                             # heard it, no room today
6d. if prop is not None:
    (trust, competence) = credibility_inputs    § 10.3
    eff_trust = trust * obs.quality * clamp01(assertion_strength)
    eff_trust *= source_credence(agent, event, obs)    § 10.4
    skepticism = clamp01(agent.skepticism + climate.skepticism(place))
    belief, r = belief.update(agent.beliefs, prop, ...)
    belief.hops = 0                                if not relayed
                = relayed                          if belief is new
                = min(prior_hops, relayed) otherwise
    took_in.append((observer, quality))
6e. if standing enabled and prop is not None:
    deltas, r = standing.judge(agent, prop, belief.expected_prob)
    apply relationship deltas / reputation deltas
6f. if event.actor and event.actor != agent.id:
    pull = appearance.drawn_to(agent, actor, world)
    if pull: relationships[actor].attraction = pull
6g. identity_threat(agent, event)            § 10.5
6h. memory encode:
    type      = "commitment" if event.type == "promise" else "episodic"
    content   = payload.summary or f"{type} by {actor}"
    proposition= event.proposition            (the UNINTERPRETED one)
```

```

        importance = payload.importance (default 0.5)
        valence     = appraisal.desirability (default 0.0)
        source_conf= obs.quality

6i.  affect.update(agent.affect, event.appraisal, dt)
6j.  persist belief / memory / affect / trace    (if this agent is projected)

7.  common_knowledge.observe(world, event, took_in)    → "_common_knowledge"
8.  promises.record(world, event, [id for id,_ in took_in]) → "_promises"
9.  store.commit()                                     (one commit per event)
10. merge traces:  standalone call REPLACES last_traces;
                   inside a tick, layer buckets ACCUMULATE

```

10.2 Six ordering decisions that matter

1. **Climate reacts before anything else.** The room is already different by the time the claim lands in it, and the trace should say so rather than make a designer infer it.
2. **A question never becomes evidence.** It names a proposition without asserting it. It is still perceived, remembered and reacted to, because being asked something is a thing that happened to you.
3. **You learn nothing from hearing yourself.** Without this, a character asserting something becomes surer of it: their own utterance returns as evidence from a fresh origin. That is a provenance loop and turns repetition into proof.
4. **Attention is spent only on new claims.** Being reminded of something you already hold is free.
5. **Memory stores what the observer *understood*, not what happened.** The interpreted proposition, or nothing when they took nothing in. A `question` is the exception: it names a proposition without asserting it, and being asked about something is what makes it retrievable later. Storing the event's own proposition made belief and recall disagree about the same moment, and handed the precise claim back through `retrieve_memories` to an observer the simulation had just decided could not identify it.
6. **Hops take the minimum.** Hearing your own rumour come back from three mouths away does not relocate you three mouths from what you witnessed.

10.3 Credibility inputs, by how the claim arrived

Arrival	trust	competence
broadcast	<code>payload.credibility</code>	<code>payload.credibility</code>
witnessed act (attack, threat, crime, gift)	1.0	0.95
told (claim, promise, ...)	<code>observer.trust_in(speaker)</code>	<code>speaker.competence_in(payload.domain)</code>
no speaker	0.3	0.5

Seeing it is not being told it. Every other proposition reaching this point is testimony and is rightly discounted by how far the source is trusted. An act you *watched* is evidence about the world; how much you trust your assailant has nothing to do with whether they hit you. Before this rule existed, a character hit in front of a room believed it at $p = 0.52$.

`obs.quality` still discounts it, so a bad line of sight in a dark room is a poor witness — which is right.

Competence is domain-specific. A doctor on medicine and a bartender on gossip are not equally credible. An earlier version read `domain` and then discarded it in favour of a flat 0.5, so every character's authored education had no effect on belief at all.

10.4 Source credence — *whom they believe*

Every other input to belief is a fact about the **claim** — how close you were, how competent the speaker is, how credible the channel says it is. None of them can say *this one takes the intercom as gospel and their colleagues as noise, and that one is the exact reverse*.

A character may author `epistemic.source_trust`, a multiplier looked up **most specific first, first hit wins** (not multiplied — somebody who distrusts City Radio specifically should not also be charged for distrusting radio in general):

Arrival	Keys tried, in order
broadcast	<code><channel id></code> , <code><channel type></code> , <code>broadcast</code>
media modality	<code>broadcast</code>
witnessed act, or no actor	<code>witnessed</code>
told	each of the speaker's <code>role</code> names, then <code>told</code>

The multiplier is clamped to `[0, 2]`. Absent table → 1.0 and nothing changes, which is why adding this left every shipped pack byte-identical.

Measured effect: the same announcement in the same second credited at **0.698** by one character and **0.571** by another.

10.5 Identity threat

When one of yours is hurt.

```
if event.type in (attack, threat, crime):
    target = world.agents[payload.target]
    if target is None or target is observer: return
    shared = {i.id for i in observer.identities} & {i.id for i in target.identities}
    for identity_id in sorted(shared):
        proposals, appraisal_boost = social_identity.on_event(...)
        # proposal amount is +0.6
        observer.identity_threat[id] = min(1, current + 0.6 * severity)
        affect.update(observer.affect, appraisal_boost, 1.0)
        # boost = {desirability: -0.6, agency: "other",
        #          praiseworthiness: -0.3, target: event.actor}
```

This is felt as an offence against you, by the person who did it, even though it did not happen to you — which is Tajfel's point.

`SocialIdentityEngine` was written for this and was unreachable for a long time, because it asked the *event* which identity was threatened and no event ever said. Whether an event threatens **your** belonging depends on who you are, so it can only be decided per observer.

Identity threat decays with a **24-hour half-life** and is dropped below 0.01. It is alarm, not a fact about the world; a threat that never faded would mean one attack permanently rewrote which identity a character leads with, which is personality drift by another route.

Measured: bystanders sharing a belonging with an assault victim feel it at `resident: 0.48`, halving in a day.

10.6 Reason-trace accumulation

Two different questions, two different behaviours:

- **Per-agent buckets** (`agent:voss`) answer *what did this event do to this character*. They always hold the most recent event. Accumulating them across a tick cost 553,454 entries and 36 MB per hour of world time at 2,000 characters, and answered nobody's question.
- **Layer buckets** (`_climate`, `_diffusion`, `_routine`, `_pursuit`, `_notes`, `_promises`, `_common_knowledge`, `_actions`, `_director`, `_revision`) answer *what happened during this time advance*. These **accumulate**, capped at **400 entries per bucket per tick**, with an exact `trace.truncated` count appended when the cap binds.

A standalone `emit_event` still **replaces**, because "what did this event cause" is the right answer there.

11. The tick pipeline (`advance_time`)

Contract: advance the world by `delta` minutes; return the accumulated layer traces.

11.1 Order, and why it is that order

```
elapse(minutes):                                # time itself, and nothing social
    world_time += minutes
    action_bridge.expire(world_time)             → intents nobody answered time out
    climate.step(minutes)                       → relax toward authored baseline
    grow_familiarity(minutes)                   → time spent in the same room
    routine.step(minutes)                      → the cast goes about its day

advance_time(delta):
    last_traces = {} ; collecting = True
    target = world_time + delta

    1. due = scheduled events in (world_time, target], sorted by (time, event_id)
       for each: elapse(e.world_time - world_time) # move the cast up to it
                  process_event(e)                 # THEN let it happen
                  remove it from the schedule

    2. elapse(target - world_time)                → the rest of the window
    3. pursuit.step(world, delta)                 → whoever has a reason to speak, speaks
    4. promises.step(world, delta)                → whatever came due is settled
    5. notes.step(world, delta)                   → anything left lying about is picked up
    6. diffusion.step(world, delta)               → who happens to talk, talks
    7. per agent: affect.decay, identity_threat fade,
                  memory.decay_and_consolidate
    8. store.commit()
    9. director.step(world, delta)                → clocks, heat, mobilisation
       emit trace.truncated for every capped bucket
```

The window is walked in order, not batched. Scheduled events used to be processed first, against the positions everybody held at the *start* of the window, so **who witnessed an event depended on how large a step the caller took**: one

`advance_time(30)` made a character a witness to something at minute 20 in a room they had left at minute 10, and `advance_time(10)` then `advance_time(20)` did not. Both end in the same place at the same time, and only one of them can be right about the past.

Splitting the window is exact rather than approximate: climate relaxation and familiarity growth are both $x += (target - x) \cdot (1 - k^t)$, and $1 - k^{(a+b)}$ is precisely the two applied in turn.

Every one of these positions was chosen and several were wrong first:

- **Routines before diffusion.** Moving people after they gossip means everyone gossips with yesterday's company.
- **Familiarity before routines.** The hours just elapsed are credited to where people actually spent them, not to where they are about to go.
- **Pursuit before diffusion.** A purposeful conversation should get the room before the random ones do.
- **Promises before notes and diffusion.** A character let down last night meets the world knowing it.
- **Notes after pursuit.** Otherwise a note left this tick is read in the same tick, which makes it a conversation rather than a note.
- **Diffusion before decay.** A retelling creates memories, and memory caps are enforced during consolidation. Running diffusion afterwards left the tick ending *over* the cap — a bound that holds only until the next tick is not a bound.
- **The director last.** The macro layer reacts to a world that has already talked.

11.2 Familiarity growth

```
hours = delta / 60
share = 1 - (1 - 0.01)^hours          # FAMILIARITY_PER_HOUR = 0.01
for each place, for each ordered pair (one, other) co-located:
    f = relationships[other].familiarity
    if f < 0.999:
        relationships[other].familiarity = min(1, f + share * (1 - f))
```

Symmetric, because knowing somebody is not one-sided. At 0.01 it takes about seventy hours together to close half the remaining distance: a few weeks of sharing a street, not an afternoon.

Time and not events, and the first attempt was events. That version grew familiarity only while something was happening, so two people who shared a room all day and had a quiet week ended the week as strangers — and it stopped entirely once a world saturated, which is exactly when a cast should be settling into knowing each other. Measured: 0.600 → 0.631 on the first day, then nothing for six more.

11.3 Decay

```
for agent in world.agents.values():
    if population_lod and not agent.affect.active_emotions: continue
    dt = target - last_decay[agent.id]
    if dt > 0:
        affect.decay(agent.affect, dt)
        fade_identity_threat(agent, dt)
    memory.decay_and_consolidate(agent.memory, target)
    last_decay[agent.id] = target
```

`dt` is measured **since this agent was last decayed**, not the raw window: a mid-window event may already have caught the agent up under LOD, and double-counting that span would over-decay affect.

12. The turn pipeline (a player says something)

```
submit_player_text(text):
  1. parsed = parser.parse(text)           # closed command catalogue
  2. dispatch by intent:
      move      → move the player, emit a move event
      ask       → topic lookup; respond(addressee, topic=...)
      claim     → player_says(...) as a claim event
      promise   → emit a promise event with target
      threat    → emit a threat event; conduct claim if standing on
      attack    → emit an attack event; conduct claim if standing on
      assist    → emit a gift event; conduct claim if standing on
      accuse    → ...
      observe/inspect/time/media/wear/system → the corresponding surface
  3. run the topic's authored reactions as events
  4. return TurnResult { receipts, npc_response, developer_trace }
```

12.1 `respond(agent_id, topic, interlocutor)`

```
respond:
  catch_up(agent)
  direct = _direct_answer(agent, interlocutor, topic, gate_reasons)
  if direct: return direct
  raw = core.say(agent, interlocutor)           # full deliberation
  return DialogueResponse(..., reasons = gate_reasons + raw.reasons)
```

Carrying the gate reasons forward matters: `say()` overwrites the agent's trace, so a secret gate would otherwise produce an unexplained deflection — the one line a studio most needs explained.

12.2 `_direct_answer` — the answer path, in order

```
1. project_for(agent)                # the player has now met them
2. conv = conversation state for (agent, interlocutor) # persists across save/load
   times_asked = conv.discussed_topics.get(topic, 0)

3. TRUST GATE FIRST
   guarding = secrets whose guards_topics contains this topic
   spent     = [s for s in guarding if secret_is_out(agent, s)] # common knowledge
   guarding -= spent
   trust     = clamp01(agent.trust_in(interlocutor)
                     + climate.stranger_trust(agent.location))
   withholding = [s for s in guarding if trust < s.min_trust]
   if withholding:
       lying = first s in withholding with agent.will_lie_about(s)
       if lying: return _tell_a_lie(...) # cover story path
       emit dialogue.secret_gate; return None # fall through to deflection

4. CANDIDATES
   protected = protected_keys(agent.secrets) - keys_already_out(agent)
   for belief in agent.beliefs:
       confidence = max(p, 1-p)
       if confidence < 0.6: skip
       fit = relevance(belief.proposition, topic_query)
       if topic_query and fit == 0: skip
       affirmed = p >= 0.5
       if core_key in protected or str(fact) in protected: skip
       candidates.append(Candidate(proposition, confidence, fit, affirmed))
   if no candidates: return None

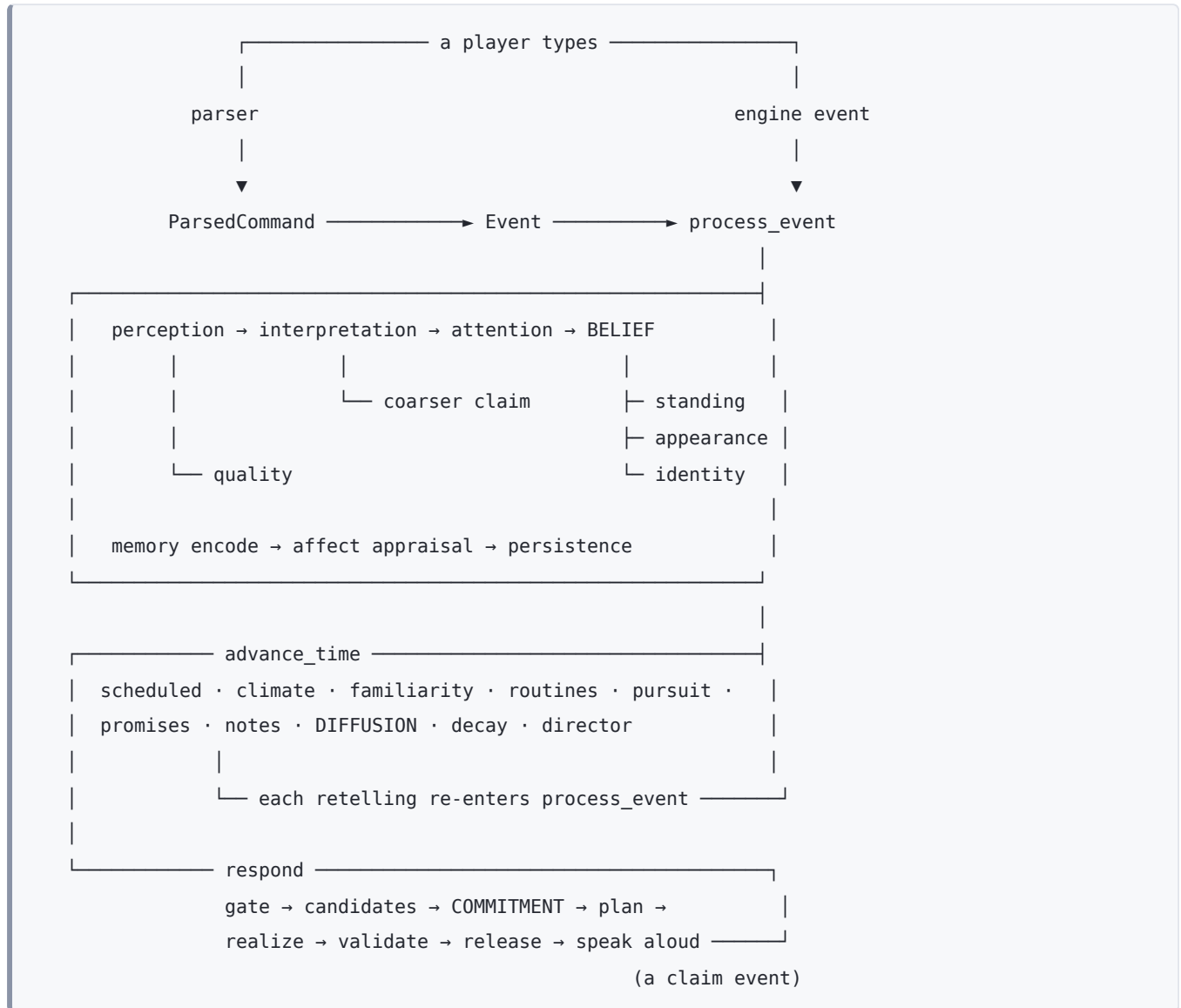
5. COMMITMENT (the decision the text layer used to make)
   commitment = commitment.select(candidates, act="assert", limit=2,
                                  seed_parts=(global_seed, agent_id,
                                                conv.turn_count, world_time,
                                                "dialogue"),
                                  forbidden=avoid_labels(guarding or secrets))

6. PLAN
   style     = style_for(agent, interlocutor)
   phrasing  = topic phrasings for affirm/deny as the belief actually stands
   plan     = DialoguePlan(..., avoid_topics=labels, phrasing=...).commit(commitment)
   if times_asked >= 1: plan.max_length //= 2 # re-asked → terser

7. return _deliver(...)                § 36
```

The gate is checked **before** the fact filter, and that ordering was a bug fix: checking it afterwards hid the gate exactly when the character's only knowledge of the topic was the protected proposition — which is the case it exists for.

12.3 The order of the whole system, in one picture



The loop closes: what a character says re-enters the world as an event that other characters perceive. That is why the text layer must not choose the content.

Part IV — Engines, in full

Each engine below is stated as: **state** → **formula** → **parameters** → **why** → **failure mode it prevents**. Engines in layer 1 know nothing about a world, a player or a text layer. That is what makes each one testable in isolation and replaceable without touching anything else.

13. Belief

13.1 State

```
Belief:
    proposition      : Proposition      # always stored with polarity "+"
    logit_val        : float            # log-odds the proposition holds
    support_for      : float            # accumulated |positive log-evidence|
    support_against  : float            # accumulated |negative log-evidence|
    provenance       : list[dict]       # one entry per piece of evidence
    origin_counts    : {origin_key: n}  # distinct origins, and hearings each
    provenance_dropped : int            # entries trimmed, surfaced honestly
    hops             : int              # shortest known distance to first-hand
    spreading        : bool             # Daley-Kendall: still passing it on?
    valid_start      : int
    txn_time         : int              # when last updated
```

A provenance entry:

```
{ claim_id, origin_event, kappa, eta, delta, speaker }
```

`delta` is the **signed log-odds this piece of evidence actually contributed**, and `speaker` is who said it. Both exist so the belief can be **recomputed** later — see §14. Without `delta`, exposing a liar could only change what they convince you of *next* time.

13.2 Update

```
credibility(trust, competence, skepticism):
    k = kappa0 + alpha*trust + gamma*competence - delta_*skepticism
    return clamp(k, kappa_min, kappa_max)

correlation_discount(belief, origin_event):
    if origin_event is None: return 1.0
    heard = belief.origin_counts.get(str(origin_event), 0)
    return rho ** heard

update(beliefs, prop, asserter_polarity, trust, competence, skepticism,
       claim_id, origin_event, world_time, speaker):
    key = prop.core_key()
    belief = beliefs.setdefault(key, Belief(prop with polarity "+", logit=0))
    kappa = credibility(trust, competence, skepticism)
    eta = correlation_discount(belief, origin_event)
    sign = +1 if asserter_polarity == "+" else -1
    evidence = eta * ln(kappa / (1 - kappa))
    contribution = sign * evidence
    belief.logit_val += contribution
    # THE POOL FOLLOWS THE CONTRIBUTION, NOT THE ASSERTION. Below kappa = 0.5 the
    # log-odds term is negative -- the source is credible enough to be worth
    # believing the opposite of -- so a positive claim from them LOWERS p.
    if contribution >= 0: belief.support_for += contribution
    else: belief.support_against += -contribution
    record_provenance(claim_id, origin_event, kappa, eta,
                      delta=contribution, speaker=speaker, limit=64,
                      origin_limit=512)
    belief.txn_time = world_time
    for other in beliefs: if contradicts(belief.prop, other.prop): report it
```

13.3 Derived quantities

```
expected_prob = sigmoid(logit_val)
ignorance = 1 / (1 + support_for + support_against)
conflict = 2 * min(SF, SA) / (SF + SA) (0 if both ~0)
```

Why dual support. A single probability cannot tell "nobody has said anything about this" from "two people have said opposite things and they cancelled". The first is 0.5 with `ignorance ≈ 1`; the second is 0.5 with `conflict = 1`. A character should behave differently in those two states, and a designer must be able to see which one they are in.

13.4 Parameters

Name	Default	Meaning
kappa0	0.55	base credibility of any source
alpha	0.25	weight of trust
gamma	0.20	weight of domain competence
delta	0.30	weight of the listener's skepticism
kappa_min	0.20	floor — must be > 0 , $\ln(0)$ otherwise
kappa_max	0.95	ceiling — must be < 1 , division by zero otherwise
rho_correlated	0.10	geometric attenuation per repeat from one origin
provenance_limit	64	individual claim records kept, for the inspector
origin_limit	512	distinct sources one belief tracks before it starts forgetting

The index does not evict. At capacity a *new* origin is refused: it is worth exactly nothing and is counted in `origins_refused`. An origin already in the index always counts up, however full the index is.

That is the whole of the saturation guarantee, and it took three attempts to learn why nothing else works. Each of the first three bounded the index by evicting and then correcting for it:

Design	What it bounded	How it leaked
evict, no correction	nothing	one origin contributed 2.94, was evicted, and its bare repetition contributed 2.94 again — 5.89 against a 3.27 ceiling, $0.95 \rightarrow 0.997$
a re-admission is worth ρ^1	one return	cycling <code>origin_limit</code> uninformative sources bought a fresh geometric ladder each cycle: $0.950 \rightarrow 0.99985$ over twenty, 8.83 log-odds
all re-admissions share $\rho/(1-\rho)$	the re-admissions	it did not count repeats already paid while the origin was still recognised (0.294 granted again on top of a saturated 3.2716), and a re-admitted origin was back <i>in</i> the index, so its next hearing took the ρ^{heard} branch and skipped the allowance: 6.18 log-odds after a hundred cycles

Every one of them was a correction applied at **one step** of a sequence whose length an adversary chooses. **An index that forgets cannot recognise a repeat**, and no amount of paying at the door changes that.

Not evicting makes the bound a consequence of the arithmetic rather than a rule laid on top of it: the n -th hearing from an origin is worth $\rho^{(n-1)}$ for every n , so its total is a geometric series under $1/(1-\rho)$ — for **any** ordering of events, across save and load, forever. It is checked that way too: twelve random orderings over 575 origins, with a save and a load in the middle, are compared against an eight-line reference model that implements only the stated rule.

The cost is stated rather than hidden. At capacity a *genuinely* new source is refused, so a belief that has met 512 distinct sources over-doubts the 513th. Doubt is the safe direction; the alternative was proof. `origins_refused` says how often it happened, so a pack that hits it can be seen to have hit it.

13.5 The correlation discount, and why it is geometric

The n -th assertion from an origin already heard $n-1$ times contributes $\rho^{(n-1)}$ of a single hearing. Because $\rho < 1$ the series converges:

```
max contribution of ONE origin = 1 / (1 - rho) = 1.111 hearings    (rho = 0.10)
```

This is a theorem, not a policy — see the non-evicting index above. That matters because the bound is a *product* claim (`single_origin_max_confidence` in the benchmark), and a claim that quietly becomes "holds until 512 sources have been seen" or "holds for the orderings we happened to test" is not the claim.

This bound is the point. The previous implementation returned a flat 0.1 for any repeat. That attenuates but does not saturate, so a single rumour repeated often still accumulated without limit:

200 repetitions of one origin → **p = 0.995**

That is the exact failure the feature exists to prevent, and it is the difference between "*a repeated rumour is weak evidence*" and "*a repeated rumour is proof if you are patient*".

Two records, because they answer different questions. The provenance list is what a person reads and is trimmed oldest-first to `provenance_limit`, with `provenance_dropped` incremented. `contributions` is what the *arithmetic* reads — cumulative signed log-odds per (`speaker`, `origin`) — and is not trimmed with it.

This separation is load-bearing and was once absent. Revision summed the deltas in the **display** list, so once a claim had been heard more than 64 times the early, heavy entries were gone and exposing the only source who ever supplied it moved the belief by nothing at all: 0.96344 before, 0.96344 after. A record kept "for the inspector" was silently the one the guarantee depended on.

Both indexes are bounded at `origin_limit` (512), and they are *different* bounds. The origin index counts distinct **origins** and refuses past capacity (above). `contributions` is keyed on the (**speaker**, **origin**) pair, because that is what `revision` matches on — so 400 origins each relayed by two mouths make 800 pairs while the origin index holds 400. Pairs past the limit fold into one unattributable bucket: their evidence still counts towards the log-odds, and revision can no longer unwind it.

Only the origin index carries the saturation guarantee. Losing attributability costs a *revision* its precision; it can never make a repeated claim worth more.

Origins are normalised to strings. JSON has only string keys; a saved belief that came back with string keys while origins were integers silently started treating a repeated rumour as independent evidence again.

13.6 Worked example

A stranger (`trust` = 0.3, `competence` = 0.5, listener `skepticism` = 0.4) tells you something with full quality and full assertion strength:

```
kappa    = 0.55 + 0.25(0.3) + 0.20(0.5) - 0.30(0.4) = 0.605
evidence = 1.0 * ln(0.605 / 0.395) = 0.4262
logit    = 0.4262      → p = 0.605
```

They tell you the same thing again, same origin:

```
eta      = 0.10^1 = 0.10
evidence = 0.10 * 0.4262 = 0.0426
logit    = 0.4688      → p = 0.615
```

A **different** witness with the **same** origin (they both read the same report) adds `0.10^2 = 0.01` of a hearing. An independent origin adds a full one.

13.7 Effective trust at the call site

The engine takes `trust` as given. The runtime supplies:

```
eff_trust = trust
    * obs.quality                # partial hearing → weaker evidence
    * clamp01(payload.assertion_strength) # conviction along the chain
    * source_credence(observer, event) # whom THIS person believes
```

and `skepticism = clamp01(agent.skepticism + climate_skepticism(place))`.

14. Revision: unwinding a liar

Lowering trust in a liar changes what they will convince you of **next** time. It does nothing about what they already convinced you of — and that is the part that matters. The whole point of exposing someone is that the things they told you stop counting.

14.1 Algorithm

```
discredit(world, source, factor=0.25, reason="exposed", predicate=None):
    factor = clamp(factor, FLOOR=0.1, 1.0)
    for agent in world.agents where agent.id != source:
        for key, belief in agent.beliefs:
            if predicate and belief.predicate != predicate: continue
            # From the ARITHMETIC record, never from the trimmed display list.
            matched = [k for k in belief.contributions
                       if k != FORGOTTEN and key_traces_to(*split(k), source)]
            if not matched: continue
            positive =  $\sum$  contributions[k][0] for k in matched
            negative =  $\sum$  contributions[k][1] for k in matched
            adjustment = (positive + negative) * (factor - 1)
            if |adjustment| < 1e-9: continue
            belief.logit_val += adjustment
            # EACH POOL LOSES WHAT IT WAS GIVEN. Adding a negative adjustment to
            # support_against reports weakened evidence as a contradiction, and
            # one discredited uncontested source produced conflict 0.947 out of
            # nothing. Weakening evidence raises IGNORANCE; only a counter-claim
            # raises CONFLICT.
            support_for = max(0, support_for + positive * (factor - 1))
            support_against = max(0, support_against + (-negative) * (factor - 1))
            for k in matched: contributions[k] *= factor
            mark the matching provenance entries revised (display only)
            record change {agent, belief, was, now, entries_reweighted,
                          sources_reweighted, independent_entries,
                          still_supported}
```

```
key_traces_to(speaker, origin, source) = speaker == source or names(origin, source)

names(origin, source):    source appears in origin BOUNDED BY a separator
                        from {start, end, _, :, /, |, ., -, space}
```

The second clause is what makes it propagate: a claim carries its origin through retelling, so a rumour three hops from the liar still names them.

The boundary is not decoration. A plain `source in origin` substring test means substrings of ids are other ids: discrediting `agent:ann` also unwound everything relayed from `said_agent:anna_100`, taking an uninvolved bystander's belief from 0.95 to 0.676.

14.2 Rules

- **Revision only ever reweights evidence that exists.** It never invents a belief, never removes one, and never touches a belief that has an independent source for what it holds.
- **FLOOR = 0.1**, not zero. People who lie about one thing were sometimes telling the truth about another, and a model that erased everything they said would make exposing them strictly *better* than never having heard them.
- **The liar's own beliefs are untouched.** A liar's beliefs are not evidence from them.
- `predicate` narrows the revision: being caught lying about where somebody was says little about your account of the weather.

This is a small incremental truth-maintenance system, deliberately not a general one — no justification lattice, no nogoods, no assumption sets. The honest description is: *evidence has weights, a weight changed, add it up again*.

`would_change(world, source)` answers the same question without changing anything, which is what a player asks before deciding whether exposing someone is worth it.

15. Memory

15.1 State

```
Memory:
    memory_id, owner, type, content
    proposition      : Proposition | None
    importance       : [0,1]
    emotional_valence : [-1,1]
    presentations    : list[int]      # world times of encode/recall
    last_recalled    : int
    source_conf      : [0,1]          # the observation quality it came from
    world_time       : int
    episodes         : int            # for a summary: how many it stands for
    about            : (subject, predicate) # judgements only
    evidence         : tuple[memory_id] # judgements only
```

15.2 Types and decay

```
DECAY_BY_TYPE = { judgement 0.28, emotional 0.25, commitment 0.30,  
                  summary 0.35, semantic 0.40, social 0.40, episodic 0.55 }  
                  default 0.5
```

A conclusion drawn from many occasions outlasts any one of them — that is the point of forming it: the episodes fade, the pattern stays.

15.3 Base-level activation (ACT-R, corrected twice)

```
base_activation(t, eps=1.0, time_scale=60.0, importance_slows_decay=0.6):  
    d = DECAY_BY_TYPE[type]  
    d *= max(0.05, 1 - importance_slows_decay * clamp01(importance))  
    total =  $\sum$  over presentations t_k of max(eps, (t - t_k)/time_scale + eps) ^ (-d)  
    return ln(total)          (or -10.0 if total <= 0)
```

Two corrections over the textbook form, both found by watching a long run rather than by reading the formula:

1. **Time scale.** World time is in minutes; feeding minutes into a power law put a once-seen memory below threshold after thirty of them. A character forgot a murder before the player left the room. Activation is measured in **hours**, which is the unit a scene works in — "last night", "this morning".
2. **Importance.** It previously had *no effect on recall whatsoever*: a witnessed murder at importance 0.95 decayed identically to a passer-by at 0.05, because importance only decided consolidation order. Important things get rehearsed, so importance now **slows the decay exponent** rather than adding a one-off bonus: it changes how long something lasts, not how loudly it scores once.

Result: being threatened stays retrievable about a week; something trivial, about three days.

15.4 Retrieval

```
retrieve(memories, t, topics=None, mood_valence=0.0):
    for m in memories:
        a = m.base_activation(t)
        if topics: a += w_topic * ln(1 + max_relevance(m.proposition, topic))
        a += w_emotion * |m.emotional_valence|
        a += w_mood_congruence * mood_valence * (+1 if valence >= 0 else -1)
    sort by a descending
    for (a, m) in sorted:
        prob = sigmoid((a - tau_ret) / noise_s)
        # A DRAW, not a threshold. `prob >= 0.5` is exactly `a >= tau_ret` for
        # any positive noise, so the parameter moved the reported number and
        # never the decision. The draw comes from the seed algebra, so a save
        # still replays exactly.
        came_up = (prob >= 0.5) if seed_parts is None else (
            seeded_uniform(derive_seed(*seed_parts, t, "memory",
                                     m.memory_id)) < prob)
        if came_up or (topics and a > tau_ret - 1.0):
            if m.last_recalled != t: reinforce(m, t)    # retrieval strengthens
            emit(round(a,3), round(prob,3), m)
        if len(out) >= working_cap: break
```

Retrieval **strengthens** what it retrieves — a presentation is appended — which is the ACT-R property that makes rehearsal work.

15.5 Consolidation and eviction

Three caps, enforced in this order every tick:

Pool	Cap	Why
episodic	200	the original bound
everything except summary	600	semantic, commitment and social memory was previously unbounded
summary	40	summaries are memories too. Excluding them created an unbounded path instead: one summary per tick, forever — 1,905 stored memories against a cap of 600 after 173 simulated days, 1,595 of them summaries

```

_consolidate(memories, t, pool, cap, scope):
    if len(pool) <= cap: return
    pool.sort(key = (importance, base_activation(t)))    # ascending
    to_drop = pool[: len(pool) - cap]
    reasons = _form_judgements(memories, to_drop, t)    # BEFORE the detail goes
    episodes =  $\Sigma$  m.episodes for m in to_drop
    append Memory(type="summary", importance=0.3,
                  content=f"(a blur of {episodes} minor moments)",
                  episodes=episodes, presentations=[t])
    remove every m in to_drop

```

Merging summaries and summing `episodes` keeps an honest count no matter how many times memory has been compressed — and it is also what actually happens to people: the details of what you forgot become *"a lot happened that month"*.

15.6 Judgement formation

Ninety percent of memory loss in a long run is capacity eviction. Evicting episodes without drawing anything from them means a character can live through the same thing fifty times and conclude nothing.

```

_form_judgements(memories, expiring, t):
    groups = {}
    for m in expiring where m.proposition is not None:
        for value in m.proposition.slots.values():
            subject = str(value)
            if not subject.startswith("agent:") or subject == m.owner: continue
            groups[(subject, m.proposition.predicate)].append(m)

    for (subject, predicate), occasions in sorted(groups.items()):
        if len(occasions) < JUDGEMENT_THRESHOLD (= 3): continue
        valence = mean(emotional_valence of occasions)
        strongest = 5 most important occasions
        if a judgement about (subject, predicate) exists:
            episodes += len(occasions)
            importance = min(0.95, importance + 0.05)
            emotional_valence = (existing + valence) / 2
            evidence = last 4 kept + 2 new strongest
        else:
            new Memory(type="judgement",
                      content=f"{subject} keeps being involved in {predicate} (n times)",
                      importance = min(0.9, 0.5 + 0.08 * n),
                      emotional_valence = valence,
                      about = (subject, predicate),
                      evidence = tuple(strongest ids))

```

The pattern looked for is deliberately narrow: the same predicate, about the same person, on several separate occasions. That is the shape a character actually generalises from — *"he keeps doing this"* — and the one that can be stated without

inventing anything. Two is coincidence; three is the threshold.

The judgement keeps pointers to the episodes it came from. A conclusion that cannot name its evidence is a prejudice, and the inspector has to answer "why does she think that about him" after the individual occasions are gone.

15.7 Bounded presentations

Every recall used to append a timestamp forever: 301 entries per memory after 300 retrievals, with activation and every save walking the whole list. Old presentations contribute $\Delta t^{(-d)}$, negligible next to recent ones, so a rolling window of 32 is a bounded and faithful approximation.

15.8 Parameters

Name	Default	Meaning
<code>tau_ret</code>	-1.5	retrieval threshold. Was 0.0, which with the corrected time scale meant "anything older than an hour is gone"
<code>noise_s</code>	0.4	logistic noise on the retrieval decision. It now decides: <code>retrieve</code> takes <code>seed_parts</code> and draws
<code>working_cap</code>	7	how many memories one retrieval returns
<code>w_topic</code>	1.0	cue spreading activation
<code>w_goal</code>	0.6	(declared; not read on the retrieval path)
<code>w_emotion</code>	0.5	emotional importance bonus
<code>w_mood_congruence</code>	0.3	mood-congruent recall bias
<code>episodic_cap</code>	200	
<code>total_cap</code>	600	
<code>summary_cap</code>	40	
<code>presentation_window</code>	32	
<code>interference</code>	0.3	(declared; the cue-competition penalty is not applied)

Two parameters — `w_goal` and `interference` — are declared and not read. Stated here rather than left for a reimplementer to discover by diffing behaviour.

16. Affect

ALMA-based: personality sets the PAD mood baseline, OCC appraisal produces discrete emotions, emotions push the PAD vector, mood is the slow PAD integral, emotions decay fast, and active emotions produce Frijda action tendencies.

16.1 State

```
AffectState:
    mood          : Vec3          # the slow integral
    baseline      : Vec3          # from personality; NEVER moved by play
    active_emotions : {key: intensity} # key = "emotion" or "emotion@target"
```

16.2 Personality → baseline

Traits in `[0,1]` are converted to `[-1,1]` as `2x - 1`, then:

```
P = 0.21*E + 0.59*A + 0.19*S
Ar = 0.15*O + 0.30*A - 0.57*(-S)      # instability raises arousal; S = -N
D = 0.25*O + 0.17*C + 0.60*E - 0.32*A
baseline = clamp_signed(P, Ar, D)
```

(O openness, C conscientiousness, E extraversion, A agreeableness, S emotional stability.) The coefficients are ALMA calibration defaults and are documented as such — verify against ALMA before freezing.

16.3 Appraisal → emotions (the OCC subset)

Input dict may carry: `desirability [-1,1]`, `agency ∈ {self, other, none}`, `praiseworthiness [-1,1]`, `expectedness [0,1]`, `prospect ∈ {confirmed, disconfirmed, None}`, `attraction [-1,1]`, `prior_fear`, `prior_hope`, `target`.

```
uncertainty = 1 - expectedness

prospect == "confirmed"    and prior_fear → relief      (prior_fear)
prospect == "disconfirmed" and prior_hope → disappointment (prior_hope)

desirability given:
    prospect is None and uncertainty > 0.3 and des > 0 → hope (des * uncertainty)
    prospect is None and uncertainty > 0.3 and des < 0 → fear (-des * uncertainty)
    des > 0 → joy      (des)
    des < 0 → distress (-des)

praiseworthiness given and agency == "self" → pride / shame      (|praise|)
praiseworthiness given and agency == "other" → admiration / reproach(|praise|)

both given and agency == "other":
    des > 0 and praise > 0 → gratitude (min(1, des*praise + 0.2))
    des < 0 and praise < 0 → anger      (min(1, (-des)*(-praise) + 0.2))

attraction given → liking / disliking (|attraction|)

merge duplicates by max, clamp to [0,1]
```

16.4 Emotions → PAD, and the update

EMOTION_PAD (Mehrabian octants):

joy, gratification	(1.0, 0.3, 0.3)
hope, relief	(1.0, 0.2, 0.1)
pride	(1.0, 0.4, 0.6)
admiration, gratitude,	
liking	(1.0, 0.2, -0.1)
distress, disappointment	(-1.0, -0.2, -0.4)
shame, remorse	(-1.0, -0.1, -0.6)
fear	(-1.0, 0.6, -0.7)
anger, reproach	(-1.0, 0.6, 0.5)
disliking	(-0.6, 0.1, 0.1)

```
update(state, appraisal, dt):
    for (emo, intensity) in appraise(appraisal):
        key = emo if target is None else f"{emo}@{target}"
        state.active_emotions[key] = clamp01(max(existing, intensity))
        impulse += EMOTION_PAD[emo] * intensity * k_impulse

    # emotions decay fast
    active = { e: v*exp(-dt/tau_emotion) for e,v in active if v' >= epsilon }

    # mood relaxes toward baseline (exact exponential; t_half = tau*ln2)
    homing = 1 - exp(-dt / tau_mood)      (0 if dt <= 0)
    relaxed = mood + (baseline - mood) * homing

    # soft saturation, PER AXIS
    soft = ( impulse.p * (1 - |relaxed.p|),
             impulse.a * (1 - |relaxed.a|),
             impulse.d * (1 - |relaxed.d|) )
    mood = clamp_signed(relaxed + soft)
```

Target-aware emotion keys. `anger@player` is not `anger@police`. Without the suffix a character angry at one person spoke angrily to everybody.

Soft saturation is what stops one strong event slamming an axis to ± 1 : the impulse pushes less the closer mood already is to the rail, so mood approaches it asymptotically.

Decay is composable. $\exp(-dt/\tau)$ over one long interval equals many short ones, which is what makes population level-of-detail *exact* rather than approximate.

16.5 Action tendencies (Frijda)

```
bias[class] =  $\Sigma$  over active emotions of intensity * ACTION_TENDENCIES[base][class]
               clamped to [-1, 1]
```

Emotion	Tendencies
fear	flee 1.0, hide 0.8, seek_safety 0.8, comply 0.6, evade 0.6, approach -0.7, attack -0.8
anger	attack 1.0, threaten 0.9, confront 0.8, comply -0.6, affiliate -0.7
joy	affiliate 0.9, cooperate 0.8, help 0.7, share 0.7
gratitude	help 0.9, affiliate 0.7, cooperate 0.6
distress	withdraw 0.8, wait 0.5, initiate -0.6
pride	assert 0.8, display 0.7, lead 0.6
shame	conceal 0.9, withdraw 0.8, repair_face 0.7, evade 0.5
liking	approach 0.8, help 0.6
disliking	avoid 0.7, sanction 0.6
reproach	sanction 0.7, avoid 0.5

This is a **bias** on the policy score, added rather than averaged — see §29.

16.6 Derived

```
stress(state) = clamp01(mood.a * max(0, -mood.p))
emotions_toward(state, target) = {base: v for keys ending in "@target"}
```

16.7 Parameters

Name	Default	Meaning
k_impulse	0.6	how hard an emotion pushes PAD
tau_mood	240.0 min	mood relaxation constant (half-life $\approx$ 166 min)
tau_emotion	30.0 min	emotion decay constant
epsilon	0.05	intensity below which an emotion is dropped

tau_mood and **tau_emotion** must be > 0 — they divide.

17. Perception

Who could perceive the event, and how well.

```

filter(event, world):
    place = world.places[event.location]
    noise = place.noise_level          # [0,1]

    if event.type == "broadcast":
        channel = payload.channel
        for agent in world.agents:
            exposure = world.media_exposure[(agent.id, channel)]
            if exposure:
                quality = max(0, exposure.attention)          # default 0.5
                emit Observation(agent, modality="media", quality)
        return

    audience = set(payload.audience) if payload has "audience" else None

    for agent in world.occupants(event.location):          # INDEXED, not a scan
        if agent.id == event.actor: continue
        if audience is not None and agent.id not in audience: continue
        seed = derive_seed(global_seed, agent.id, event.event_id,
                           world_time, "perception")
        jitter = (seeded_uniform(seed) - 0.5) * 0.1
        quality = max(0, base_quality * (1 - noise) + jitter)
        modality= "hearing" if event.type in (claim, promise, threat, question)
            else "sight"
        emit Observation(agent, modality, quality)

```

`base_quality = 0.95`.

Two properties worth preserving:

1. **An explicit audience bounds earshot.** A word between two people in a crowded square is heard by those nearby, not by everyone in the square. Without it, every retelling was perceived by the entire population of a place — which is both wrong and quadratic in crowd size.
2. **world.occupants(place) is an index, not a scan.** Perception used to walk the whole cast per event. With hundreds of characters that is quadratic — more people produce more events *and* make each event more expensive to route.

18. Interpretation

Seeing is not understanding. Three stages sit between an observation and a belief, and each can end the chain in a different state.

18.1 Attention — did they notice?

```
attention(agent, event, proposition, world, quality):
    score = 0.35 + 0.5 * payload.importance          (importance default 0.5)
    if proposition is not None:
        about = {str(v) for v in proposition.slots.values()}
        if agent.id in about:                        score += 0.5
        elif about & set(agent.relationships):       score += familiarity_weight (0.35)
        domain = world.predicate_domains.get(predicate, "street")
        competence = agent.competence_in(domain)
        if competence >= 0.5:                        score += role_weight (0.40) * competence
    curiosity = agent.trait("curiosity")              # default 0.5
    if curiosity != 0.5:                             score += 0.4 * (curiosity - 0.5)
    score *= max(0.1, quality)
    return clamp(score, 0, 1.5)
```

Below `attention_floor = 0.18` the observer **forms no belief at all** — which is a different state from disbelieving, and the trace says so (`interpretation.unnoticed`).

The model is deliberately **additive and small**. This decides whether something is noticed, not what it means, and a model with many multiplied terms would make "noticed nothing" the common case.

18.2 Recognition — could they identify it?

```
recognises(agent, proposition, world):
    need = world.predicate_competence.get(predicate)  # authored per pack
    if not need: return (True, 1.0, "no expertise required")
    have = agent.competence_in(need.domain)
    return (have >= need.level - recognition_slack (0.15), have, detail)
```

The requirement is **world content**: *"you need to be an engineer to see that this was deliberate"* is a fact about a setting, not about an engine.

18.3 The pass

```
interpret(agent, event, proposition, world, quality, modality):
    score = attention(...)
    if score < attention_floor: return None      # took nothing in
    if proposition is None:      return None

    if modality in ("hearing", "media"):
        return proposition                    # see below

    if recognises(...): return proposition

    # out of their depth: they saw something, they could not say what
    bias    = agent.trait("confirmation_bias")    # default 0.5
    weights = { levelling: 1 - bias, assimilation: bias,
                sharpening: 0, inversion: 0 }
    outcome = distortion.distort(proposition, speaker=agent, world=world,
                                seed=..., weights=weights)
    return coarser proposition (or the original if nothing could be simplified)
```

Why words bypass recognition. Recognition is about *identifying* something you perceived, not about understanding a sentence. You can disbelieve a radio report that a seal failed in Stores; you do not mishear "Stores" as "the med bay". When a claim arrives in words, the speaker has already done the identifying, so competence decides whether you **believe** them — which the belief engine handles — and not what you took in.

Why the coarsening reuses distortion. A layman losing the technical detail *is* levelling, and a suspicious character reading an ambiguous scene as the thing they already fear *is* assimilation. Modelling them twice would let the two drift apart.

The consequence worth having: several characters witness one event and end up holding genuinely different claims about it, each first-hand, each traceable to the same origin.

19. Distortion

How a claim changes shape as it is passed along. Because a proposition is predicate + slots + polarity, each of Allport & Postman's processes has a precise structural meaning:

```
was_at(agent=agent:okada, place=place:clinic, when=last_night)

levelling    → was_at(agent=agent:okada, place=place:clinic, when=None)
sharpening   → economy:owes_amount(amount=300) becomes amount=450
assimilation → agent=agent:okada becomes <someone the teller actually knows>
inversion    → NOT was_at(...)
```

19.1 Weights and constants

Process	Weight	Note
levelling	0.55	Allport & Postman's central finding: retelling mostly <i>loses</i> information
sharpening	0.20	quantities grow in the telling
assimilation	0.20	the story drifts toward the teller's own world
inversion	0.05	rare. A story rarely becomes its own opposite — which is exactly why the previous coin-flip model was wrong

`SHARPEN_FACTOR = 1.5`, applied once. Runaway exaggeration over several hops is produced by **repetition**, not by one large step.

19.2 Algorithm

```
distort(proposition, speaker, world, seed, weights=None, protected=set()):
    kind = weighted pick over (levelling, sharpening, assimilation, inversion)
    using derive_seed(seed.hex(), "kind", 0, 0, "distortion")
    for attempt, candidate in enumerate([kind] + the other three in fixed order):
        result = apply(candidate, proposition, speaker, world,
                       derive_seed(seed.hex(), candidate, attempt, 0, "distortion"))
        if result is None:
            continue # not applicable
        if nothing actually changed:
            continue
        if distorted.core_key() in protected:
            continue # would garble into a secret
        return (distorted, candidate, detail)
    return None
```

The retry over kinds matters: a proposition with one slot cannot be levelled, and one with no numbers cannot be sharpened, so a single draw would frequently produce nothing.

The protected-key guard is not decoration. A character must not leak a secret by accidentally garbling a *different* claim into it.

19.3 The four processes

```
levelling:      order = catalogue slot order
                droppable = [names after the FIRST whose value is not None]
                pick one seeded; set it to None
                # the first slot is the SUBJECT: losing it would not make the
                # story vaguer, it would make it about nobody

sharpening:     numeric slots (matching ^-?\d+([.]\d+)?$) → value * 1.5
                (integers stay integers; otherwise two decimals)
                else if degree is not None → min(1, degree * 1.5)
                else not applicable

assimilation:   for each slot valued "agent:": candidates = familiar agents
                for each slot valued "place:": candidates = familiar places
                pick a slot seeded, then a replacement seeded
                familiar agents = top 3 by (familiarity + liking), score > 0, excluding
                                the current value and the speaker
                familiar places = the speaker's own routine blocks in order, then their
                                current location; top 3, excluding the current value

inversion:      proposition.negated()
```

19.4 The two properties that make this epistemically interesting

1. **A distorted claim is a different proposition** with a different canonical key, so the listener forms a genuinely separate belief. A town can end up confidently believing something that never happened.
2. **The claim still carries the original origin.** It is one source, and that source is now wrong. Correlation discounting keeps working, so a distorted rumour repeated ten times is still one distorted rumour.

20. Diffusion

Who tells whom what, and it is emitted as an ordinary **claim** event so that perception, belief, memory and affect all apply unchanged. Three things fall out of that for free: **auditability** (every retelling is a ledger row), **bystanders** (whoever else is in the room overhears, weighted by noise), and **provenance** (the retelling carries the *original* origin).

20.1 The step

```
step(world, runtime, delta_minutes, player_id):
    hours = delta_minutes / 60
    emitted = 0
    for place_id, occupants in populated_places(world, player_id):    # >= 2 people
        g = place.gossip_factor (default 1.0)
        if g <= 0: continue
        g *= climate.modifiers(place).tempo                        # 1.0 when climate off

        expected = encounters_per_hour * g * hours * len(occupants)
        wanted    = min(int(expected), max_encounters_per_tick - emitted)
        frac      = expected - int(expected)
        if seeded_uniform(derive_seed(seed, place_id, world_time,
                                      len(occupants), "diffusion", "fraction")) < frac:
            wanted += 1
        if wanted <= 0: continue

        for index in range(wanted):
            speaker, listener = participants(world, occupants, place_id, index)
            tie = clamp01(0.7 * network.tie_strength(speaker, listener.id)
                          + 0.3 * network.bridge_score(speaker))
            if tie < min_tie: record diffusion.no_tie ; continue
            if _tell(...): emitted += 1
```

Encounters are sampled, not enumerated. A crowd of n has $n(n-1)$ ordered pairs; walking them made a tick quadratic — 200 co-located agents is ~40,000 candidate pairs per tick, a real cost even when almost all are rejected. People do not each hold a conversation with everyone present: **the number of conversations grows with the size of the room, not with its square.**

```
participants(world, occupants, place_id, index):
    speaker = occupants[ floor(u(seed(place, time, index, "speaker" )) * n) % n ]
    others  = occupants without speaker
    listener = others [ floor(u(seed(place, time, index, "listener")) * m) % m ]
```

`occupants` is **sorted by agent id** before use, so the draw is order-independent.

20.2 Tie strength

```
tie_strength(agent, other) = clamp01(0.5*familiarity + 0.5*liking)
bridge_score(agent)        = 0.8 if any role in
                             (role:bartender, role:fixer, role:trader) else 0.3
tie                        = clamp01(0.7*tie_strength + 0.3*bridge_score)
```

The bridge term is a stand-in: a pack with a real social graph would compute it from the graph. It is what puts service roles at the crossing points of a community.

20.3 What a speaker will pass on

```
tellable(speaker, world, threshold_shift):
    protected = protected_keys(speaker.secrets)
    floor     = min_confidence_to_tell + threshold_shift    # climate raises the bar
    for key, belief in speaker.beliefs:
        skip if key in protected
        confidence = max(p, 1 - p)
        skip if confidence < floor
        skip if not belief.spreading           # they lost interest in this one
        skip if belief.hops >= max_hops       # safety guard only
        recency = 1 / (1 + max(0, world_time - belief.txn_time) / 240)
        score   = confidence * (0.4 + 0.6 * recency)
    sort by (-score, key)
```

The pick is **seeded among the top three**, so a speaker does not repeat the single most salient item to everybody they meet.

Note the sort key includes the proposition key as a tiebreaker: floats compare equal often enough that a stable tiebreak is required for determinism.

20.4 Delivering one retelling

```
_deliver(world, runtime, speaker, listener, place_id, key, belief, medium=None):

1. COMMON KNOWLEDGE GATE (categorical)
    if common_knowledge.is_common(key, speaker.id, listener.id):
        record common_knowledge.not_news ; return False
        # both were there when it became public: there is nothing left to tell

2. TOPOLOGY GATE
    weight, (code, detail) = topology.may_pass_on(world, proposition,
                                                speaker, listener, place_id)

    if weight <= 0: record code ; return False          # hard wall
    if seeded_uniform(seed(..., "topology")) > weight:
        record diffusion.not_worth_telling ; return False

3. DISTORTION
    listener_knew      = key in listener.beliefs
    asserted_positive = belief.expected_prob >= 0.5
    p_distort = distortion_prob * (medium.distortion_scale() if medium else 1.0)
    if seeded_uniform(seed(..., "distort")) < p_distort:
        outcome = distortion.distort(belief.proposition, speaker, world,
                                    seed(..., "distort-kind"),
                                    weights=distortion_weights,
                                    protected=protected_keys(speaker.secrets))
        if outcome: told, kind, detail = outcome
        if kind == inversion: asserted_positive = not asserted_positive

4. THE EVENT
    origin  = belief.primary_origin or f"origin:{speaker.id}:{key}"
    hops    = belief.hops + 1
    audience = _audience(...) if medium is None or medium.audience else []
    strength = fidelity_decay ** hops
    if medium: strength *= medium.trust_factor
    emit Event(type="claim", actor=speaker, location=place_id, payload={
        proposition: told, asserter_polarity: "+"/"-", origin_event: origin,
        importance: 0.35, domain: "gossip", audience: audience,
        medium: medium.name or "in_person",
        assertion_strength: round(strength, 4),
        diffusion: {speaker, listener, hops, distorted, distortion_kind,
                    distortion_detail, fidelity} })
    runtime.process_event(event)

5. STIFLING (Daley & Kendall 1964)
    if listener_knew and seeded_uniform(seed(..., "stifle")) < stiffling_prob:
        belief.spreading = False
```

6. CONTAGION

```
contagion.between(speaker, listener)      # a little of the mood moves
```

```
_audience(world, speaker, listener, place_id):
    others = sorted(ids at place_id excluding speaker and listener)
    limit = bystanders (3)
    start = floor(seeded_uniform(seed(..., "audience")) * len(others))
    nearby = [others[(start + i) % len(others)] for i in range(min(limit, len(others)))]
    return [listener.id, *nearby]
```

20.5 Fidelity decay

`strength = fidelity_decay ** hops` is passed as `assertion_strength` and **multiplies effective trust** in the belief update. A fifth-hand rumour genuinely persuades less than a first-hand one.

This was for a long time computed, written into the payload, and read by nothing — a documented parameter with no effect. It is the kind of defect a reimplementation should test for explicitly.

Calibrated against serial reproduction: roughly two thirds of detail is gone after five or six retellings, which `0.80` per hop reproduces ($0.80^5 = 0.33$).

20.6 Parameters

Name	Module default	Meaning
<code>encounters_per_hour</code>	0.35	chance a given co-located connected pair talks in one hour
<code>max_encounters_per_tick</code>	48	hard ceiling so a crowded scene cannot make a tick quadratic
<code>min_confidence_to_tell</code>	0.60	a speaker passes on only what they believe
<code>min_tie</code>	0.05	below this people do not chat unprompted
<code>fidelity_decay</code>	0.80	conviction surviving one hop
<code>distortion_prob</code>	0.18	chance a retelling changes the claim
<code>distortion_weights</code>	null	override the four process weights
<code>stifle_prob</code>	1.0	Daley–Kendall stifling
<code>include_player</code>	false	whether NPCs gossip to the player unprompted
<code>max_hops</code>	12	safety guard only; termination should come from stifling
<code>bystanders</code>	3	how many people overhear one exchange

Packs override these. `relay-station`, for example, ships `encounters_per_hour = 0.45`, `distortion_prob = 0.16`.

`encounters_per_hour` sets pace, not reach. Reach is set by topology and by stifling. Turning the rate up makes the same saturation arrive sooner; it does not make a claim cross a wall it could not cross.

21. Topology: where information stops

Diffusion answers *how* information moves. This answers *where it stops*, which in a city-sized cast matters more. Left to co-location and trust alone, a claim eventually reaches anyone who ever shares a room with anyone — slowly, and a long-running world has plenty of time.

21.1 Knowledge is not a hierarchy

A street vendor knows more about the local gang than a corporate director does. Rank is not a knowledge ordering, so a single pyramid would be wrong. A claim is classified on several **independent** axes and a listener is reachable on some and not others.

```
TRANSMISSIBILITY = { public 1.00, common 0.85, local 0.60,  
                      trade 0.45, restricted 0.20, secret 0.05 }  
DEFAULT_CLASS = "common"
```

`trade` is shop talk: passed on readily, but only to those who care.

21.2 Circles

```
circles(agent) = { every networks entry }  
                  u { role["role"], role["context"] for each role }  
                  u { identity["group"] for each identity }
```

One flat set on purpose. A claim circulating among engineers and one circulating in the night shift are the same kind of restriction; separate namespaces would only mean writing the same rule twice.

`bridges(world)` = characters in more than one circle. Those are the people through whom anything crosses between groups — and "who could this have reached" is a question a designer asks constantly.

21.3 The gate

```
may_pass_on(world, proposition, speaker, listener, place_id) -> (weight, reason)

k = class_for(world, proposition)

if k.locality and place_id not in k.locality:
    return 0.0, topology.wrong_place          # HARD
if k.access and not (set(k.access) & circles(listener)):
    return 0.0, topology.outside_the_circle   # HARD

weight = k.transmissibility
if shared_circles(speaker, listener):
    weight = min(1.0, weight * 1.35)
if k.domain and k.domain != "street":
    interest = listener.competence_in(k.domain)
    weight *= 0.4 + 0.6 * min(1.0, interest * 2.0)
return clamp01(weight), topology.transmissibility
```

The result is a **multiplier**, not a yes/no, because every one of these is a tendency — people do leak restricted things, and a model where they never do is not a model of people. Zero is reserved for the two cases that really are categorical.

Access restricts circulation, never perception. Seeing a thing you have no clearance for is exactly how interesting trouble starts.

Measured on one crew, one speaker, one room, one day: a public claim reaches **all nine**; the same speaker passing on something restricted to the engineers reaches **two** and is walled off from **seven** — not slowed, walled. Ten of the cast stand in more than one circle, and those are the people through whom anything crosses at all.

21.4 Attention is finite

```
attention_budget(agent) = max(1, round(6.0 * (0.5 + agent.trait("curiosity"))))
                        # 6 at neutral curiosity

spend_attention(agent, world):
    today = world_time // 1440
    if agent.attention_day != today:
        agent.attention_day = today ; agent.attention_spent = 0
    if agent.attention_spent >= attention_budget(agent): return False
    agent.attention_spent += 1 ; return True
```

Day-bounded rather than continuous, because attention recovers by sleeping and because a per-day figure is something an author can reason about. Only **new** claims cost attention.

This is the third brake, and it is what stops a long-lived world becoming a single shared memory.

22. Medium and notes

22.1 Media richness, in three numbers

Medium	fidelity	audience	needs co-location	trust factor
in_person	1.00	yes	yes	1.00
phone	0.72	no	no	0.88
note	0.88	no	no	0.78

```
distortion_scale() = 2.0 - clamp01(fidelity)
    in_person 1.00    phone 1.28    note 1.12
```

Each number lands on machinery that already exists:

- **fidelity** → the probability the retelling **levels**, which is the process Allport & Postman measured;
- **audience** → whether bystanders exist. **Nobody overhears a call, which is exactly why a call is where a secret goes;**
- **trust_factor** → the assertion strength the listener weighs, because a voice with no face behind it is harder to judge and everybody knows it.

22.2 Who can reach whom

```
contacts_of(agent) = agent.contacts if authored
                    = {other : relationship.familiarity >= 0.45} otherwise

for_exchange(speaker, listener, enabled):
    if not enabled: return IN_PERSON if same location else None
    if same location: return IN_PERSON
    if listener.id in contacts_of(speaker): return PHONE
    return None
```

The familiarity fallback is a **default, not a claim**: a pack that cares about who can reach whom should say so, and a pack that does not still gets a plausible small-world graph rather than a fully connected one.

22.3 Notes

A note is not a slower phone call. It differs in one way no other channel does: **it is still there afterwards**, so it can be found by the wrong person, and it is the runtime's first piece of *evidence* rather than testimony.

```

leave(world, author, intended_for, place_id, key, belief):
    polarity = "+" if p >= 0.5 else "-"
    if seeded_uniform(seed(..., "write")) < 0.18 * NOTE.distortion_scale():    # 0.2016
        distort once, here, at write time
    store Note{author, intended_for, place, key, proposition, polarity,
               origin, hops, written_at, distorted_as, read_by}
    evict oldest beyond max_per_place

```

Distortion happens once, at writing. Whatever is going to be lost — the tone, the hesitation, the thing the author decided not to put in writing — is lost when it is written, and after that the words are fixed. A note that says the wrong thing goes on saying exactly the wrong thing to everybody who reads it, which is the difference between a rumour and a document.

```

step(world, runtime, delta):
    hours = delta / 60
    for each place, for each note:
        if world_time - written_at > lifetime_minutes: drop it (note.went_unread)
        for each reader present:
            if reader is intended_for:                deliver
            elif seeded_chance < intercept_base * hours * curiosity_factor:
                                                         deliver (once per note)

```

Parameter	Default
max_per_place	12
lifetime_minutes	7,200 (five days)
intercept_base	0.02 per hour

`intercept_base` is deliberately low and interception is capped at one per note. At the first value tried, **more notes were intercepted than delivered over a week**, which made being read by a stranger the normal fate of a letter rather than the thing that goes wrong.

23. Climate

Not who anyone is, but what it is like **here**. Characters' traits are fixed — a personality that drifts under play is how every character in a long game converges on the same one — but a *place* can change its mind about strangers.

23.1 State

Three dispositions per place, each in `[0, 1]` :

candour	how readily what is known here gets passed on
suspicion	how much a claim is discounted for coming from outside
openness	how readily a stranger is spoken to at all

23.2 Baseline, derived from what packs already declare

```
authored "climate" block wins. Otherwise:
    candour    = clamp01(0.30 + 0.30*gossip_factor - 0.25*surveillance_level)
    suspicion  = clamp01(0.35 + 0.35*surveillance_level)
    openness   = clamp01(0.65 - 0.45*privacy_level)
```

so a gossipy market is candid, a place under surveillance is guarded, and a private room is closed to strangers — without anybody re-authoring a pack.

23.3 The four numbers it lends the rest of the runtime

```
NEUTRAL = { tempo: 1.0, tell_threshold: 0.0, skepticism: 0.0, stranger_trust: 0.0 }

tempo          = 1.0 + 0.45 * (candour    - 0.5) * 2      # ±45% encounter rate
tell_threshold = -0.12 * (candour    - 0.5) * 2          # ±0.12 on a 0.60 threshold
skepticism     = 0.18 * (suspicion - 0.5) * 2            # ±0.18
stranger_trust = 0.15 * (openness  - 0.5) * 2            # ±0.15
```

Expressed as deviations from neutral so a caller can apply them without knowing anything about this module, **and so that "off" is exactly the same arithmetic as "average"**.

23.4 Reactions

At full severity (`payload.severity` , else `importance` , else 0.5), deltas on (candour, suspicion, openness):

Event	Δ candour	Δ suspicion	Δ openness
attack	-0.22	+0.26	-0.24
crime	-0.18	+0.22	-0.20
threat	-0.14	+0.16	-0.18
faction_mobilize	-0.10	+0.14	-0.12
discredit	-0.04	+0.20	-0.06
promise	+0.06	-0.04	+0.10
ally_arrives	+0.04	+0.02	+0.06

These are keyed on the event types the **runtime actually emits** (`threat` , `crime`) rather than on the verbs a player types. Guessing the names here would have produced a layer that ran, reported, and never once moved.

23.5 Relaxation and spread

```
step(world, delta):
    relax = 1 - 0.5 ** ((delta/60) / HALF_LIFE_HOURS)      # 30 hours
    for each place: climate.blend(baseline, relax)

carried(from_place, to_place):      # applied when somebody actually walks
    to.blend(from, CARRIED_PER_ARRIVAL = 0.055)
    total carried in one step is capped at MAX_CARRIED_PER_STEP = 0.22
```

The mood travels with the traffic or it does not travel. An earlier version drifted every place toward its neighbours by the clock, which produced the same picture and modelled the wrong thing: two rooms joined by a door nobody uses would have converged anyway, and a room emptied by a curfew would have kept absorbing its neighbours' mood.

Measured: after a killing in the square, openness `0.605 → 0.197`. After striking somebody in the open, openness `0.56 → 0.40` and suspicion `0.56 → 0.77`, relaxing back over the following days.

This is the honest version of "culture spreads". It does not make anybody kinder or crueller; it changes the ambient conditions they read from where they are. Anything stronger would be personality drift wearing a different hat.

24. Contagion

When two characters exchange a claim, a little of the speaker's mood moves to the listener (Hatfield, Cacioppo & Rapson, 1994: automatic mimicry followed by convergence of feeling, below the level of anybody deciding anything).

```
between(speaker, listener):
    rel          = listener.relationships[speaker.id]
    familiarity   = rel.familiarity
    liking        = rel.liking
    tie           = max(0, familiarity + 0.35 * max(0, liking))
    if tie < min_tie (0.08): return

    stability     = listener.big_five.emotional_stability      # default 0.5
    weight        = transfer (0.14) * min(1, tie) * (1 - 0.6 * stability)

    for axis in (p, a, d):
        step = clamp((theirs - mine) * weight, ±max_step (0.22))
        if |step| >= 1e-4: listener.mood[axis] = clamp_signed(mine + step)
```

Three properties make it safe rather than a mood engine that eats the world:

- **damped by emotional stability** — a steady person catches less, which is what the trait is for;
- **weighted by how much the listener cares about the speaker** — a stranger's bad afternoon is not contagious and a friend's is;
- **capped per exchange and decaying like any other mood**, so a chain of conversations converges rather than running away.

Trust is deliberately not in the tie. You can catch a mood from somebody you do not believe.

It moves *mood*, never *personality*: the baseline is untouched. It stores nothing of its own, so there is no restore path and no schema bump — a save taken with the layer on loads with it off and simply stops transferring.

Measured: three quarters of mood transfers in a running world are second-hand.

25. Relationship, reputation, identity

These three are the **owners** of cross-cutting state. Everything else *proposes* deltas through typed keys; only these apply them.

25.1 Typed deltas

```
StateKey      { module, field, entity_id, subject_id?, dimension? }
ProposedDelta{ key, amount, reason, source_module }

trust_delta(owner, subject, amount, reason, src)
liking_delta(owner, subject, amount, reason, src)
relationship_delta(owner, subject, field, amount, reason, src)
reputation_delta(subject, audience, dimension, amount, reason, src)
identity_threat(owner, identity_id, amount, reason, src)
```

Argument order differs and it is easy to get backwards. `reputation_delta` takes (*subject, audience*) — who it is **about** first, whose view second — while `trust_delta` and `liking_delta` take (*owner, subject*).

25.2 RelationshipEngine — asymmetric trust kinetics

```
apply(agent, deltas):
    group deltas by (subject, field)
    for each group:
        cur = relationships[subject].get(field, 0.3 if field == "trust" else 0.0)
        pos = Σ amounts >= 0
        neg = Σ amounts < 0          # negative
        if field == "trust": new = cur + eta_up * pos + eta_down * neg
        else:                new = cur + eta_other * (pos + neg)
        new = clamp01(new)        if field in (trust, respect, fear, dependence)
                                clamp_signed(new) otherwise
```

Parameter	Default
<code>eta_up</code>	0.10
<code>eta_down</code>	0.40
<code>eta_other</code>	0.20

Slow up, fast down, and positive and negative applied **separately** so that a turn containing both does not silently net them out at one rate.

Measured: breaking a promise takes the person you owed from **0.30 trust to 0.055**; keeping it raises them to **0.415**.

25.3 ReputationEngine

```
apply(agent, deltas):
    for d in deltas:
        store = agent.reputation.setdefault(d.key.subject_id, {})
        dim = d.key.dimension or "general"
        store[dim] = clamp_signed(store.get(dim, 0.0) + d.amount)
```

Reputation is **per observer**: `agent.reputation` is *this* agent's view of others, which is what makes "four people end up holding *unreliable* about you, three of whom only watched" a representable state.

Implementation note for a reimplementer: the engine declares parameters `eta_image = 0.25`, `eta_rep = 0.10`, `decay = 0.001`, and the apply path does not read them — the delta is added directly and reputation does not decay. Reproduce that behaviour, not the parameter block.

25.4 IdentityEngine — which belonging is salient

```
salience(agent, context, threat_by_identity):
    for each identity i:
        fit[i] = w_acc*accessibility + w_comp*comparative_fit
                + w_norm*normative_fit + w_threat*threat[i]
    fit["_neutral"] = neutral_fit # 0.4: "no identity strongly salient"
    softmax with temperature beta: s = exp(beta*(fit - max)) / Σ
    if previous salience exists:
        s = inertia*prev + (1 - inertia)*s ; renormalise
    agent.identity_salience = s
    return argmax(s), s
```

Parameter	Default
<code>beta</code>	4.0
<code>neutral_fit</code>	0.4
<code>inertia</code>	0.5
<code>w_acc</code> , <code>w_comp</code> , <code>w_norm</code> , <code>w_threat</code>	1.0 each

The **neutral baseline** and the **inertia** are what prevent rapid identity switching. `w_threat` was multiplied by zero in every world until the runtime started producing identity threat per observer (§10.5) — a quarter of what decides which identity somebody leads with.

25.5 The social feature providers

These read state and emit value terms and proposals. They own nothing.

Engine	What it contributes
ValueEngine	which Schwartz values are live given the salient identity: <code>value = base * (1.0 if amplified by identity else 0.5)</code> . What an identity amplifies is world content (world.json: identity_values)
SocialExchangeEngine	on a resolved commitment: kept → trust +0.8, liking +0.4; broken → trust -0.7, liking -0.5, reputation reliable -0.4. Also <code>dependence(current, best_alt) = current / (current + best_alt + ε)</code>
SocialIdentityEngine	on an in-group threat: identity_threat +0.6, appraisal boost {desirability -0.6, agency other, praiseworthiness -0.3, target actor}
NormEngine	threaten → norm term <code>-(sanction 0.6 × observability × legitimacy)</code> ; share/affiliate → +0.1
SocialNetworkEngine	tie strength and bridge score (§20.2)
TheoryOfMindEngine	for threaten: tom term <code>0.4·fear - 0.4·(1 - fear)</code> . Works if they fear you; backfires if they do not
ImpressionManagementEngine	front stage when audience_size > 0 or surveillance > 0.3. Then threaten in a formal public role → face -0.4; affiliate/repair_face → +0.2. Group weight <code>1 + (0.8 front stage else -0.6)</code>
InterpersonalEngine	not constructed by the runtime. It computes an (agency, communion) stance that nothing consumes. Kept because the mapping is right and the socket is missing; stated here so a reimplementer does not wire a component with no consumer

26. Standing

You were seen doing something to somebody — what does that do to how you are treated afterwards?

26.1 Why it is driven by belief and not by witnessing

The obvious implementation loops over everyone in the room. That gives a reputation counter, which every game has had since 1997. This one instead attaches a proposition to the act — `mistreated(who=victim, by=actor)` — and lets the existing machinery carry it:

- whoever is in the room perceives it and forms a belief, with provenance;
- diffusion passes it on, it decays, and it can be doubted;
- standing then moves **in proportion to how much the hearer believes it**.

Three things fall out for free, and they are the whole difference:

1. Somebody who was not there can come to think badly of you, **because they were told**.
2. What you saw yourself weighs more than what you heard, because **certainty** scales the delta — not a hop count somebody tuned.
3. A character can **lie** about what you did, and the lie moves standing exactly as far as it is believed. No extra code: it is a claim like any other.

Constant	Value	Meaning
BASE_STEP	0.5	how far one believed act moves standing, before certainty and care
STRANGER_FLOOR	0.6	how much of the judgement survives when the hearer has no feeling about the victim. Cruelty to a stranger still counts — most of it does
MAX_STEP	0.35	a single event should not make somebody's mind up on its own; that is what repetition is for
FEAR_SHARE	0.8	of a believed cruelty, how much becomes wariness of you
RESPECT_SHARE	0.5	of a believed kindness, how much becomes standing rather than affection

Fear and respect had been read by four engines and written by none. `fear` decides whether a threat is expected to work or to backfire and pulls a character's stance away from warmth; `respect` is what lets somebody be deferred to without being liked. Both could be authored and then stayed frozen for the rest of a world's life. Conduct produces them now.

Note this is **wariness, not fright**. Acute fear is an emotion and lives in `affect`, where it decays over an afternoon; this is the part that does not.

Why only the change. Hearing about an act again is not a second act; it is the same one restated. Paying out the full judgement on every report made repetition a punishment the belief engine had already, correctly, refused to be persuaded by: twenty reports of one incident moved the belief from 0.54960677 to 0.54960683 and liking from -0.198 to -0.659. The ledger is keyed on the proposition, which is the granularity the belief engine files it under — two reports that are the same claim are one incident, and a pack that means two qualifies them apart. It is persisted, or a reload re-applies everything the save had already paid out.

Measured: six threats put fear at **0.208** and make a further threat expected to land at **+0.40**, against **0.00** for a stranger the actor has done nothing to and **-0.22** after six kindnesses — the estimate comes from what the actor has been seen to do, not from reading the other person's mind. Six kindnesses put respect at **0.091** and move how somebody speaks to you toward deference: more formal, more hedged, without being any warmer.

26.4 Reading it back — the half that makes the other half worth having

```
warmth(agent, other) = clamp_signed( 0.30 * liking
                                     + 0.20 * (trust - 0.3)/0.7      # centred on the
                                                                     # stranger default
                                     + 0.50 * decent                # reputation dimension
                                     + 0.25 * attraction )

bias_for(agent, action, other) = DECISION_PULL * WARMTH_PULL[action_id] * warmth
WARMTH_PULL = { answer_truthfully +1.0, greet +0.6, evade -1.0, threaten -0.8 }
DECISION_PULL = 0.4

influence(agent, other, bridge_score) =
    clamp01(0.35*respect + 0.30*dependence + 0.25*fear + 0.10*bridge_score)
```

Before `bias_for` existed, standing moved and **nothing read it when deciding what to say**: whether a character answered you depended on their agreeableness, their secrets and their safety, and not at all on what they thought of you. Standing nobody acts on is a number in a save file.

decent carries half the warmth, deliberately: it is the only one of the three that is about your **conduct** rather than your dealings with *this* person — it is what a bystander has, and what somebody who was *told* about you has. Weighting it below liking made the layer quietest in exactly the case it exists for.

DECISION_PULL = 0.4 is of the same order as the outcome values already in the action catalogue (evading is worth +0.3 safety, answering +0.2 relationship) and not larger. Standing should tip a close call and lose to a character's own judgement about their safety. A number that overrode that would produce an NPC who tells a stranger a secret because they were nice once — worse than not modelling this at all.

influence is **deliberately not liking**. Somebody you are fond of and neither respect nor need has no particular sway over you.

26.5 The honest limit

A standing does not heal on its own. Trust rises slowly, falls fast, and is then simply where it is. A reputation that quietly faded would let a player wait one out, and that is an author's decision rather than one this layer should make behind their back.

And this half needs the world to actually gossip: in a cast that rarely meets, the people who were standing there are the only ones who will ever know.

27. Appearance

How somebody is read **before they have said anything**. Two effects, because they are the two a player notices and they are not the same thing:

status	how seriously they are taken. Lands on perceived social status, which the register reads — so it changes how somebody speaks to you before they know anything about you
threat	how dangerous they seem. Gang colours are not low status; they are something else entirely, and that difference is the whole point of having two numbers

```

tokens(agent) = every string value in agent.appearance, sorted by key,
                  lists flattened          # garment first, then symbols

read(observer, agent, world):
    reactions = world.appearance_reactions
    if no reactions or no tokens: return zeros
    sensitivity = clamp01(observer.epistemic.appearance_sensitivity) # default 1.0
    status =  $\sum$  reactions[token].status ; threat =  $\sum$  reactions[token].threat
    signals = ordered unique reactions[token].signals
    return { status: clamp_signed(status) * sensitivity,
            threat: clamp01(threat) * sensitivity,
            signals, tokens }

drawn_to(observer, agent, world):
    wanted = set(observer.epistemic.drawn_to)          # AUTHORED, only authored
    hits    = wanted & set(tokens(agent))
    return min(1.0, 0.55 * sqrt(len(hits)))            # saturating

bias_for(observer, action, other, world):
    r = read(observer, other, world)
    pushed =  $\sum$  over (table, pull, amount) in
        ((THREAT_ACTIONS, 0.35, r.threat), (STATUS_ACTIONS, 0.15, r.status))
        of pull * table[action_id] * amount
    THREAT_ACTIONS = { evade +1.0, greet -0.7, answer_truthfully -0.5 }
    STATUS_ACTIONS = { answer_truthfully +1.0, greet +0.6 }

```

There is no engine here that decides what is attractive. `drawn_to` is authored per character, because that is a fact about a person and a setting and not about software. Familiarity is deliberately absent: being struck by a stranger across a room is the case worth modelling; growing fond of somebody over years is `liking`.

It does not change what anybody believes. Being read as dangerous makes a character wary of you; it does not make them think you *did* anything. Evidence comes from events, and a coat is not evidence.

Inert until a pack says otherwise. With no `appearance_reactions` every read is empty and a run reproduces byte-identically — which is why this is not an optional layer with a flag. Authored data that defaults to nothing needs no switch.

Measured, with a reactions table authored: a tailored suit reads **+0.45** on status and makes a character readier to answer; a worn jacket reads **-0.30** and they talk over you. Gang colours read **+0.35** on threat and signal the circle they belong to; wearing them moves wanting distance from you from **0.28 to 0.41**.

28. Sociolinguistics: the style vector

This is not a good-vs-bad speech ladder. It models register and prestige variation — formality, sentence length, lexical breadth, slang, politeness — plus accommodation toward the interlocutor and audience design. A low-status speaker is not "worse"; they use a different, fully competent variety. Education must not be equated with intelligence.

28.1 The vector

```
StyleVector:
  formality          [0,1]
  lexical_sophistication [0,1]
  mean_sentence_length target tokens (default 12)
  slang_level        [0,1]
  directness         [0,1]
  hedging            [0,1]
  dialect            str | None
  jargon_domain      str | None
  slang_lexicon      str | None
  politeness_strategy bald | positive | negative | off_record | neutral
```

28.2 Computation

```
edu          = agent.education["general"]          (default 0.35)
status       = social_status(agent)                (own view)
setting      = world.place_formality(place, DEFAULT 0.4)
role_formal  = 0.7 if any role in (bartender, officer, doctor) else 0.4
arousal      = mood.a ; neg = max(0, -mood.p) ; stress = clamp01(arousal * neg)
membership   = 1.0 if any identity id in ("gang_member",) else 0.0
ingroup_pull = membership * (1 - setting)           # code-switching

formality = clamp01( w_edu*edu + w_setting*setting + w_status*status
                    + w_role*role_formal
                    - w_arousal*stress - w_ingroup*ingroup_pull )

lexical     = clamp01(0.4*edu + 0.6*(edu * formality))
sent_len    = max(4, base_sentence_len + k_len*lexical*formality - k_stress*stress)
slang       = clamp01(membership * (1 - formality))
anger       = emotions_toward(affect, interlocutor)["anger"]
directness  = clamp01(0.5 + 0.3*(1 - agreeableness) - 0.3*formality + 0.4*anger)
hedging     = clamp01((1 - directness) * (0.4 + 0.4*formality))
```

Parameter	Default
w_edu	0.4
w_setting	0.3
w_status	0.2
w_role	0.2
w_arousal	0.4
w_ingroup	0.5
lambda_accommodation	0.4
base_sentence_len	8.0
k_len	12.0
k_stress	6.0

28.3 Perceived status

```

social_status(agent, observer=None, world, appearance):
    base = agent.social_status # [0,1]
    if observer is None: return clamp01(base)
    rep      = observer.reputation[agent.id]
    perceived = 0.6*base + 0.4*clamp01(0.5 + 0.5*rep.get("status", 0.0))
    if appearance and world:
        perceived += 0.25 * appearance.read(observer, agent, world).status
    return clamp01(perceived)

```

28.4 Accommodation (Giles) — a dial, not a switch

```

target_formality = 0.5 + 0.5 * social_status(interlocutor, observer=agent, ...)
direction = sign(liking) * min(1.0, |liking| * 2.0)
formality = clamp01(formality + lambda_accommodation
                    * (target_formality - formality) * direction)

```

This used to be `(1 if liking >= 0 else -1)`, a **cliff at exactly zero**: a stranger, whose liking is 0.000, got the full pull toward the other person's register, and one small unkindness later — liking -0.15 — got the full *push away* from it, taking formality from 0.568 to 0.097. A whole register collapsing on a fifteen-hundredth of a unit is not accommodation, and it saturated the vector so completely that nothing else added to it could be seen.

`min(1, 2|liking|)` reaches full strength at 0.5, which is where a relationship in this runtime is unmistakable.

28.5 Regard — how somebody speaks to a person they are wary of

Passed only when the standing layer is on (otherwise fear and respect never move, so it would be a constant that perturbs every line):

```

formality = clamp01(formality + 0.25*fear + 0.15*respect)
directness = clamp01(directness - 0.30*fear)
hedging = clamp01(hedging + 0.30*fear + 0.10*respect)
politeness = politeness(formality, directness) # recomputed

```

Wariness makes people careful: they hedge, they stop being blunt, and they reach for a more formal register. Respect makes them deferential **without** making them warm. Neither is expressible through liking, which is why this exists at all.

*The first attempt used the (agency, communion) pair from `InterpersonalEngine` and failed: communion is mostly liking again, and the accommodation step already consults liking, so the effect vanished under the existing one — measured at **0.007** on directness, which is decoration.*

28.6 Politeness strategy (Brown & Levinson)

```

formality > 0.7 and directness < 0.5 → "negative"    (deferential, indirect)
formality < 0.3 and directness > 0.6 → "bald"        (direct, no redress)
directness < 0.4                      → "off_record"
otherwise                              → "positive"

```

28.7 Register selection at realization

```

register = "formal"    if formality >= 0.6
          "vernacular" if formality <= 0.3
          "neutral"    otherwise

```

The realizer then looks up the authored phrasing for that register, applies slang substitution when `slang_level > 0.3`, and truncates to the first sentence when `mean_sentence_length < 7`.

One computation site. `style_for` exists because three call sites computed this independently — the deliberation path and two in the SDK — and adding fear and respect to one of them changed the register in a scene driven through `decide` and left it untouched in the two paths a game hits most.

29. Policy: choosing an action

Three roles kept strictly distinct: **additive value terms**, **multiplicative personality/identity weights**, and **pushes** (emotion tendencies and modifiers).

29.1 The action catalogue

Actions are drawn from a **closed catalogue**, never invented. Outcomes are explicit so the optional Prospect transform applies to outcomes rather than to arbitrary terms.

```

ActionOutcome { dimension, delta, probability, reference_point=0.0, source }
ActionDefinition { action_id, action_class, target, outcomes[],
                  preconditions[], is_dialogue, dialogue_act }

```

The reference catalogue for "a guarded NPC talking to a stranger":

action_id	class	act	outcomes
evade	evade	evade	safety +0.3 @ p 0.8
answer_truthfully	share	inform	relationship +0.2 @ 0.6; safety -0.4 @ 0.7
threaten	threaten	threaten	safety +0.2 @ 0.4; relationship -0.5 @ 0.8
greet	affiliate	greet	relationship +0.1 @ 0.9
ask_for_payment (if player_owes)	assert	request	income +0.4 @ 0.5
speculative_side_job (if allowed)	gamble	request	income +1.2 @ 0.55; safety -0.10 @ 0.30

A production pack supplies a larger affordance query. **A character must always have at least one thing they could do**, even if it is to say nothing — an empty candidate set raises with a message saying exactly that.

29.2 Scoring

```
score(agent, candidates, weights, term_providers, action_tendencies, bias_providers):
    for each candidate:
        groups = {}
        # 1. outcomes contribute to their own dimension
        for oc in candidate.outcomes:
            groups[oc.dimension].append(outcome_value(oc))
        # 2. module-provided value terms
        for (group, value) in term_providers(candidate):
            groups[group].append(value)
        # 3. bounded aggregation, then weighting
        total =  $\Sigma$  over groups of weights[group] * bounded_aggregate(values)
        # 4. emotion action tendency: a PUSH
        total += clamp_signed(action_tendencies[candidate.action_class])
        # 5. every other push: standing, appearance
        total += bias_providers(candidate)
    sort by utility descending

bounded_aggregate(values) = clamp_signed(mean(values))    # NOT the sum
outcome_value(oc)         = clamp_signed(oc.delta * oc.probability)
```

29.3 The difference between a term and a bias

This cost an afternoon and it is the most transferable lesson in the file.

A **term** is an opinion about an outcome, and terms in a group are **averaged** so a group stays in `[-1, 1]` however many there are. That is right for what it was built for and **wrong for a modifier**: a bias meant to make evading a little more attractive, contributed as a term worth +0.12 into a group already holding +0.30, **lowers** the group to +0.21.

The observable symptom: *somebody dressed to look dangerous made a character **less** likely to back away from them* — and every number moved the wrong way round while every individual piece of arithmetic was correct.

So a **bias is added**, exactly as the emotional action tendency always has been. That channel was the shape the newer modifiers needed, and it was already there.

29.4 Group weights (personality and identity)

```
exchange      = 1.0 + 0.5 * agreeableness
norm          = 1.0 + 0.6 * conscientiousness
relationship   = 1.0 + 0.5 * agreeableness
safety        = 1.0 + 0.6 * active_values["security"]      (default 0.4)
income        = 1.0
face          = 1.0 + (0.8 if front_stage else -0.6)
tom           = 1.0
```

29.5 Selection — Boltzmann with a personality-scaled temperature

```
select(agent, scored, event_id, world_time):
    if not scored: raise (with a message naming the affordance query)
    impulsivity = big_five["impulsivity"] or 1 - conscientiousness
    arousal     = mood.a
    T = max(0.05, temperature * (1 + w_impulsivity*impulsivity
                                + w_arousal*max(0, arousal)))
    exps = [exp((u - max_u) / T) for u in utilities]
    seed = derive_seed(agent._global_seed, agent.id, event_id, world_time, "policy")
    idx  = seeded_choice(seed, exps)
```

Parameter	Default
temperature	0.3
use_prospect	false
w_impulsivity	0.4
w_arousal	0.3

The trace keeps **three distinct facts** apart so that a stochastic pick never reads as a bug: the **highest-rated** action, the action actually **sampled**, and the **selection probability** of every candidate.

29.6 Prospect Theory (off by default)

```
prospect_value(x, ref, alpha=0.88, beta=0.88, lam=2.25):
    z = x - ref
    return z^alpha          if z >= 0
        -lam * (-z)^beta   otherwise

prospect_weight(p, gamma=0.61):
    0.0                      if p <= 0
    1.0                      if p >= 1
    p^gamma / ((p^gamma + (1-p)^gamma)^(1/gamma)) otherwise

outcome_value(oc) = clamp_signed(prospect_value(oc.delta, oc.reference_point)
                                * prospect_weight(oc.probability))
```

When on, the reference point is set per character from liquid resources (`risk_reference_point`), so *below* → *risk-seeking*, *above* → *risk-averse*.

29.7 Reactive rules preempt deliberation

```
ReactiveRule { rule_id, condition(agent, ctx) -> bool, action_id, priority, reason }

default rules:
    summon_kin_when_hurt    : injured AND threatened          → mobilize_allies, 2.0
    flee_when_outmatched    : threatened AND severity > 0.7
                           AND NOT has_backup                 → flee, 1.5

fire_reactive_rules(agent, ctx, rules, reasons):
    evaluate each; a rule that RAISES is reported as agency.rule_failed and
    skipped – the others still run
```

When a rule fires: the triggered action is ensured to exist as a candidate, its utility gets `+= priority`, and the selection temperature drops to **0.05** for that decision only.

The temperature restore is in a `finally`. Without it, an exception during selection left the temperature pinned at 0.05 for the rest of the process, quietly turning every later decision near-deterministic.

A rule that throws is reported, not swallowed. Rules come from packs and from studios, so they *will* have bugs; a designer looking at a character who does not react to being attacked needs something to go on.

30. Agency, allies and leverage

Social-power primitives the deliberative layer lacked.

```

authority(caller, ally):
    status_gap = clamp01((caller.social_status - ally.social_status) + 0.5)
    rank       = 0.4 if same concrete group and caller holds
                    role:boss / role:officer / role:fixer else 0.0
    return clamp01(0.6 * status_gap + rank)

leverage(a, b):
    dep         = b.relationships[a.id].dependence
    holds_secret = 1.0 if any secret of a has b.id in secret.about else 0.0
    res_control  = clamp01(a.resources - b.resources + 0.3)
    status_gap   = clamp01(a.social_status - b.social_status)
    return clamp01(0.4*dep + 0.3*holds_secret + 0.2*res_control + 0.1*status_gap)

```

`holds_secret` reads the secret's **structured** `about` field. It used to be a substring search for the target's id inside the secret's text.

30.1 Mobilisation

```

potential_allies(caller, world):
    reachable if kinship > 0
                or loyalty (else liking) > 0.3
                or same concrete group
                or (ally holds role:officer AND caller.social_status >= 0.6)
    # status GATES institutional reach

ally_response(caller, ally, world, urgency, event_id):
    kin  = rel.kinship ; loyal = rel.loyalty or rel.liking
    favor = rel.favor_debt or rel.owes_favor
    auth  = authority(caller, ally)
    cost  = 0.5 * (1 - ally.agreeableness)
    x     = 2.0*loyal + 2.5*kin*(0.5 + 0.5*urgency) + 1.5*favor + 1.2*auth - 1.0*cost
    prob  = sigmoid(x - 1.0)
    will  = seeded_uniform(derive_seed(seed, ally.id, event_id, world_time,
                                        "ally_response", caller.id)) < prob
    latency = 2 minutes if co-located else 15

mobilization_candidate(caller, world, urgency, event_id):
    expected = Σ probabilities
    reach    = 0.7 + 0.6 * caller.social_status
    safety    = clamp01(0.2 * expected * reach)
    → ActionDefinition("mobilize_allies", "seek_safety",
                      outcomes=[ActionOutcome("safety", +safety,
                                              min(1, 0.4 + 0.4*expected))])

```

`Runtime.mobilize(caller)` resolves responses and schedules an `ally_arrives` event per responder at `world_time + latency`.

31. Factions and the Director

The macro layer: *the society develops even when nobody is watching*. Progress clocks and heat, after *Blades in the Dark*.

```
ProgressClock { name, size, filled } # tick(n) returns True on completion
Faction      { faction_id, influence [0,1], resources [0,1], heat [0,1],
              territory[], goals[], clocks{} }
```

31.1 Heat

Raised on the event path:

```
on threat / attack / crime with a location:
    faction = the one whose territory contains the place
    heat += 0.25 * |severity| * (0.5 + surveillance_level)
```

31.2 The step

```
step(world, delta, schedule):
    for each faction:
        heat = clamp01(heat - heat_decay * (delta / 60))

        rate = clock_base_rate * (0.5 + influence) * (0.5 + resources)
        for each goal:
            clock = clocks.setdefault(goal, ProgressClock(goal, size=6))
            if seeded_uniform(derive_seed(seed, faction_id, goal,
                                         world_time, "clocktick")) < rate:
                completed = clock.tick(1)
                report director.clock (+ director.clock_complete)

        if heat >= heat_threshold:
            latency = int(mobilize_latency * (1.5 - influence)) # influence → faster
            schedule a faction_mobilize event at world_time + latency
            heat = clamp01(heat - 0.4) # mobilising discharges
```

Parameter	Default
heat_decay	0.02 per hour
heat_threshold	0.6
clock_base_rate	0.15
mobilize_latency	20 minutes
clock size	6 segments

Social standing matters here: influence scales how fast clocks fill *and* how quickly a faction mobilises. A `faction_mobilize` event returns early from the event path — it is narrated and discharges threat rather than asserting anything.

32. Routines

What makes a cast a society rather than a tableau.

Before routines existed, characters never moved: whoever a pack placed somewhere stood there for the rest of the session. In two of three reference packs **no two characters ever shared a place**, so the diffusion layer could not fire at all — and the benchmark honestly reported it as SKIP rather than as a pass, which is how it surfaced. A social simulation whose social graph is frozen at load time cannot produce the behaviour it is sold on.

32.1 The model

Deliberately the smallest one that buys the most: **a schedule of places by time of day**, authored per character. Not pathfinding, not needs-driven planning, not GOAP.

```
"routine": [  
  { "from": "06:00", "place": "place:schmiede", "activity": "arbeiten" },  
  { "from": "12:00", "place": "place:dorfplatz", "activity": "essen" },  
  { "from": "19:00", "place": "place:schmiede", "activity": "schlafen" }  
]
```

```
parse_clock("07:30") = 450          # minutes past midnight; plain ints accepted  
block_at(world_time) = the last block whose "from" <= (world_time mod 1440),  
                        wrapping past midnight
```

A block runs until the next one starts. A character with no routine stays put.

32.2 The step

```
step(world, runtime, delta):
    for each agent with a routine:
        target = block_at(world_time).place
        if agent.location == target: continue
        if action_bridge enabled:
            issue a MOVE_TO intent; do NOT move the agent
            (routine.move_requested)
        else:
            perform_move(world, runtime, agent, target, activity)

perform_move(...):
    agent.location = place          # bumps the location epoch
    if emit_events:
        emit Event(type="move", actor=agent, location=place,
                    payload={ importance: arrival_importance })
```

Parameter	Default
<code>emit_events</code>	true
<code>arrival_importance</code>	0.15

Movement is a **real event**, so an arrival is perceived, remembered and on the ledger: *"who was in the bar at 21:00"* is a query. Importance is low so arrivals consolidate away before anything that matters to a character.

32.3 The action bridge

Everything the runtime decides is otherwise a belief or a sentence, both cheap to be wrong about because nothing visible depends on them. **Movement is not:** if the routine layer sets `agent.location` directly and the engine renders the character somewhere else, there are now two worlds.

With `action_bridge` on, the runtime **issues an intent** and waits:

```
ActionIntent { intent_id, actor, type: "MOVE_TO", params, issued_at }
report_action_result(intent_id, status, detail)
    status SUCCEEDED → apply the effect (perform_move)
    anything else    → record that it did not happen; the character stays put
expire(world_time) → intents older than action_timeout_minutes become TIMED_OUT
```

The runtime is not the authority on whether a character crossed a room — **the engine is, because the engine is what the player watched.**

33. Pursuit

Diffusion is who happens to talk. Pursuit is who has a reason to. The same act — one character asserting something to another — chosen by a different question: diffusion asks *who is here*, pursuit asks *who needs this said, and to whom*.

The runtime's own evidence run is the argument for it:

After saturation, little happens. The rumours are through the population within days, everyone becomes a stifler, and the world goes quiet — across 83,245 simulated days the long case demonstrates **stability, not liveliness**.

33.1 Who is due

```
_due(agent, world_time):
    priority = max(goal.priority for goal in agent.goals)      (default 0.5)
    interval = interval_minutes / max(0.05, priority)
    return world_time - last_acted[agent.id] >= interval
```

Priority 1.0 acts every three hours; priority 0.5 every six. **Deliberately slow**: a cast that acts on its goals every twenty minutes is not alive, it is frantic, and it saturates the world faster than the diffusion layer it complements.

33.2 What they raise

```
_wants(agent) = union of keywords(goal["about"] entries) and keywords(goal.content)
                  (stopword-filtered, words longer than 2)

_relevant(agent, wanted):
    best over beliefs where confidence >= min_confidence:
        overlap = |wanted n keywords(belief_key)|
        skip if overlap == 0
        score = overlap + confidence          # RELEVANCE FIRST, conviction second
```

Somebody trying not to be arrested raises the thing that bears on being arrested, not the thing they happen to be surest about.

A goal may name its subject precisely with `"about": [...]`. Where it does not, the words of the goal are used — imprecise, and better than a layer that only works for packs written after it.

33.3 Whom they tell

```
_listener(agent, occupants, world):
    here = the person present they trust most who they can reach
    if media off: return here
    away = the person elsewhere they trust most whose number they have
    if trust(away) >= trust(here) + worth_a_call: return away by PHONE
    return here
```

The phone is a fallback, not a preference. Ranking every contact in the world against the room turned five days into **ninety-eight phone calls and thirty-four conversations**, which is not a market square. `worth_a_call = 0.12` is a *margin*, not a floor: the default trust between two people who have never met is 0.3, so any absolute floor above that sent everybody to the phone.

If they can reach nobody and the notes layer is on and the margin `worth_writing = 0.20` is met, they leave a note instead.

The delivery itself goes through `diffusion.tell_about(...)` — the same topology gate, distortion, claim event, earshot, provenance and hop count. **A purposeful conversation must not be a second kind of conversation.**

Parameter	Default
<code>interval_minutes</code>	180.0
<code>min_confidence</code>	0.55
<code>max_per_tick</code>	12
<code>worth_a_call</code>	0.12
<code>worth_writing</code>	0.20

34. Common knowledge

Everybody knowing is not the same as everybody knowing that everybody knows.

```
private    I know it.
mutual     We all know it – and I have no idea whether you do.
common     We all know it, we all know we all know, and so on without end.
```

Lewis (1969) and Aumann (1976) give the definition; Chwe (2001) gives the mechanism, and that is the part a game can use: **common knowledge is not produced by more people finding out. It is produced by a public event** — a broadcast, an announcement, something said in the open before a crowd — because what makes it public is that everybody there can see everybody else taking it in. A thing whispered to fifty people one at a time is mutual knowledge fifty times over and common knowledge to nobody.

34.1 Recording

```
observe(world, event, learned):          # learned = [(agent_id, quality)] from perception
    proposition or return; skip questions
    how = _how_public(world, event) or return
    members = { id for (id, q) in learned if q >= min_quality }
    members |= { event.actor } if it is an agent      # the speaker knows it is out
    if len(members) < min_witnesses: return
    record Public(key=proposition.core_key(), members=frozenset(members),
                  since=world_time, how=how, place=event.location)

_how_public(world, event):
    "broadcast"    if event.type == "broadcast"      # Chwe's canonical case
    None           if payload.medium in (phone, note) # no crowd can see itself
    None           if place.privacy_level > public_privacy
    "in_the_open"  otherwise
```

34.2 Publics do not merge

```
_record(public):  
    if public.members  $\subseteq$  any existing crowd for this key: nothing new  
    drop existing crowds that are subsets of this one      # one moment, not two  
    append; keep at most MAX_PUBLICS_PER_KEY (6)
```

Two separate public moments about the same fact produce two publics, not one merged crowd. Somebody who was in the square on Tuesday does not thereby know about the people who were in the bar on Wednesday. Overlapping subsets are absorbed; distinct crowds stay distinct.

34.3 What it changes

```
is_common(key, *agent_ids) = some ONE crowd contains all of them
```

Two consequences, both about **what may be done**, never about what is believed:

1. **Nobody passes on what is already common knowledge between them.** A categorical stop in diffusion, distinct from the Daley–Kendall coin: that one models a spreader losing interest, this one models there being nothing left to tell.
2. **A secret that has become common knowledge in a character's own community stops working as a secret.** Concealing what everybody knows everybody knows is not concealment; refusing to discuss it protects nothing and only marks you as the person refusing. It does *not* make the character believe anything new and does not oblige them to be helpful — it removes the grounds for stonewalling.

It does not touch the belief maths. A public event does not make anybody surer: it is one origin, weighed once. Treating publicity as evidence would be the same error as treating repetition as proof.

Parameter	Default	Why
<code>min_witnesses</code>	3	two people talking already know that they both know; that is mutual knowledge and it is what a conversation is. Chwe's argument is about an audience, so three is the smallest honest number
<code>public_privacy</code>	0.35	packs already declare <code>privacy_level</code>
<code>min_quality</code>	0.5	half catching something is not what a public event means
<code>MAX_PUBLICS_PER_KEY</code>	6	a bound, not a claim

35. Promises

A promise that nothing ever checks is not a promise, it is a line of dialogue.

The runtime took promises from the beginning — parsed, emitted, remembered as `commitment` memories — and nothing resolved one, so undertaking something cost nothing. `SocialExchangeEngine.on_commitment_resolved` had always known exactly what a kept and a broken promise are worth and **was never called by anything**.

That single gap is why three engines looked dead: `ReputationEngine` had no producer, so `agent.reputation` was empty in every world at every time, and `sociolinguistics` read it to compute perceived status and was therefore reading a permanently empty mapping. **The missing piece was never those engines. It was the thing that resolves a promise.**

35.1 How a promise is judged

Not against the truth. This runtime holds no ground truth about whether somebody paid — it holds what people believe, and inventing an oracle here would contradict the whole design.

A promise is kept when **the person it was made to comes to believe it was**.

That is the right answer for a social simulation and not a compromise. Reputation has always been about what people think happened. A promise quietly fulfilled where the promisee never finds out damages the promiser exactly as much as one that was broken — which is unfair, true, and the reason people make a point of being *seen* to deliver.

35.2 Recording

```
record(world, event, witnesses):
    require event.type == "promise" and a proposition
    promiser = event.actor
    promisee = payload.target or payload.addressee
    if not both, or they are the same: ignore
        # a promise to nobody in particular is a statement of intent

    due = proposition.valid_end if the proposition names its own end
        else world_time + horizon_minutes
    prior = promisee's current belief in the proposition, else 0.5

    Promise{ promise_id, promiser, promisee, key, made_at, due_at,
              promisee_prior, witnesses = sorted(witnesses u {promiser, promisee}),
              state: "open" }

    evict the oldest beyond max_open_per_agent
```

35.3 Settling

```
step(world, runtime, delta):
    settle every open promise whose due_at <= world_time

_settle(world, runtime, promise, forced=None):
    confidence = promisee's belief in promise.key (0.0 if none)
    kept = forced if the game said so
        else confidence - promisee_prior >= kept_margin
    apply exchange.on_commitment_resolved(promisee, promiser, kept):
        kept → trust +0.8, liking +0.4
        broken → trust -0.7, liking -0.5, reputation "reliable" -0.4
    apply the reputation delta to EVERY witness as well
```

kept_margin = 0.002, a margin rather than an absolute threshold, and that is a correction worth reproducing. The first version asked for a fixed 0.55, which measurement showed was the wrong *shape*: one ordinary piece of evidence about a fresh proposition moves a belief from 0.500 to 0.537, so a fixed 0.55 silently required **two independent confirmations** before anybody was credited with keeping a promise made to them personally. That is the right bar for a

bystander weighing a rumour and the wrong one for the person who was owed. Asking about the *movement* says what was meant — *did anything reach them that says you came through* — and survives a later recalibration of the belief constants, which an absolute number would not.

Who learns of it: only the promisee, and whoever was in earshot when the promise was made. It does not become a fact about the town, because a broken promise is not news; it is a disappointment, and it travels the way disappointments do — through the diffusion layer if at all.

`settle(promise_id, kept, note)` is the **authoritative** path: a studio that scripts "the player hands over the money" should not have to hope an NPC was paying attention. Everything downstream is identical.

Parameter	Default
<code>horizon_minutes</code>	2,880 (two days)
<code>kept_margin</code>	0.002
<code>max_open_per_agent</code>	24

Measured: keeping a promise raises the promisee's trust from 0.30 to **0.415** and **nobody else records a thing**. Breaking it drops trust to **0.055** and **four people end up holding "unreliable" about you, three of whom only watched**. A kept promise is expected; a broken one is news.

Part V — Speech

The safety boundary. **The runtime decides *what* is said; a text layer decides only *how it is worded*, and a validator checks the wording before anyone sees it.**

36. The speech pipeline

```
decide → plan → COMMIT → realize → validate → release → speak aloud
  ↑       ↑       ↑       ↑       ↑       ↑       ↑
policy  dialogue commitment provider validator player an EVENT
                        + grounding                in the world
```

36.1 Planning

```
plan(agent, conv, chosen_action, addressee, style, goal, world_seed, world_time):
    act = chosen_action.dialogue_act or "inform"
    avoid = avoid_labels(agent.secrets) # LABELS, never secret content
    stance = "deceptive" if act == "evade" else "asserted"

    if act in ASSERTING_ACTS (inform, answer, confide, warn):
        candidates = beliefs with expected_prob >= C_ASSERT (0.6)
            excluding protected core keys
        plan.commit(commitment.select(candidates, act="assert", limit=1,
            seed_parts=(world_seed, agent.id,
                conv.turn_count, world_time,
                "deliberate"),
            forbidden=avoid))
    else:
        plan.commit(Commitment(moves=(), forbidden=avoid)) # assert no fact
```

```
DialoguePlan:
    speaker, addressee, dialogue_act, goal
    moves : tuple[SemanticMove] # the ONLY factual content
    allowed_facts : list[str] # rendered form of the moves
    avoid_topics : list[str] # LABELS
    stance : "asserted" | "deceptive"
    style : dict
    fallback_template_id : "deflect"
    max_length : 30
    phrasing : {register: line} # authored, for THIS answer
    request_id : str # match a late line to its turn
```

allowed_facts is not a menu. It is exactly the commitment, rendered. An earlier design handed the provider a *list* of facts it was permitted to mention and let it choose — which meant the model decided what information entered the society, because a spoken claim is a real event.

36.2 Anti-loop

```
anti_loop_ok(conv, act):  
    False if the last three acts are all the same AND act repeats it
```

`ConversationState` persists across turns **and across save/load**: `discussed_topics` (topic → times asked), `recent_lines` (bounded to 5, the anti-repeat window), `turn_count`, `last_acts`, `active_topic`.

A re-asked topic gets a terser answer (`max_length` halved).

36.3 Realize → validate → release

```
for attempt in range(2):  
    try:  
        text      = realizer.realize(plan, plan.style)  
        authored = realizer.is_deterministic          # our own text is correct  
                                                       # by construction  
    except ProviderError:  
        text = fallback_realizer.realize(plan, plan.style) # deterministic  
        plan.dialogue_act = plan.fallback_template_id  
        authored = True  
    result = validator.check(text, plan, agent, recent,  
                             canon_mode = CANON_OFF if authored else configured)  
    if result.verdict == ACCEPT: break  
    if result.verdict == REJECT_HARD: plan.dialogue_act = fallback_template_id  
else:  
    plan = a fresh "deflect" plan ; text = fallback_realizer.realize(...)
```

Buffered, never streamed. The utterance is generated fully, validated, and only then released. Streaming unvalidated tokens can leak.

36.4 Speaking aloud — the loop closing

There is one implementation of this, and there used to be one and a half. The SDK's answer path emitted the committed claim as an event; `Runtime.say()` — also public, and the path deliberation takes — realised a line, validated it and returned the text. A character reached through the lower API could therefore say something in a room full of people and nobody learned anything; the defect this runtime exists to remove, reintroduced through a second door. `speak_aloud`, `earshot`, `spoken_origin` and the commitment read-back now live on `Runtime`, and the SDK delegates.

```

_speak_aloud(agent, moves, addressee):
    for each factual move:
        strength = { certain: 1.0, probable: 0.8, uncertain: 0.55 }[move.certainty]
        emit Event(type="claim", actor=agent, location=agent.location, payload={
            proposition:      move.proposition,
            audience:         _earshot(agent, addressee),    # addressee + up to 3
            importance:       0.45,
            assertion_strength: strength,
            domain:           world.predicate_domains[predicate] or "street",
            spoken:           { addressee, act, certainty, honesty },
            origin_event:     _spoken_origin(agent, move) })

```

Two rules here carry the whole design:

Only the commitment is emitted. The provider's wording is presentation and never reaches the belief system. That is what makes the text layer replaceable without changing the simulation.

A conversation is not a public announcement. `_earshot` bounds the audience to the addressee plus up to three others, chosen deterministically. Without it, answering one question in a crowded place updated every person in it: measured at 600 agents, 825 events produced **137,000 belief updates**.

```

_spoken_origin(agent, move):
    if move.honesty != "lie" and the speaker holds this belief with an origin:
        return that origin                # pass on evidence that already existed
    return f"said_{agent.id}_{move.proposition.core_key()}"

```

Dialogue did not follow the origin rule for a long time: every answer minted `said_<speaker>_<time>`, so **asking one person the same question twice produced two origins and the runtime counted its own repetition as independent corroboration** — the precise failure this system exists to prevent, arriving through the channel a player uses most.

A lie is the principled exception: it has no evidential ancestor, so the claim genuinely begins with the speaker. **But it names the claim as well as the speaker**, and that second half was missing. Keyed on the clock instead, the same person telling the same lie a minute later minted a *second* origin, so the runtime counted its own repetition as independent corroboration — the same failure, arriving one level up. Measured before the fix: one liar, one claim, eight tellings, eight origins, `p` from 0.513 to 0.612. Keyed on the proposition it saturates at 0.5145, and a *different* lie from the same mouth is still a different origin.

Provenance records the speaker separately in both cases, which is what `discredit` matches on.

36.5 Voiding a commitment

Two checks, and if either fails **nothing is asserted**:

```

expressed = realizer.expressed_commitment          if the realizer reports
            = all(grounding.expresses(text, move.proposition, world)
                  for factual moves)                otherwise

spoken_moves = () if (fallback_used or not expressed) else plan.moves

```

The player must never read a deflection while the room hears an assertion. A provider that does not report cannot be taken at its word, so the released line is read back: does it actually name what was committed?

Nothing is committed that the answer cannot say. Selection picks up to two propositions (§37.1); a pack authors one phrasing per *topic*, and it words one thing — whether the topic's own query holds. A factual move the phrasing does not word is dropped from the commitment before the plan is built, reported as `commitment.unsayable_dropped`. Without that, a character who held both "*the clinic was shut last night*" and "*the clinic is open today*" said the first and taught the room both — the same mismatch controlled release exists to prevent, arriving from the runtime's own side rather than a provider's.

A rejection is two different answers, and conflating them was a defect of its own. `validator.SAFETY_CODES` — `secret_leak`, `secret_referent`, `canon_violation` — mean the line is unsafe: the commitment is void, the plan falls back to its deflection template, **and plan.phrasing is cleared**, without which the lookup returns the authored sentence again and the "deflection" is a no-op wearing a different act name.

Everything else, `validator.repetition` above all, is a *quality* objection. The runtime tries another complete wording of the same commitment (`dialogue.reworded`) and, failing that, releases the line as it stands (`dialogue.released_anyway`) — **and the returned verdict, the audit row and the world all say so**. Releasing the line while returning the rejection is the same divergence one field further out: from the fourth identical question onward the claim reached the room, `accepted_output` was blank and the caller was handed `REJECT_SOFT`, so a client that hides rejected answers would show the player nothing the room had just heard. The release decision is made once and recorded once. It never answers a repetition by dropping the claim. Treating the two alike let arbitrary earlier prose — a provider's greeting from the previous turn, sitting in `recent_lines` — decide whether a later, fully controlled fact entered the society at all: same seed, same two calls, and only that first sentence differing, one run created the belief and the other did not.

37. Commitment and deception

```
SemanticMove:
  act                : assert | deny | evade | advise | greet
  proposition        : what is ASSERTED – not necessarily what is believed
  certainty          : certain | probable | uncertain
  disclosure         : direct | partial | hedged
  honesty            : honest | lie | exaggeration | understatement
  believed_probability : what the speaker actually holds

certainty_for(confidence) = certain   if >= 0.85
                           probable   if >= 0.65
                           uncertain   otherwise
```

Four things kept apart: what the speaker *believes*, what they *want*, what they *say*, and what they intend the listener to believe afterwards.

37.1 Selection

```
select(candidates, act="assert", limit=1, seed_parts, forbidden, max_sentences=2):
    ranked = sort by (-round(relevance,4), -round(confidence,4), str(proposition))
    while pool and len(chosen) < limit:
        best = pool[0]
        band = [c in pool with the SAME relevance and confidence within 0.05]
        pick = band[0] if the band is a singleton
            = band[seeded_choice(derive_seed(*seed_parts, "commitment",
                                            f"pick{n}"), uniform weights)]
        proposition = pick.proposition if pick.affirmed else its negation
        chosen.append(SemanticMove(act = act if affirmed else "deny",
                                   proposition, certainty_for(confidence),
                                   believed_probability = confidence))
```

Everything within a small band of the best score is treated as equally good, and among those the choice is **seeded rather than first-past-the-post**. That is the whole argument against letting a model pick: seeded selection gives the same unpredictability from the player's side, and stays in the event log.

Note the explicit rounding in the sort key and the string tiebreaker — both required for cross-language determinism.

37.2 Deception

```
deceive(move, assert_instead, honesty="lie"):
    SemanticMove(act = move.act,
                 proposition = assert_instead,      # the cover story
                 certainty = move.certainty, disclosure = move.disclosure,
                 honesty = honesty,
                 believed_probability = move.believed_probability) # UNCHANGED
```

The speaker's own belief never changes — lying is not self-persuasion — and `believed_probability` stays what it was, so the record shows the divergence and a later contradiction is findable.

The lie path in full:

```
_tell_a_lie(agent, interlocutor, topic, secret, conv):
    template = resolve_template(secret.cover_story, player_id, interlocutor)
    if no template: return None          # no cover story, no lie: they refuse
    claim = Proposition.from_dict(template)
    really_thinks = agent's own belief in that claim, else 0.0
    move = SemanticMove(assert, claim, CERTAIN, DIRECT, LIE, really_thinks)
    plan = DialoguePlan(..., goal="protect " + secret.secret_id,
                        avoid_topics=[secret.avoid_label],
                        phrasing=secret.cover_phrasings).commit(Commitment((move,)))
    plan.stance = "deceptive"
    return _deliver(...)                # THE SAME release path
```

A lie is not a special case of speaking. It is an ordinary commitment whose proposition happens not to be what the speaker believes. Giving it its own release path would be the place a bug would hide. It goes through the same realizer, the same validator, the same grounding read-back, and the same `_speak_aloud`.

The listener acquires the lie **with this character as its origin**, which is what makes it refutable later and what makes exposing the liar do something.

38. Validator and grounding

Four layers, in order. Layer 0 is architectural rather than a check.

Layer 0 — context minimisation

Forbidden content never enters a prompt at all. The dialogue planner passes an **avoid-label**, never the secret itself. A protected proposition is filtered out of the candidate set before a commitment exists, at any trust level.

Layer 3a — secret surface-form scan

```
for topic in plan.avoid_topics:
    for form in secret_surface_forms[topic]:      # lowercase substrings, per pack
        if form in utterance.lower():
            return REJECT_HARD (validator.secret_leak)
```

`secret_surface_forms` is **world vocabulary** gathered from the loaded pack's secrets: *"back room"* gives away a secret in one world and means nothing in another. This table used to be hardcoded for a single character in a single pack, which meant every other secret in every other pack was unprotected.

`unprotected_labels(avoid_topics)` reports avoid-labels with **no** surface forms — the scan cannot see those, and saying so is better than implying protection.

Layer 3b — canon parse-back

Only for external providers. Deterministic output is authored and correct by construction; parse-checking it would be theatre.

Modes: `off` (no check), `warn` (record and release), `strict` (reject softly and fall back to authored text). Default for external providers: **strict**.

The rule is narrow and deterministic, which is what makes it trustworthy:

Proper nouns and numbers carry factual specificity, and every one of them in provider output must be licensed by the plan. Ordinary words carry stance and are left alone — the model is free to phrase, not to invent.

```

licence text = allowed_facts + speaker + addressee + goal + phrasing values,
              with [:_\-(=,.)+ replaced by spaces, lowercased
              # ids read like "place:back_room", so the word "room" in an
              # utterance matches the id it came from

numbers:      any \d+([.,]\d+)? not present in the licence → finding
proper nouns: per sentence, any capitalised word where
              lowered not in NEUTRAL_CAPITALS
              and lowered not in licence
              and NOT (position == 0 and the word is not a known world term)
→ finding, classified as
    "unlicensed_referent" if it names a real thing in this world
    "proper_noun"        otherwise
de-duplicate by (kind, lowered token), preserving order

```

`NEUTRAL_CAPITALS` is deliberately small — 30-odd words: pronouns and contractions of *I*, `ok`, `yes`, `no`, honorifics, greetings, `god`, `well`, `look`, `listen`, `sorry`, `please`, `thanks`, `good`, and the times of day. Anything world-specific belongs to a pack; anything longer would start hiding real hallucinations.

Sentence-initial capitals are skipped only for unknown words. A capital at position 0 is usually grammar and an unknown word there cannot be told from an ordinary one — but *"Kane told me."* is never grammar, and skipping position 0 wholesale let exactly that through.

Before this layer existed the check was a comment, and the only active guards were a secret substring scan and exact-repetition detection, so:

```
check("The mayor was murdered by Kane in the harbour last Tuesday.") → ACCEPT
```

Layer 3b' — grounding: invented referents

The proper-noun rule cannot see *"the old mill by the river"* in a world with no mill: no capital, no number. A character asserting the existence of a thing that does not exist is exactly the failure mode that makes a language model untrustworthy in an NPC's mouth.

No language model is involved. A grounding check that needs a model to work cannot be the thing that makes the model safe.

```

WorldLexicon.build(world):
    index every place label and alias, agent public/objective name and aliases,
    entity name/aliases and the tail of its id ("item:sealant" → "sealant"),
    topic aliases, and every word longer than 3 characters in every declared
    PREDICATE name – if a pack declares `seal_breached`, "seal" is a thing in
    that world

    ALSO index the individual words of multi-word names: "The Harbour Front" is
    a place, so "harbour" is this world's vocabulary. Without this, every world's
    own nouns read as inventions the moment they appear on their own.

    decorative = GENERIC_NOUNS u world.decorative_vocabulary

```

```

unknown_referents(text, world, licensed):
    walk tokens left to right
    if the lexicon has a longest match at this index: skip the whole phrase
    if the PREVIOUS token is a determiner (the, a, an, that, this, those, these,
                                         his, her, their, my, your, our, its):
        walk forward to the HEAD of the noun phrase, stopping at a decorative or
        known noun, a phrase-ending function word, another determiner, or a word
        that looks verbal (>4 chars ending in "ed"/"ing")
        if head is not decorative, not licensed, not a known term, len > 2:
            report it as an invented referent

```

Each candidate is **classified rather than uniformly rejected**: *"the door"* is furniture, *"the old mill"* is a claim about what exists. Walking to the head is why *"the old mill"* is a claim about a mill and not about an *"old"*.

The `_looks_verbal` test is English morphology, not a parser; it exists so *"a courier dropped"* reads as a courier and a verb. Getting it slightly wrong **costs a deflection, never a leak**, because the check only ever adds candidates to an already-conservative list.

A token that trips both rules is reported **once**: a capitalised invented noun is a proper noun *and* an invented referent, and reporting it twice makes the record read as two problems where there is one.

Layer 3c — repetition guard

```

if utterance in recent_utterances[-repetition_window:]: # window 3
    return REJECT_SOFT (validator.repetition)

```

Layer 4 — fallback

`REJECT_HARD` or `REJECT_SOFT` sends the turn to `fallback_template_id (deflect)`, rendered deterministically and re-checked with `canon_mode = off`. **The commitment is void**, so nothing propagates: the player reads a deflection and the room hears nothing.

38.1 The other grounding question

```

anchors_for(proposition, world):
    for each slot value: every phrase in the lexicon that resolves to it,
                        plus the tail of the id with underscores as spaces

expresses(text, proposition, world):
    True if there are no anchors (nothing to look for; do not block on it)
    True if ANY anchor appears in the text

```

Deliberately generous: one anchor is enough. The purpose is to catch a sentence that says nothing about the subject at all — a deflection released while the room heard an assertion — not to grade paraphrase quality. Being strict would reject good writing; being absent lets the simulation and the script describe different conversations.

39. Realizers, and where a language model may sit

39.1 The interface

```
ModelProvider.realize(plan, style) -> str
    is_deterministic      : bool      # governs whether canon parse-back runs
    expressed_commitment  : bool      # set per call; whether the words carried it
```

39.2 The deterministic realizer

```
realize(plan, style):
    register = "formal" if formality >= 0.6 else
        "vernacular" if formality <= 0.3 else "neutral"
    text = plan.phrasing[register] or plan.phrasing["neutral"] or any phrasing
        if the plan carries authored phrasing
    else templates[dialogue_act][register] or templates[dialogue_act]["neutral"]
    factual = any(move.is_factual for move in plan.moves)
    expressed_commitment = bool(plan.phrasing) if factual else True
    if text is None: text = templates["deflect"][register]
    if slang lexicon and slang_level > 0.3: substitute word by word
    if not factual and mean_sentence_length < 7 and "." in text:
        keep the first sentence

variants(plan, style) -> [complete wordings, best register first]
```

expressed_commitment asks whether **THESE WORDS** carried it, not whether a template existed. A factual plan realised from a *stance* template — because the pack authored no phrasing for it — used to report the commitment as expressed, and the room learned something no sentence said.

And style never shortens a line that carries a claim. The sentence-length pressure cut an authored *"Listen carefully. Place was shut that night."* to *"Listen carefully."* for an agitated speaker while the claim behind it still propagated. Style may choose between *complete* realizations — which is what `variants()` gives the anti-repeat retry — and may not produce one that carries less than the commitment does.

There is no built-in `inform` template. The built-in templates express a *stance* — evading, threatening, asking to be paid, greeting, having nothing to add — and deliberately assert no world facts. Stating what is true about a world is the world's job.

That distinction is not pedantry: the default `inform` template used to read *"the place was shut that night"*, a sentence about the reference scene's clinic. **Any other pack inherited it**, so a character could assert something they did not believe and that was not true in their world.

39.3 The two places a model may sit

At build time — generating phrasings and world packs, which are then checked by the authoring gate, committed as JSON, and shipped. Nothing infers at run time, nothing can hallucinate in front of a player, and the result is reviewable in a diff. `lwr rewrite-phrasings` does exactly this, behind three checks: the line must be acceptable prose, it must not introduce unlicensed specifics, and the authoring gate runs afterwards.

At run time — as the **wording layer only**, and only in `semantic_release="provider"`. An OpenAI-shaped HTTP endpoint receives the plan, the allowed facts, the avoid-labels and the style vector. The runtime still chooses the content; the model chooses the sentence; the validator refuses one that says something the character does not hold.

In the default "controlled" mode the provider is not called at all — see C1. The mode used to make an exception for plans that assert nothing, on the reasoning that a line carrying no commitment cannot contradict one. It cannot, but the *line* is not bound by the plan: asked for a greeting, a model wrote *"He hides behind where drinks are served"* — an indirect description of a protected place — and it was released with `ACCEPT` because the turn had been classified as carrying no fact.

Neither is required. With no model at all, every line comes from authored phrasings, which is also what the default does.

39.4 The frame budget

A model does not care about a frame budget. Measured against a local endpoint on one desktop GPU: a 4B model costs **7,493 ms cold** and **290 ms warm**. 290 ms is still past any frame budget.

So the run-time integration has three defences:

Setting	Default	What it does
<code>latency_budget_ms</code>	120	how long a turn may wait before authored text is used
<code>deferred</code>	true	keep generating past the budget; expose the result through <code>poll_line_upgrades()</code> as a line upgrade
<code>warm_up</code>	true	page the model into VRAM at load time, so the cold cost does not land on whoever the player talks to first
<code>probe</code>	true	check at startup whether this model can voice NPCs at all

A late line is matched back to its turn by `plan.request_id`, and is **re-validated before release** — a late line that fails is reported as `provider.late_line_rejected`.

Part VI — System properties

40. Persistence, snapshots, replay

Three different storage concerns, deliberately kept apart:

	What it is	Authoritative?
Event ledger	append-only stream of every event	yes
Projections	materialised per-agent belief / memory / affect	no — a read model
Snapshot	the complete mutable state needed to resume	yes, for resuming

40.1 Tables

```
event_ledger (event_id, world_time, type, actor, location, payload)
belief       (owner, key, data)
memory      (owner, memory_id, data)
affect      (owner, data)
reason_trace (event_id, owner, entries)          -- debug only, trimmed
provider_call (turn_id, world_time, provider, model_id, prompt_hash,
              raw_output, accepted_output, validation_result, data)
snapshot     (snapshot_id, label, data)
meta        (k, v)
```

`provider_call` is what makes a model run auditable: **every line the model tried, and what was released.** `prompt_hash` is a SHA-256 over the plan fields (`speaker` , `addressee` , `act` , `allowed_facts` , `avoid_topics` , `style`) with sorted keys, so an identical request is recognisable without storing prompts.

Writes are **targeted**: an event changed at most one belief, added one memory and moved affect. Rewriting an agent's whole projection per event made persistence dominate the event path. One commit per event; one commit per tick.

40.2 Snapshot

```
SNAPSHOT_SCHEMA_VERSION = 10 .
```


Over the world's **shape** — who and where — rather than over the pack files. A studio reformats JSON, renames a directory or regenerates a pack, and none of that should invalidate a player's save. What must match is that the characters and places a save talks about are the ones this world has.

```
inspect_state(blob, world) -> { usable, code, detail, ... }
  not_a_state_blob      missing the marker
  envelope_too_new      format_version ahead of this build
  save_from_newer_build snapshot schema ahead of this build
  different_world       this save belongs to another world
  otherwise usable, reporting agents that would be dropped or defaulted
  ok                    the declared ids match and the shape is unchanged
  world_changed         same world, patched cast or map
  world_unverified      neither side declares an id; the match was INFERRED
```

A world says what it is. `scenario.json` carries `world_id`, and two declared ids decide the question outright — matching or not — with no inference involved. Every shipped pack declares one.

The heuristic is the compatibility path for saves and packs written before the field existed, and its bar is a **character who is not the player**. Overlap in place names is not identity: two unrelated demo worlds that both name a room `place:spawn` accepted each other's saves, and raising the threshold to two shared names only moved the coincidence rather than removing it — `place:spawn` and `place:street` did it again, jumping a noir clock from 1200 to 4334. When the heuristic does let a save through it reports `world_unverified` rather than `ok`, so a loader can refuse an inferred match on its own.

A patch is not a corruption. A studio ships an update that adds an NPC or removes one; a save from before it must still load, reporting what it could not place (`snapshot.agent_not_in_snapshot`, `snapshot.unknown_agent`, `snapshot.unknown_faction`) rather than refusing.

This is the call a load screen makes: it can offer *continue* or *start over* on the strength of it, and never has to catch an exception to find out which.

Size, for scale: **142 kB raw and about 11 kB gzipped for eight characters** — small enough to embed in any save format without anybody thinking about it.

40.4 Replay

```
replay_commands(commands) -> ReplayResult
```

Same seed + same commands $\Rightarrow$ byte-identical state. This is the property everything else rests on, and it is what the conformance fixtures compare.

Measured: same seed, same conduct $\rightarrow$ **0 of 26** beliefs differ. Same seed, *different* conduct $\rightarrow$ **7 of 30** beliefs differ after three simulated days, and one of six places has a different mood. The world diverges **because of the player** rather than because of noise — and either half of that without the other is worth nothing.

41. Optional layers and the tuning surface

41.1 The nine layers

Every one defaults to **off**, and off is byte-identical (asserted by test).

Flag	What it adds	Requires
<code>action_bridge</code>	the runtime asks your engine to carry out physical acts and waits to be told what happened	
<code>climate</code>	places have a mood, moved by what happens in them and carried between rooms by whoever walks	
<code>pursuit</code>	characters act on their goals instead of only moving on routines and gossiping at random	
<code>media</code>	a claim can travel by telephone: fewer bystanders, more detail lost, weighed lower	<code>pursuit</code>
<code>notes</code>	somebody who cannot reach the person they needed leaves word where they stand	<code>pursuit</code>
<code>promises</code>	a promise comes due and is judged on what the promisee came to believe	
<code>contagion</code>	a little of a speaker's mood moves to whoever they are talking to	
<code>common_knowledge</code>	track who watched everyone else find out; a secret in the open stops working	
<code>standing</code>	how you treat one person changes how the rest treat you, carried as a claim	

Why "off is byte-identical" is a real engineering constraint and not a slogan: a studio that does not want a mechanic should not have to pay for it, and — more importantly — a layer that perturbs a run *slightly* when off is a layer nobody can reason about. It is why the climate modifiers are expressed as deviations from neutral, why `regard` is passed to the register only when standing is on, and why `appearance` needs no flag at all (authored data that defaults to nothing).

41.2 The tuning surface

How this world works is content. A mediaeval village and a surveillance state are not the same simulation with different names in it. Every engine's parameters are authorable from one block in `world.json`:

```
"tuning": {
  "diffusion": { "distortion_prob": 0.30 },
  "memory":    { "tau_ret": -1.2 },
  "contagion": { "transfer": 0.20 },
  "promises":  { "horizon_minutes": 720 }
}
```

Fourteen tunable engines: `belief`, `memory`, `affect`, `diffusion`, `pursuit`, `contagion`, `notes`, `promises`, `common_knowledge`, `policy`, `relationship`, `reputation`, `identity`, `socioling`.

Three rules, and the first is the one that matters.

1. **An unknown key is an error, never a shrug.** A tuning value silently ignored because it was misspelled is the worst failure this surface could have: the world behaves as though nothing was set, the author believes it was, and the gap between them is invisible. Every name is checked against what the engine actually has, and a near miss is named back ("*Did you mean ...?*").
2. **Types are checked.** A string where a number belongs would either crash deep inside a tick or, worse, compare as something unexpected.

3. **Nothing here changes what is possible, only how much.** These are rates and thresholds. No amount of tuning makes a character know something they were never told, or say something the validator would stop — **so a pack cannot tune its way out of the guarantees the runtime exists to make.**

41.3 Bounds that must be enforced

Only parameters with a bound that can be *justified* are constrained. Inventing a plausible-looking range for the rest would refuse worlds for no reason.

Parameter	Bound	What happens outside it
<code>belief.kappa_min</code>	<code>(0, 1)</code> open	<code>ln(0)</code>
<code>belief.kappa_max</code>	<code>(0, 1)</code> open	division by zero in <code>ln(k/(1-k))</code>
<code>belief.rho_correlated</code>	<code>[0, 1)</code>	at 1.0 repetition never attenuates
<code>belief.provenance_limit</code>	<code>≥ 1</code>	
<code>affect.tau_mood</code> , <code>affect.tau_emotion</code>	<code>> 0</code>	division by zero in the decay
<code>memory.episodic_cap</code> , <code>total_cap</code> , <code>working_cap</code> , <code>presentation_window</code>	<code>≥ 1</code>	a cap of zero silently discards everything it caps, which looks exactly like a broken layer
<code>notes.max_per_place</code>	<code>≥ 1</code>	
<code>notes.lifetime_minutes</code> , <code>promises.horizon_minutes</code>	<code>> 0</code>	
<code>promises.max_open_per_agent</code>	<code>≥ 1</code>	
<code>pursuit.interval_minutes</code>	<code>> 0</code>	
<code>pursuit.min_confidence</code> , <code>max_per_tick</code>	<code>[0,1]</code> , <code>≥ 0</code>	
<code>diffusion.max_hops</code>	<code>≥ 1</code>	
<code>diffusion.bystanders</code>	<code>≥ 0</code>	
<code>diffusion.fidelity_decay</code>	<code>(0, 1]</code>	at 0 nothing survives one hop
<code>diffusion.distortion_prob</code> , <code>stifle_prob</code> , <code>min_confidence_to_tell</code>	<code>[0, 1]</code>	
<code>diffusion.encounters_per_hour</code>	<code>≥ 0</code>	
<code>contagion.transfer</code> , <code>max_step</code>	<code>[0,1]</code> , <code>(0,1]</code>	
<code>contagion.min_tie</code>	<code>≥ 0</code>	
<code>common_knowledge.min_witnesses</code>	<code>≥ 2</code>	below two the word stops meaning anything
<code>common_knowledge.public_privacy</code> , <code>min_quality</code>	<code>[0, 1]</code>	
<code>promises.kept_margin</code>	<code>[0, 1]</code>	
<code>policy.temperature</code>	<code>> 0</code>	
<code>relationship.eta_up</code> , <code>eta_down</code>	<code>≥ 0</code>	

All three of `kappa_max = 1.0`, `kappa_min = 0.0` and `tau_mood = 0` are values a pack could legally set, and all three crashed mid-tick with a traceback from three files away. That is why the bounds exist and why the error message

names the parameter.

42. Scale: cost, bounds and level of detail

42.1 Measured cost

Both languages running the same generated pack from disk — a district of rooms with staggered shifts, grown from `relay-station` so every character is a complete one rather than an empty struct.

A turn — a player has asked a question and somebody is answering. The only number on a player's critical path:

Cast	Python	C++	of the 100 ms at which a reply stops feeling immediate
11	0.187 ms	0.083 ms	0.1%
51	0.366 ms	0.126 ms	0.1%
201	1.036 ms	0.345 ms	0.3%
1,001	5.006 ms	1.497 ms	1.5%

An hour of world — everybody moves, meets, talks, forgets and passes things on:

Cast	Python	C++	of a 60 Hz frame (16.67 ms)
11	0.610 ms	0.173 ms	1.0%
51	1.128 ms	0.331 ms	2.0%
201	3.686 ms	1.089 ms	6.5%
1,001	28.740 ms	10.009 ms	60.1%

Three things to take from this rather than the headline:

- **One call covers a whole simulated hour of everybody's day.** A game does not tick this per frame; it advances world time on its own schedule. At two hundred characters that is one millisecond for an hour of a whole town.
- **A turn costs more as the cast grows** — sub-linear, but growth where a reader might expect none: answering one question should not depend on how many people live in the district.
- **The C++ core is about three times faster than Python.** It was verified for correctness bit-for-bit long before anybody measured what it cost.

42.2 The optimisations that mattered, and what each was worth

Change	Before	After
Occupancy index instead of scanning the cast per event	quadratic in population	linear
Sampled encounters instead of enumerating pairs	200 co-located agents $\approx$ 40,000 candidate pairs/tick	$O(\text{occupancy})$
Bounded earshot on speech and diffusion	600 agents: 825 events $\rightarrow$ 137,000 belief updates	≤ 4 listeners per exchange
origin_counts index instead of scanning provenance	$O(\text{claims})$ per belief update; unbounded record	$O(1)$; 64-entry cap
Presentation window on memory	301 entries after 300 retrievals	32
Trace caps (400/bucket/tick)	2,000 characters, one hour: 553,454 entries, 36 MB, tick 141 $\rightarrow$ 300+ ms	bounded, with an exact dropped count
Targeted persistence writes	whole projection rewritten per event	one belief, one memory, one affect
projection_scope	writing projections for everybody	80% of tick cost at large casts

42.3 Population level of detail

```
population_lod = True:
    per-tick decay is SKIPPED for agents with no active emotions
    a dormant agent is caught up lazily on access:
        dt = world_time - last_decay[agent]
        affect.decay(dt) ; fade_identity_threat(dt) ; memory consolidation
```

Exact, not approximate. Exponential decay is composable, so one catch-up over the full elapsed `dt` yields the same state as having decayed every tick. The tick becomes `O(active)` rather than `O(population)`.

`_catch_up` is called before an agent reacts to an event, before they speak, and before any read of their state (`structured_agent`, `face_packet`).

42.4 Projection scope

Value	Who gets a resident SQL projection
<code>all</code>	everybody. Correct for tens of NPCs; the default
<code>focus</code>	the focus agents, the player, and anyone the player has met
<code>none</code>	ledger only. Smallest and fastest; inspection needs a replay

The ledger is authoritative regardless; this decides only whether the read model is kept warm.

43. Integration surface

43.1 Two ways in

A Python service beside the game, on PC	works today. HTTP/JSON; an engine that is not on the supported list is a client away rather than a port away
A C++17 core with a C ABI and one thin binding per engine	for a console, which will not run an interpreter at all

43.2 The HTTP/JSON service

Three rules hold throughout: **client mistakes are 4xx with a code a programmer can act on**; **every request is validated before it reaches the runtime**, because the runtime trusts its caller and a negative time delta or a malformed event would corrupt world state silently rather than fail loudly; and **authoring/debug endpoints are separately gated**.

GET

Path	Returns
/health	liveness
/capabilities	runtime version, which optional layers are on, which capabilities are BUILTIN/EXTERNAL/DISABLED
/bridges	the engine bridge profiles
/state/scene	the player's place, the NPCs in it, the exits
/state/world	world time, factions, heat, clocks, rumour exposure
/state/knowledge , /v2/state/knowledge	who knows what, how far from the source, how it travelled
/state/agent , /v2/state/character	one mind: beliefs + provenance, memory decay, affect, relationships, the turn's reason trace
/v2/state/export	the whole mutable world as one embeddable blob
/v2/state/public	what is common knowledge, and among whom
/v2/promises	open and settled promises
/v2/actions/pending	intents awaiting an engine's answer
/v2/resting_on	what currently rests on one source's word
/notes	what is lying about at a place
/inspect/world , /inspect/agent	human-readable dumps
/avatar/face	ARKit-52 blendshapes + gaze + prosody for one agent
/snapshots	list
/authoring/pack	the pack as a document (gated)

POST

Path	Does
/turn	submit player text; returns receipts, the NPC response and a developer trace
/avatar/turn, /v2/avatar/turn	the same, plus a MetaHuman packet
/event	inject a world event
/advance	advance world time by N minutes
/snapshot, /restore	create / restore a snapshot
/v2/state/inspect	what <i>would</i> happen if this blob were imported
/v2/discredit	expose a source; recompute everything resting on them
/v2/actions/{id}/result	the engine reports what happened to an intent
/v2/promises/{id}/settle	the game says a promise was kept or broken
/generate/characters	deterministic NPC generation
/authoring/preview, /authoring/save	the studio editor (gated)

43.3 The SDK facade

`LivingWorldsRuntime` is the object a studio integrates against. The methods that matter:

```

create(config) / close()
submit_player_text(text) -> TurnResult
respond(agent_id, topic=, interlocutor=) -> DialogueResponse
advance_time(minutes)
emit_event(event) -> EventReceipt
query_belief(agent_id, proposition) / retrieve_memories(agent_id, topics)
explain_knowledge(agent_id) / inspect_agent / inspect_world
structured_agent / structured_knowledge / structured_world / structured_scene
export_state() / inspect_state(blob) / import_state(blob)
create_snapshot(label) / restore_snapshot(id) / list_snapshots()
replay_commands(commands) -> ReplayResult
discredit(source_id, factor=, reason=, predicate=) / resting_on(source_id)
pending_actions() / report_action_result(intent_id, status, detail)
promises_report() / settle_promise(id, kept, note)
public_knowledge() / is_public_between(key, *agents) / notes_at(place)
poll_line_upgrades() / pending_line_count()
face_packet(agent_id) / avatar_turn(text) / voice_plan(packet) / speak(packet, path)
climate_report() / layers() / validate_world_pack(path)

```

43.4 The engine bridge contract

A native plugin maps its local events into the neutral JSON contract and maps responses back:

```
bridge_event_to_world_event(payload) -> Event
dialogue_response_to_bridge(response) -> { text, act, verdict, style,
                                           honesty, emotion, blendshapes,
                                           gaze, prosody, posture }
```

The `honesty` field is part of the contract. The Unreal plugin compiled cleanly for months and still dropped it, so a game could not see whether a character had lied — a defect **not findable by compiling**, which is the argument for behavioural tests inside an editor rather than a build check.

43.5 The face

The runtime owns *why* a character feels and says something; a face system owns how the face looks. The adapter maps `AffectState` (PAD mood + active OCC emotions) onto:

- **ARKit-52 blendshape weights** — the Live Link Face standard;
- **gaze aversion**, from shame/fear/low dominance;
- **prosody**: speech rate, pitch shift, loudness — a frightened character reads faster and higher, a dominant one slower and lower;
- **posture**, from Frijda action tendencies.

Prosody was emitted on every spoken line long before anything consumed it. The voice layer consumes it now, and **the runtime still synthesises nothing**: it decides what a line should sound like and hands that to whatever backend is installed (`espeak-ng` , `piper`), which is why it exists without the SDK gaining a dependency.

43.6 The C ABI

One header, **no C++ across the boundary**, nothing that throws, every structured answer as JSON. Proved by a program in C99 compiled by a C compiler — which is the only proof that the boundary is really C.

Bindings: Unity (C# P/Invoke), Godot (GDExtension), Unreal (plugin, HTTP or native). Each is verified against the real library by a real toolchain: a C99 program drives the ABI, a real C# compiler drives the P/Invoke layer, a headless Godot loads the GDExtension and drives it from GDScript, and the Unreal plugin compiles and links with UnrealHeaderTool over its reflection macros.

None has been run inside a game. That is the gap, and it is stated as one.

Part VII — Building and proving it

44. Reimplementation order

Each step is verifiable before the next begins. **That is the point of the order:** a port that can only be judged at the end is a port nobody can schedule.

#	Build	Why here	How you know it works
1	determinism	everything stochastic derives from it	reproduce a table of (inputs → uniform) from the reference
2	types, ontology, statekey	the two-branch sigmoid, propositions with the contradiction logic, typed deltas. core_key() is the identity a belief is filed under, so it matters far more than its size suggests	key strings match byte for byte; C1/C2/C3 truth table
3	belief	log-odds accumulation, the correlation discount, the provenance chain	a claim heard twice from one origin does not count twice
4	memory, affect, relationship	one mind, no world	ACT-R activation curves; mood relaxation half-life; asymmetric trust
5	actions, policy	utility aggregation and the seeded choice	the term/bias distinction must be preserved exactly ; merging them reintroduces the bug where looking dangerous made a character <i>less</i> likely to back away
6	contracts, events, social, content	vocabulary, feature providers, a pack's own words	contracts is mostly constants — compare them word for word : a layer description worded differently is a lie in a /capabilities response
7	agent, world, routine, topology, perception, distortion, diffusion	the society	where seeded draws start happening inside loops over collections — seven of them in diffusion alone
8	agency, sociolinguistics, dialogue, standing, appearance, commitment, factions	how somebody is read before they speak, how they sound, what the town thinks of them, who comes when they call	
9	climate, pursuit, promises, notes, medium, contagion, common_knowledge, tuning	the optional layers, one at a time	each has a conformance fixture that switches it on
10	grounding, interpretation, revision, parser, snapshot	what a sentence asserts about a world that may not contain it; what a witness made of what they saw; what a liar already convinced you of	
11	runtime, sdk, the pack loader	the tick loop, the turn loop, the reader that turns a directory of JSON into a world	all five conformance scenarios play end to end
12	the ABI / service boundary	one header, no exceptions across it, every structured answer as JSON	a program in the <i>plain</i> boundary language, compiled by that language's compiler

44.1 What has to be ported, and what does not

Measured on the reference implementation:

Layer	Lines	Modules
foundation	659	8
engines	2,253	15
world	4,038	23
runtime	2,673	5
to port	9,140	49
stays out of the game (CLI, service, authoring, evidence, benchmarks, demos)	9,757	25

Roughly **half the codebase never ships inside a game.**

44.2 Milestones a schedule can hang on

1. **A claim heard twice from one origin does not count twice.** Belief alone.
2. **Four witnesses of one event hold three different claims.** Perception + interpretation.
3. **A rumour reaches a third party nobody told directly, carrying the original origin.** Diffusion.
4. **A restricted claim reaches two people and is walled off from seven.** Topology.
5. **Exposing a liar recomputes the beliefs resting on them and leaves the independently sourced ones alone.** Revision.
6. **A character with a cover story lies rather than refusing, and the lie is on the record as a lie.** Commitment + secrets.
7. **A model-written line naming an invented place is rejected and replaced by authored text.** Validator + grounding.
8. **The same seed and the same commands produce the same exported state.** Everything.

45. Conformance and verification

45.1 The fixtures

Five scenarios, each (world pack, seed, layers, commands) → expected_state.

Fixture	Layers	What it is for
asking	—	belief, provenance and the correlation discount: the same claim heard twice from one origin must not count twice
time	pursuit	routines, diffusion and decay across a day — where iteration order and seeded sampling show up
violence	climate, pursuit	affect, identity threat, place climate — the paths a port leaves out because nothing asks them a question
conduct	pursuit, standing	standing, including what reaches somebody who was told rather than present
promises	pursuit, promises	a commitment coming due, and reputation reaching the people who heard it made

A second implementation is correct when it reproduces expected_state. Not the transcript: the human-readable message is recorded so a person reading a diff can see what the run was doing, and a port is free to phrase it differently. What it is not free to do is compute a different belief, a different provenance chain, a different mood or a different climate.

The two things this exists to catch:

- **Floating point.** `logit += eta * ln(kappa/(1-kappa))` has to land on the same bits in another language. It can; it is work rather than translation.
- **Iteration order.** Anything that walks a collection while drawing a seeded number needs an explicit sort. That bug shows up once in a thousand ticks and a byte comparison finds it on the first.

Running the fixtures against the reference implementation is **not** pointless: it catches an accidental behaviour change there too, which is what keeps them trustworthy enough to hold a port to.

45.2 The test suite

202 checks in one file, no framework, no third-party dependency — the same constraint the runtime is under, for the same reason: a suite that needs something installed is a suite that stops being run. Nothing is mocked; every check drives the real runtime over a real world pack, because the failures worth catching are about what a character comes to believe, and a mock cannot be wrong about that.

Family	What a failure means
Determinism	the same seed stopped producing the same world. Every other guarantee rests on this
Belief	a claim heard twice from one source started counting as two, or exposing a liar stopped unwinding what he convinced people of
Perception / interpretation	somebody learned something nobody told them, or four witnesses stopped disagreeing
Memory	a character forgot a murder before the player left the room, or recalled everything they ever heard at once
Diffusion	a rumour crossed a boundary it should not have, or circulated forever
Safety	a secret leaked, or a character asserted something the world does not contain
Optional layers	"off" stopped being byte-identical
Save and load	a save stopped restoring the world it was taken from
Scale and cost	the tick got slower, or traces grew without bound
The service	an endpoint changed shape under a shipped client
The port	the port stopped agreeing with the reference
The bindings	an engine can no longer call the runtime

Checks needing a toolchain **skip rather than fail** when it is absent — a check that cannot run is not a regression.

45.3 Mutation testing — because a green suite proves nothing on its own

The suite is itself tested. Deliberate defects are introduced into the runtime and the suite is asked whether it notices. Nineteen defects so far, and **five behaviours this runtime is sold on turned out to have nothing asserting them** — including *whether a retelling cites the source it first heard*, which is the sentence the whole design rests on. All five are now pinned at the boundary.

The same discipline holds the port: **504 deliberate defects introduced into the C++, 487 caught**; the seventeen that are not are documented at the line as equivalent or unreachable.

Two of the escapes were lessons about the *probe* rather than the port:

- an `extremes` case used 400 repetitions, which is **past** the window where the two sigmoid forms differ — by then both return 0, and the case passed a deliberately broken sigmoid;
- a `retrieval_ties_are_stable` case had six memories, which passed a deliberately unstable sort, because `libstdc++` falls back to insertion sort below sixteen elements and insertion sort is stable. Forty memories catch it.

Exercising an extreme is not the same as exercising the extreme that matters.

There is also a recorded trap in the harness itself: a poisoned-bytecode cache made one mutation report *caught* when it was not, which is why the harness now deletes `__pycache__` between mutations.

45.4 The benchmark and the evidence runs

The **benchmark** answers *does an invariant hold* with a number:

Metric	Claim
unsourced knowledge	nobody ends up knowing something they were never told
single-origin certainty	a repeated rumour never hardens into proof
secret containment	a secret never slips out
provenance completeness	every belief can name where it came from
memory / provenance / snapshot bounded	nothing grows without bound
diffusion reach	information actually moves
determinism	the same seed reproduces

The **evidence run** answers the harder question a studio asks in a meeting: *show me a character learning something, tell me who told them, show me it changing on the way, show me them forgetting it, and show me the one thing they must not say staying unsaid.*

Latest sealed run: two hours, four world packs, 36 cases — 35 held, 1 untestable in the world given, none failed. **3,282,740 turns, 83,245 simulated days, 65,653 invariant checks, 0 violations**, with every optional layer off (the configuration the byte-identical claim is about).

Sealed to the commit that produced it, and re-run when that commit changes. The check compares **content hashes of the core modules**, not paths — the first version used `git diff --name-only` and broke silently when the repository was flattened, reporting all ten files as changed because none existed at the sealed commit under the name it was given. *A check that cries wolf is a check nobody reads twice.*

Two findings in it are worth reading as findings rather than as a scoreboard:

- **Forgetting is 91% eviction, not decay** — 26.1M against 2.7M. Over 88,000 simulated days every character sits permanently against their memory cap. That is correct behaviour, but what a player would ever see is the decay, and decay is the smaller tenth.
- **After saturation, little happens** — 58 retellings and 91 beliefs acquired across 83,245 days. The long case demonstrates **stability, not liveliness**. This is what the pursuit layer exists to address, and the measurement is the argument: on `market-square` over thirty simulated days, without pursuit there are 54 conversations on days 0–2 and **zero** on days 27–29; with it, 96 and **90**.

And a discipline worth copying: the previous sealed run recorded 3,560,466 turns in the same two hours against this run's 3,282,740 — **7.8% fewer**. The runtime got measurably slower per turn, and that is stated rather than quietly replaced,

because a number that only ever improves is a number nobody measured.

45.5 Adversarial search

A golden scenario proves that a playthrough *somebody thought of* still behaves. That is the wrong shape for the failure studios actually ship: nobody writes a test for the leak they did not imagine.

```
lwr qa <rules.json> --world-pack <pack>
```

searches for the **shortest sequence of player actions that breaks a stated rule** — “*is there ANY way the player can make Clara know this*” — and, when it finds none, says **how far it looked**. Rules cover: a proposition never believed by an agent, a maximum number of independent sources, provenance completeness, and a form of words never spoken.

46. Traps: bugs that were real

Every one of these shipped-looking mistakes was found by measurement rather than by reading. A reimplementa-tion will meet them again.

46.1 Numerical

Trap	The wrong obvious thing	What it costs
Rounding	<code>nearbyint(x*1000)/1000</code> for <code>round(x, 3)</code>	disagrees with correct decimal rounding on 3 of 4,006 random values — always a tie. <code>round(0.0005, 3) = 0.001</code> because that double sits a hair <i>above</i> one half; multiplying by 1000 first destroys exactly the information the decision needed. Format to a fixed number of decimals and parse back: zero disagreements over 40,010 values
Sigmoid	<code>1/(1+exp(-x))</code>	equivalent until the log-odds pass about -710, where <code>exp(710)</code> overflows to infinity and the result collapses to a flat 0 — while the two-branch form still returns <code>5.07e-314</code> . That is where a character has finally made up their mind: the worst possible place to lose the answer
Summation	a loop of additions for <code>sum()</code>	CPython ≥ 3.12 gives <code>sum()</code> a fast path using Neumaier compensated summation . A naive loop over a softmax gives <code>0.9999999999999999</code> where Python gives exactly <code>1.0</code> , and dividing by one or the other is the difference. The reference calls <code>sum()</code> over floats in nineteen places. (Ruled out first, at cost: FMA contraction — <code>-00</code> , <code>-02</code> , <code>-03</code> , <code>-march=native</code> , <code>-ffp-contract=off</code> all gave the same wrong answer)
<code>sum()</code> of nothing	assuming a float	<code>sum(x for x in [])</code> is the integer <code>0</code> , and <code>round(0, 3)</code> is still <code>0</code> . That reaches the exported state as <code>0</code> rather than <code>0.0</code> — a <i>type</i> difference, invisible to any comparison that casts before comparing

Version note: `sum()` compensates from CPython 3.12. On 3.10 the committed fixtures do not reproduce bit-exactly. Measured rather than feared, by substituting a naive `sum` across every core module: **12 fields move, worst error 0.9 ULP, 0 discrete outcomes change, 0 failures under port tolerance**. Nothing behaves differently — but somebody running the conformance check on 3.10 will see twelve exact failures and deserves to find that stated.

46.2 Structural

Trap	Symptom
Iteration order	a seeded draw inside a loop over an unordered map. Python dicts preserve insertion order; most hash maps do not. Shows up once in a thousand ticks
Term vs bias	a modifier contributed as a value term is <i>averaged into its group</i> and can lower the thing it meant to raise. Somebody dressed to look dangerous made a character less likely to back away
Aliased snapshots	<code>as_dict</code> handing out live lists made later mutations rewrite the "before" picture, so a lossy round-trip looked lossless
Rounding in persistence	rounding in <code>as_dict</code> makes save/load lossy and a restored session drift from the one it was taken from
Trace overwrite	per-event assignment threw away everything earlier events in the same tick produced. A week reported <code>climate.moved: 0</code> while the layer demonstrably worked
Uninitialised loop variable	<code>belief</code> carried over from the previous observer: a <code>NameError</code> on the first iteration and a <i>wrong answer</i> on the rest
Temperature not restored	an exception during selection left the policy temperature pinned at 0.05 for the rest of the process, quietly turning every later decision near-deterministic
String vs integer keys	origins normalised to strings on save but not on load: a restored belief stopped matching integer origins and silently started treating a repeated rumour as independent evidence
Reason codes keyed on the wrong names	the climate reaction table was keyed on the verbs a player types rather than the event types the runtime emits, producing a layer that ran, reported, and never once moved

46.3 Design-level

Trap	Why it happens
Read with no writer	a field consulted by four engines that only ever holds what a pack authored. This has occurred six times (<code>reputation</code> , <code>fear</code> , <code>respect</code> , <code>appearance</code> , ...), each found by measurement. The fix is a declared list of authored-only fields, checked by test
Computed and never read	<code>fidelity</code> was calculated, written into the payload, and consumed by nothing — a documented parameter with no effect
Constructed and never called	an engine instantiated on every runtime whose output nothing consumes. Costs an object and, worse, <i>implies a feature that does not exist</i>
A cliff at zero	<code>sign(liking)</code> as an accommodation direction. A stranger at liking 0.000 got full convergence and -0.15 got full divergence
A threshold where a margin was meant	a fixed <code>0.55</code> for "the promisee believes it happened" silently required two independent confirmations
Guessing the shape of a bound	a flat repeat penalty attenuates but does not saturate. Only a geometric one bounds the total
Engine content leaking into the engine	a default <code>inform</code> template asserting a fact about the reference pack's clinic, inherited by every other world
One calculation in three places	the style vector was computed at three call sites; a change to one silently left the two a game hits most untouched

46.4 Toolchain

Compiling the native engine plugin for the first time found three defects that **no harness could have replaced**: `check` is a macro in the engine's assertion header and the validator has a method by that name; the module had exceptions off while the core refuses things by throwing; and that flag *looked* ignored until the shared precompiled header was turned off, because a shared PCH is built once per exception setting.

A separate first compile found a missing include that then produced three further errors about lambda captures with nothing wrong with them, and a JSON map whose key type changed between engine versions so an implicit conversion had to be written out. **Both had sat in code that "looked right" and had never been put through a compiler.**

47. Known weaknesses and open problems

Stated plainly, because a reader will find these anyway and finding them undisclosed is worse.

47.1 Unproven in the only way that counts

Nothing has been run inside an actual game. The C++ core reproduces the reference bit for bit across 50 of 51 modules and all five conformance scenarios; the C ABI is driven by a C99 program; the Unity binding by a real C# compiler against the real library; the Godot GDExtension by a headless Godot from GDScript; the Unreal plugin compiles and links with UnrealHeaderTool over its reflection macros. **Every one of those is a build or a harness.** A studio putting this in a scene and shipping it is the thing nobody has done, and no amount of the above substitutes for it.

One playable scene is not a shipped game. Nobody has yet found out whether it feels right over twenty hours rather than two minutes.

47.2 Model-level limits

Lexical checks are not semantic guarantees, and "provider" release depends on lexical checks. A negation-word rule reads *"...open all night, no doubt"* as a denial; a secret can be described without being named (*"behind where drinks are served"*). Controlled release is the answer, and its cost is the model's wording for factual lines. Everything below applies to the weaker mode.

Polarity is read lexically, and English negates lexically too. The released line has to agree with the polarity of what was committed, and the check looks for negation words. *"The shutters were down"* denies that a place was open without using one, so a **negated** commitment phrased that way is replaced by the pack's own deny phrasing. Correct output, model wording lost; positive commitments are unaffected. The cleverer rule — scoring the line against the topic's authored affirm and deny variants — was tried and scores that sentence as *affirmative*, because the deny variant says "the place" and the affirm one says "the clinic". A wrong answer with more machinery behind it fails in the direction that releases a contradiction, so the conservative rule stands.

Distortion acts on slots, not on prose. Levelling drops a `when` ; it does not rewrite a sentence into a vaguer sentence. With authored templates this is invisible, because the template renders the reduced proposition. **With a language model the seam can show.**

Entity grounding is a heuristic, not a parser. It uses English morphology and determiner position, not syntax. A real parser would be more precise; it would also be a dependency, and this layer has to work without one. Getting it slightly wrong costs a deflection, never a leak.

No learning and no personality change. Traits are fixed. Relationships move; who somebody *is* does not. That is deliberate — a personality that drifts under play is how every character in a long game converges on the same one — but it means characters do not get better at anything.

A standing does not heal. By design, and it is an author's decision to make, not this layer's.

A world left running for simulated years does not keep producing news. The director can escalate and pursuit keeps a town talking, but **a director that writes its own incidents — arrests, deliveries, arguments — is the larger, unbuilt thing.** For a shipped game this is fine, because play generates events. For the "living world" claim it is a real limit.

The judgement mechanism is narrow. Same predicate, same subject, three occasions. It cannot notice "*the two of them are always together*" or "*this never happens on a Tuesday*".

Theory of mind is depth 1 and barely used. `TheoryOfMindEngine` models whether a threat will be believed and whether somebody is unaware of a proposition. There is no modelling of what somebody believes *you* believe.

InterpersonalEngine has no consumer. The (agency, communion) stance it computes is correct and nothing reads it. Wiring it would mean the register engine taking a stance alongside its style vector and the realizer phrasing from it.

47.3 Parameters that are declared and unread

Stated so a reimplementer does not reproduce a parameter block and then wonder why changing it does nothing:

Parameter	Status
<code>memory.w_goal</code>	declared; the retrieval path does not read it
<code>belief.kappa_min</code> below 0.5	a source that credible-in-reverse is treated as evidence FOR the opposite. Bayes permits it, but it means "this person is a reliable liar" rather than "I doubt them", and no pack states which it means
<code>memory.interference</code>	declared; the cue-competition penalty is not applied
<code>reputation.eta_image</code> , <code>eta_rep</code> , <code>decay</code>	declared; <code>apply</code> adds the delta directly and reputation does not decay

47.4 Research directions with real upside

1. **Norms and institutions proper.** Climate is the weak, checkable version of "*culture shapes what travels*". A real norm layer would model sanction, legitimacy and enforcement as first-class objects rather than as terms.
 2. **A generative director.** Something that writes incidents into a saturated world in a way that respects who could plausibly have done what.
 3. **Distortion over prose.** Levelling a *sentence* rather than a slot, which is what would make a model-written retelling degrade convincingly.
 4. **Deeper theory of mind.** "*They think I do not know*" is the raw material of almost every interesting social scene and is currently unrepresented.
 5. **Learned competence.** A character who becomes more credible on a domain because they keep being right about it — without letting personality drift.
 6. **Belief in other people's beliefs as first-class propositions**, so that a character can be wrong about who knows what and act on it.
-
-

Appendices

Appendix A. Complete default-parameter table

Module defaults. A world pack overrides any of them through `world.json: tuning` (and diffusion/routine additionally through their own blocks).

belief

Key	Default
<code>kappa0</code>	0.55
<code>alpha</code>	0.25
<code>gamma</code>	0.20
<code>delta</code>	0.30
<code>kappa_min</code>	0.20
<code>kappa_max</code>	0.95
<code>rho_correlated</code>	0.10
<code>provenance_limit</code>	64

memory

Key	Default
<code>tau_ret</code>	-1.5
<code>noise_s</code>	0.4
<code>working_cap</code>	7
<code>w_topic</code>	1.0
<code>w_goal</code>	0.6 (unread)
<code>w_emotion</code>	0.5
<code>w_mood_congruence</code>	0.3
<code>episodic_cap</code>	200
<code>total_cap</code>	600
<code>summary_cap</code>	40
<code>presentation_window</code>	32
<code>interference</code>	0.3 (unread)
<code>time_scale</code> (activation)	60.0
<code>importance_slows_decay</code>	0.6
<code>JUDGEMENT_THRESHOLD</code>	3

Decay by type: judgement 0.28 · emotional 0.25 · commitment 0.30 · summary 0.35 · semantic 0.40 · social 0.40 · episodic 0.55 · default 0.5

affect

Key	Default
k_impulse	0.6
tau_mood	240.0 min
tau_emotion	30.0 min
epsilon	0.05

perception / interpretation

Key	Default
base_quality	0.95
jitter	±0.05
attention_floor	0.18
familiarity_weight	0.35
role_weight	0.40
recognition_slack	0.15

distortion

Key	Default
levelling weight	0.55
sharpening weight	0.20
assimilation weight	0.20
inversion weight	0.05
SHARPEN_FACTOR	1.5
familiar candidates kept	3

diffusion

Key	Default
encounters_per_hour	0.35
max_encounters_per_tick	48
min_confidence_to_tell	0.60
min_tie	0.05
fidelity_decay	0.80
distortion_prob	0.18
distortion_weights	null
stifle_prob	1.0
include_player	false
max_hops	12
bystanders	3
recency scale	240 min
top-N topic pool	3

topology

Key	Default
transmissibility: public / common / local / trade / restricted / secret	1.00 / 0.85 / 0.60 / 0.45 / 0.20 / 0.05
default class	common
shared-circle multiplier	×1.35 (capped at 1.0)
domain interest	×(0.4 + 0.6·min(1, 2·competence))
attention budget	max(1, round(6·(0.5 + curiosity)))

medium / notes

Key	Default
in_person fidelity / trust	1.00 / 1.00, audience yes
phone fidelity / trust	0.72 / 0.88, audience no
note fidelity / trust	0.88 / 0.78, audience no
CONTACT_FAMILIARITY	0.45
notes.max_per_place	12
notes.lifetime_minutes	7,200
notes.intercept_base	0.02 / hour
note write-time distortion	0.18 × 1.12 = 0.2016

climate

Key	Default
TEMPO_SWING	0.45
THRESHOLD_SWING	0.12
SKEPTICISM_SWING	0.18
STRANGER_SWING	0.15
HALF_LIFE_HOURS	30.0
CARRIED_PER_ARRIVAL	0.055
MAX_CARRIED_PER_STEP	0.22

contagion

Key	Default
transfer	0.14
min_tie	0.08
max_step	0.22
stability damping	$\times(1 - 0.6 \cdot \text{stability})$
liking weight in tie	0.35

relationship / reputation / identity

Key	Default
relationship.eta_up	0.10
relationship.eta_down	0.40
relationship.eta_other	0.20
default trust	0.3
reputation.eta_image	0.25 (unread)
reputation.eta_rep	0.10 (unread)
reputation.decay	0.001 (unread)
identity.beta	4.0
identity.neutral_fit	0.4
identity.inertia	0.5
identity.w_acc/w_comp/w_norm/w_threat	1.0
identity-threat half-life	24 h
FAMILIARITY_PER_HOUR	0.01

standing / appearance

Key	Default
BASE_STEP	0.5
STRANGER_FLOOR	0.6
MAX_STEP	0.35
FEAR_SHARE	0.8
RESPECT_SHARE	0.5
DECISION_PULL	0.4
trust share of a cruelty	0.6
warmth weights (liking / trust / decent / attraction)	0.30 / 0.20 / 0.50 / 0.25
influence weights (respect / dependence / fear / reach)	0.35 / 0.30 / 0.25 / 0.10
THREAT_PULL	0.35
STATUS_PULL	0.15
drawn_to saturation	$\min(1, 0.55 \cdot \sqrt{\text{hits}})$

sociolinguistics

Key	Default
w_edu	0.4
w_setting	0.3
w_status	0.2
w_role	0.2
w_arousal	0.4
w_ingroup	0.5
lambda_accommodation	0.4
base_sentence_len	8.0
k_len	12.0
k_stress	6.0
default place formality	0.4
regard: formality +=	$0.25 \cdot \text{fear} + 0.15 \cdot \text{respect}$
regard: directness -=	$0.30 \cdot \text{fear}$
regard: hedging +=	$0.30 \cdot \text{fear} + 0.10 \cdot \text{respect}$

policy / dialogue

Key	Default
temperature	0.3
use_prospect	false
w_impulsivity	0.4
w_arousal	0.3
prospect α , β , λ , γ	0.88, 0.88, 2.25, 0.61
reactive-rule temperature	0.05
C_ASSERT (min confidence to state as fact)	0.6
certainty bands	certain ≥ 0.85 , probable ≥ 0.65
commitment tie band	0.05 confidence
max_sentences	2
plan max_length	30
anti-repeat window	5 lines
validator repetition window	3
EARSHOT	3

pursuit / promises / common knowledge / director

Key	Default
pursuit.interval_minutes	180.0
pursuit.min_confidence	0.55
pursuit.max_per_tick	12
pursuit.worth_a_call	0.12
pursuit.worth_writing	0.20
promises.horizon_minutes	2,880
promises.kept_margin	0.002
promises.max_open_per_agent	24
common_knowledge.min_witnesses	3
common_knowledge.public_privacy	0.35
common_knowledge.min_quality	0.5
MAX_PUBLICS_PER_KEY	6
director.heat_decay	0.02 / hour
director.heat_threshold	0.6
director.clock_base_rate	0.15
director.mobilize_latency	20 min
clock size	6
heat discharge on mobilise	0.4

runtime / routine / revision

Key	Default
routine.emit_events	true
routine.arrival_importance	0.15
MAX_TRACE_ENTRIES_PER_KEY	400
revision.FLOOR	0.1
default discredit factor	0.25
action_timeout_minutes	60
latency_budget_ms	120

Appendix B. Reason-code catalogue

Every code a reimplementation should emit, grouped by producer. Each entry is (code, magnitude, detail).

belief / revision

belief.updated · belief.contradiction · belief.source_credence · revision.recomputed

perception / interpretation / attention

attention.about_me · attention.familiar_subject · attention.professional_interest ·
attention.curiosity · attention.exhausted · interpretation.attended · interpretation.unnoticed ·
interpretation.recognised · interpretation.not_recognised · interpretation.misread ·
interpretation.told_in_words

memory

memory.encoded · memory.consolidated · memory.judgement_formed · memory.judgement_reinforced

affect / contagion

affect.emotion_fired · affect.mood_shift · contagion.caught

diffusion / topology

diffusion.told · diffusion.distorted · diffusion.stifled · diffusion.no_tie ·
diffusion.not_worth_telling · diffusion.tick_capped · topology.transmissibility ·
topology.outside_the_circle · topology.wrong_place

relationships / standing / appearance / identity

relationship.applied · reputation.applied · standing.judged · appearance.drawn_to ·
identity.threatened · tom.updated

decision / agency / director

policy.selected · agency.reactive_fired · agency.rule_failed · agency.ally_response ·
agency.no_allies · agency.risk_reference · director.clock · director.clock_complete ·
director.heat · director.mobilize · director.arrived

dialogue / commitment / validator

commitment.selected · commitment.seeded_choice · commitment.unsayable_dropped ·
dialogue.direct_answer · dialogue.deception · dialogue.secret_gate · dialogue.topic_repeated ·
dialogue.validation_fallback · dialogue.reworded · dialogue.released_anyway ·
dialogue.wording_replaced · dialogue.commitment_unexpressed · dialogue.spoken · validator.accept
· validator.secret_leak · validator.secret_referent · validator.canon_violation ·
validator.canon_warning · validator.repetition · provider.failure · provider.late_line_rejected

world layers

routine.placed · routine.moved · routine.move_requested · pursuit.acted · note.left · note.read ·
note.went_unread · promise.made · promise.kept · promise.broken · climate.moved · climate.carried
· common_knowledge.established · common_knowledge.not_news · common_knowledge.secret_spent ·
action.resolved · action.timed_out · state.world_changed · trace.truncated · tuning.applied

snapshot

snapshot.version_mismatch · snapshot.seed_mismatch · snapshot.unknown_agent ·
snapshot.unknown_faction · snapshot.unknown_place · snapshot.agent_not_in_snapshot

parser (reference client)

parser.ask · parser.claim · parser.promise · parser.threat · parser.attack · parser.assist ·
parser.accuse · parser.observe · parser.inspect · parser.media · parser.time · parser.wear ·
parser.system · parser.fallback_say · parser.unknown · parser.empty

Appendix C. Glossary

Term	Meaning
anchor	a word a listener would have to hear for a claim to have been made; used by the grounding read-back
assertion strength	how forcefully a claim is asserted; multiplies effective trust. Fidelity along a chain $\times$ medium trust factor $\times$ spoken certainty
avoid label	the <i>name</i> of a secret, passed to the text layer instead of the secret
bias	a push on an action's utility, added rather than averaged into a group
bridge	a character standing in more than one circle; the route by which anything crosses between groups
circle	a group: a network, role, role context, or identity group
commitment	the complete semantic content of one utterance, chosen by the runtime before any words exist
conflict	$2 \cdot \min(SF, SA) / (SF + SA)$ — how far the evidence for and against a belief cancels
core key	the canonical identity a belief is filed under: predicate + ordered slots + qualifiers, without polarity
correlation discount (η)	how much <i>new</i> evidence an assertion carries, given how often its origin has already been heard
credibility (κ)	a source's reliability, in $(0, 1)$; $\ln(\kappa / (1 - \kappa))$ is the log-evidence of one hearing
fidelity decay	conviction lost per retelling hop
hops	shortest known number of retellings from a first-hand source
ignorance	$1 / (1 + SF + SA)$ — how little evidence exists either way
levelling / sharpening / assimilation	Allport & Postman's three rumour processes, given structural meaning over slots
line upgrade	a model-written line that missed the latency budget, arriving late, re-validated, matched by <code>request_id</code>
origin	the token identifying where a claim ultimately came from. It travels with the claim, not with the teller
projection	a materialised read model of an agent's state; the ledger is authoritative
public	one crowd, one fact, one moment at which it stopped being deniable
reason trace	<code>(code, magnitude, detail)</code> — why a stage did what it did
regard	fear and respect, passed to the register engine only when the standing layer is on
semantic move	one thing a character commits to doing with a sentence
spreading	whether a holder is still actively passing a belief on (Daley–Kendall stifling)
standing	what the town thinks of you because of what it saw or was told you did
stifling	a spreader who tells somebody who already knows loses interest
term	an opinion about an outcome, averaged within its value group
transmissibility	how freely a class of claim travels once somebody holds it
warmth	a summary of liking, centred trust, believed decency and attraction, in $[-1, 1]$
world pack	a directory of declarative JSON that is a world

Appendix D. Bibliography

The work each mechanism draws on. Where the runtime departs from a source, §3 says how.

- **Allport, G. W. & Postman, L.** (1947). *The Psychology of Rumor*. — levelling, sharpening, assimilation.
 - **Anderson, J. R.** ACT-R. — base-level activation, retrieval threshold, spreading activation.
 - **Aumann, R. J.** (1976). Agreeing to Disagree. — common knowledge, formally.
 - **Bartlett, F. C.** (1932). *Remembering*. — serial reproduction.
 - **Bell, A.** Audience design. — style as a function of who is listening.
 - **Brown, P. & Levinson, S.** *Politeness*. — bald / positive / negative / off-record strategies.
 - **Chwe, M. S.-Y.** (2001). *Rational Ritual*. — public events as the *mechanism* of common knowledge.
 - **Daft, R. L. & Lengel, R. H.** (1986). Media richness. — leaner media lose more.
 - **Daley, D. J. & Kendall, D. G.** (1964). Epidemics and rumours. — stifling as the termination mechanism.
 - **Frijda, N. H.** *The Emotions*. — action tendencies.
 - **Giles, H.** Communication Accommodation Theory. — convergence and divergence.
 - **Goffman, E.** *The Presentation of Self in Everyday Life*. — front stage / back stage.
 - **Granovetter, M.** The Strength of Weak Ties. — bridges and reach.
 - **Hatfield, E., Cacioppo, J. T. & Rapson, R. L.** (1994). *Emotional Contagion*.
 - **Kahneman, D. & Tversky, A.** Prospect Theory. — value and weighting functions.
 - **Lewis, D.** (1969). *Convention*. — common knowledge, defined.
 - **Mehrabian, A.** PAD temperament model; **ALMA** (Gebhard). — personality → mood baseline, emotion → PAD.
 - **Ortony, A., Clore, G. L. & Collins, A.** (1988). *The Cognitive Structure of Emotions* (OCC). — appraisal → discrete emotions.
 - **Schwartz, S. H.** Theory of basic human values.
 - **Tajfel, H. & Turner, J. C.** Social identity theory. — salience, in-group threat.
 - *Blades in the Dark* (Harper). — progress clocks, heat, factions acting off-screen.
-

Reproducing every number in this document

```
# a playable world in one command
python3 -m lwr play --world-pack worldpacks/relay-station

# the invariants, as numbers
python3 -m lwr benchmark

# a fully recorded session; the dossier is meant to be read
python3 -m lwr evidence --hours 2

# the whole suite: 202 checks, no dependencies
python3 tests/test_scenarios.py

# what one turn and one hour of world actually cost, in both languages
python3 tools/frame_bench.py

# the fixtures a second implementation must reproduce
python3 -m lwr conformance --check --out conformance/

# the shortest player sequence that breaks a stated rule
python3 -m lwr qa qa/relay-station.json --world-pack worldpacks/relay-station

# every tunable knob, with what each one does
python3 -m lwr tune

# the live layer map, checked against the real import graph
python3 -m lwr architecture
```

End of specification.